CS336: 从零构建语言模型 — 全课程中文学习笔记

斯坦福大学 CS336（Spring 2026 开设；并参考 Spring 2024/2025 存档）。授课教师：Percy Liang、Tatsunori Hashimoto。本笔记整理自公开课程网站（cs336.stanford.edu）、可执行讲义文件（`lecture_XX.py`）、讲义幻灯片 PDF 以及五次作业的 handout。

课程总览

这门课讲什么？

CS336 是一门**项目驱动（project-based）**的课程，带你走完"从零训练一个语言模型"的完整流程：预训练数据采集与清洗 → Transformer 模型构建 → 模型训练 → 部署前评测。它的灵感直接来自操作系统课——那类课程要求你从零写出一个完整的操作系统，而 CS336 要求你从零写出一个完整的语言模型。课程**几乎没有脚手架代码（minimal scaffolding）**，你要写的代码量比一般 AI 课程高一个数量级。

核心哲学：通过构建来理解（understanding via building）。 课程认为研究者正在与自己使用的技术脱节：2016 年研究者自己实现并训练模型；2018 年下载 BERT 做微调；今天则直接对 API 模型（GPT/Claude/Gemini）写 prompt。往上走一层抽象确实提升生产力，但这些抽象是**有漏洞的（leaky）**——与编程语言或操作系统不同——而且仍然有大量基础研究需要"撕开整个软件栈"。因此，完全理解这门技术是做出基础研究的必要条件。

效率思维（贯穿全课的主线）。 全课的统一视角是：

> **accuracy = efficiency × resources（效果 = 效率 × 资源）**

以及那个贯穿始终的问题：*在给定的算力与数据预算下，我能造出的最好模型是什么？* 今天我们是**算力受限（compute-constrained）**的，所以所有设计决策都体现"把给定硬件的性能榨干"：分词为了缩短序列、模型架构为了省显存/省 FLOPs、数据过滤为了不浪费算力、扩展律为了省下调参算力。课程还预判："明天，我们将变成数据受限（data-constrained）。"

课程能传授的三类知识： – **Mechanics（机制）**：事物如何运作（Transformer 是什么、模型并行怎么做）——可以迁移。 – **Mindset（心态）**：榨干硬件、认真对待扩展（scaling）——可以迁移。 – **Intuitions（直觉）**：哪些数据/建模决策能带来更好的精度——**不一定**跨规模迁移（很多只是经验性的"民间智慧"，例如提出 SwiGLU 的那篇论文里就画了一张"神恩浩荡/divine benevolence"的图来自嘲）。

先修要求（prerequisites）： – **Python 与软件工程熟练**（最重要——几乎无脚手架、代码量极大）。 – **深度学习与系统优化经验**（熟练 PyTorch，了解内存层次结构等基础系统概念）。 – **大学微积分与线性代数**（MATH 51 / CME 100 水平）。 – **基础概率统计**（CS 109 或等价课程）。 – **机器学习**（CS221/CS229/CS230/CS124/CS224N 水平）。

课程安排与规则（Spring 2026）： 每周一/周三 15:00–16:20，地点 Skilling Auditorium；5 学分；录播在 YouTube 播放列表（**非公开**，本笔记标注为受限资源）。所有作业通过 Gradescope 提交；共 6 个 late days，单个作业最多用 3 个。Modal 为选课学生赞助 GPU 算力。

荣誉准则（honor code）与 AI 政策要点： – 允许学习小组，但必须自己理解并独立完成作业，一人一份提交。 – **AI 政策（2025–26）**：允许用 AI 回答**高层概念问题**与**底层 API/文档问题**，但**不允许用 AI 实现作业的任何部分**——包括编码智能体（Cursor Agents、Codex、Claude Code）和 AI 自动补全（Cursor Tab、GitHub Copilot）。每个作业仓库都带有一个带教学提示的 `AGENTS.md`，编码智能体会自动读取并遵守；使用网页聊天界面时，需把 `AGENTS.md` 内容粘贴到每次对话开头。判断标准："**问自己：如果我在 office hour 提出这个请求，助教会照做吗？**" – **不得参考现有实现**（handout 是自包含的）——课程明确要求不要去看网上别人写的实现。 – 对评分有异议须在成绩公布后 3 天内在 Gradescope 提交 regrade。

五次作业（详见"作业汇总"一节）： 1. **Basics（基础）**：分词器、Transformer、交叉熵损失、AdamW、训练循环；在 TinyStories 与 OpenWebText 上训练；有排行榜。 2. **Systems（系统）**：性能基准测试与 profiling、激活重计算、Triton 实现 FlashAttention–2、DDP、优化器状态分片、FSDP。 3. **Scaling（扩展律）**：通过调用训练 API 拟合扩展律，预测算力最优超参数。 4. **Data（数据）**：Common Crawl HTML 转文本、过滤（质量/有害内容/PII）、MinHash 去重。 5. **Alignment & Reasoning RL（对齐与推理强化学习）**：zero/few–shot 与 CoT prompting、GRPO、策略梯度估计器变体；可选第二部分：SFT + DPO。

如何使用这份笔记。 每一讲都是一个"章节"，结构统一：**概览 → 核心概念与定义（含生活化类比）→ 代码示例与详细评注（代码做了什么 / 实现深挖 / 与作业的联系）→ 关键要点 → 常见陷阱 → 复习题（含简答）**。代码密集型讲座（1、2、6、7、10、14）会**反向拆解真实讲义代码**；幻灯片型讲座（3、4、5、8、9、11、15、16）则提炼幻灯片要点。

第 1 讲：课程总览与分词（Overview, Tokenization）

日期：3 月 30 日（周一，Spring 2026）| 讲师：Percy Liang | 材料：`lecture_01.py`（可执行讲义 + 代码）

概览

本讲先回答"这门课为什么存在"，梳理语言模型（LM）的历史脉络，然后进入第一个技术单元：**分词（tokenization）**——把原始文本（字节）转换成模型真正消费的整数序列的过程。讲义现场用 Python 实现并对比了字符级、字节级、词级以及 **BPE（Byte–Pair Encoding，字节对编码）** 分词器，其中包括一个从零写的 BPE 训练器。

核心概念与定义

- **语言模型（language model）**：一个定义在 token 序列上的概率分布。2018 年"语言模型"是拿来微调的东西（BERT）；2020 年是拿来 prompt 的东西（GPT–3）；2022 年是拿来对话的东西（ChatGPT）；2026 年是可以自主行动的东西（agents）。底层原理（attention、kernel、优化）没变，但规格变了（更长的上下文、推理效率更关键）。
- **分词器（Tokenizer）**：一个提供两个方法的类——`encode(string) -> list[int]` 与 `decode(list[int]) -> string`。它是"原始输入（字节）"与"模型操作的整数"之间的接口。

- *类比*：就像厨师备菜。你不能拿一整颗没洗的蔬菜下锅，得洗净、削皮、切成大小均匀的块（token），你的菜谱（模型）才好处理。不同的厨师（分词器）切法不同，切法直接决定菜好不好做。
- **Unicode / 码点（code point）**：原始文本是 Unicode 字符序列；每个字符对应一个码点（例如 `ord("a") == 97`，`ord("🌍") == 127757`），用 `chr` 可以转回去。
- *类比*：码点就像 ISBN 书号——人类所有文字符号（书籍）都分到了一个全局唯一的编号，无论它属于哪种语言。
- **UTF-8**：主流编码方式，把字符映射为 1–4 个字节。ASCII 字符占 1 字节，`🌍` 占 4 字节（`\xf0\x9f\x8c\x8d`）。
- **压缩率（compression ratio）**：每个 token 对应的 UTF-8 字节数。压缩率越高，序列越短——这一点非常重要，因为 Transformer 的 attention 复杂度对序列长度是**平方级**的。
- *类比*：压缩率就像打包行李。一件行李箱装 10 件衣服（高压缩率）远好过 10 个小包（低压缩率），因为航空公司（attention）是按"件数"（token 数）收费的。
- **BPE（字节对编码）**：一种**数据驱动**的子词（subword）算法（1994 年 Philip Gage 为数据压缩提出，Sennrich 等人 2016 年引入 NLP，之后被 GPT-2 采用并成为现代模型的事实标准）。做法：以字节为初始 token，反复把**出现频率最高的相邻 token 对**合并成新 token，直到达到目标词表大小。
- *机制*：`统计相邻对 → 合并最高频对 → 重复`。结果是：常见的字节序列被压缩成单个 token，罕见的序列则被拆成很多 token。
- *类比*：BPE 就像手机输入法的联想词——"brb"、"lol"、"omw" 因为在语料里高频出现而变成快捷词，而像 "supercalifragilisticexpialidocious" 这种罕见词只能一个字母一个字母打出来。
- **分词效率视角（本讲的核心论证）**：
- 缩短上下文长度（约 1000 字节 → 约 250 个 token）；
- 自适应算力分配（把更多模型容量给输入中"更有信息量"的部分）。
- **无分词器架构（the dream）**：ByteT5、MegaByte、BLT 等模型直接在字节上操作——很有前景，但尚未扩展到前沿规模。

代码示例：`Tokenizer` 接口与三种朴素分词器

讲义定义了抽象接口，并给出三种具体（但次优）的分词器。

代码（Python）：

```
from abc import ABC

class Tokenizer(ABC):
    """分词器的抽象接口。"""
    def encode(self, string: str) -> list[int]:
        raise NotImplementedError
    def decode(self, indices: list[int]) -> str:
        raise NotImplementedError

class CharacterTokenizer(Tokenizer):
    """把一个字符串表示成 Unicode 码点序列。"""
    def encode(self, string: str) -> list[int]:
        return list(map(ord, string))
    def decode(self, indices: list[int]) -> str:
        return "".join(map(chr, indices))

class ByteTokenizer(Tokenizer):
    """把一个字符串表示成字节序列。"""
    def encode(self, string: str) -> list[int]:
        string_bytes = string.encode("utf-8")
        indices = list(map(int, string_bytes))
        return indices
    def decode(self, indices: list[int]) -> str:
        string_bytes = bytes(indices)
        string = string_bytes.decode("utf-8")
        return string
```

代码做了什么： 1. `CharacterTokenizer.encode` 用 `ord` 把每个 Unicode 字符映射成码点整数（一个字符一个整数）；`decode` 用 `chr` 转回。 2. `ByteTokenizer.encode` 先把整个字符串编码为 UTF-8 字节，再把每个字节（0–255）转成整数；`decode` 把整数转回字节并做 UTF-8 解码。 3. 两者都能往返：`decode(encode(s)) == s`。

实现深挖： – **为什么说这两种是"两头不讨好"**：字符分词器的词表巨大（约 15 万个 Unicode 字符，绝大多数极罕见），压缩率却约等于 1；字节分词器词表很小（256），但压缩率**恰好等于 1**（一个字节一个 token），序列很长，直接引爆 attention 开销。讲义用 `"Hello, 🌍! 你好!"` 现场演示了字节分词器的压缩率恰好为 1.0。 – **为什么必须正确处理 UTF-8**：并非所有字符都能用一个字节表示，`bytes("🌍", encoding="utf-8") == b"\xf0\x9f\x8c\x8d"`。能否正确处理这一点，是"能用的分词器"和"坏掉的分词器"的分界线。 – **为什么 decode 要容忍非法字节**：`bytes.decode("utf-8")` 遇到非法序列会抛异常；生产级分词器用 `errors="replace"`（讲义中的 `output_tokenizer` 就是这么做的）来容忍任意字节串。

与作业的联系：作业 1 要求实现完整的 BPE 分词器。这里的 `Tokenizer` 抽象类正是你的 `BPETokenizer` 必须满足的接口，而 UTF-8 处理是你将要写的字节级 BPE（加上预分词、special token、高速合并）的基础。

代码示例：BPE 的 merge、训练与分词器

代码（Python）：

```
def merge(indices: list[int], pair: tuple[int, int], new_index: int) -> list[int]:
    """返回 indices，但把所有 pair 的实例替换成 new_index。"""
    new_indices = []
    i = 0
    while i < len(indices):
        if i + 1 < len(indices) and indices[i] == pair[0] and indices[i + 1] == pair[1]:
            new_indices.append(new_index)
            i += 2
        else:
            new_indices.append(indices[i])
            i += 1
    return new_indices

def count_adjacent_pairs(indices: list[int]) -> dict[tuple[int, int], int]:
    """返回字典：每个相邻 token 对 -> 出现次数。"""
    counts = defaultdict(int)
    for index1, index2 in zip(indices, indices[1:]):
        counts[(index1, index2)] += 1
    return counts

def train_bpe(string: str, num_merges: int) -> BPETokenizerParams:
    indices = list(map(int, string.encode("utf-8")))
    merges: dict[tuple[int, int], int] = {}          # 对 -> 合并后的新下标
    vocab: dict[int, bytes] = {x: bytes([x]) for x in range(256)}  # 下标 -> 字节

    for i in range(num_merges):
        counts = count_adjacent_pairs(indices)       # 统计所有相邻对
        pair = max(counts, key=counts.get)           # 取最高频的 pair
        new_index = 256 + i                          # 分配新下标
        merges[pair] = new_index
        vocab[new_index] = vocab[pair[0]] + vocab[pair[1]]  # 字节串拼接
        indices = merge(indices, pair, new_index)
    return BPETokenizerParams(vocab=vocab, merges=merges)

@dataclass(frozen=True)
class BPETokenizerParams:
    vocab: dict[int, bytes]                # 下标 -> 字节
    merges: dict[tuple[int, int], int]     # (i1, i2) -> new_index

class BPETokenizer(Tokenizer):
    def __init__(self, params: BPETokenizerParams):
        self.params = params
    def encode(self, string: str) -> list[int]:
        indices = list(map(int, string.encode("utf-8")))
        # 注意：这是一个非常慢的实现
        for pair, new_index in self.params.merges.items():
            indices = merge(indices, pair, new_index)
        return indices
    def decode(self, indices: list[int]) -> str:
        bytes_list = list(map(self.params.vocab.get, indices))
        return b"".join(bytes_list).decode("utf-8")
```

代码做了什么： 1. `train_bpe` 先取训练字符串的字节序列，然后循环 `num_merges` 次：统计所有相邻对 → 取最高频者 → 分配全新下标（`256 + i`）→ 记录合并规则 → 在词表里把两段字节串拼接 → 用 `merge` 重写序列。 2. `BPETokenizer.encode` 把学到的合并规则**按顺序**应用到新输入的字节序列上。 3. `BPETokenizer.decode` 查词表拿到每个下标的字节串，拼接后做 UTF-8 解码。

实现深挖： – **为什么是 `256 + i`**：前 256 个下标留给单字节；每个新合并 token 拿下一个可用下标，保证映射是单射。 – **为什么必须按训练顺序应用 merge**：BPE 是一个贪心的层次化过程；后学的合并可能包含先学的合并结果，只有按学习顺序应用，编码结果才与训练时的语义一致。 – **为什么 decode 不需要 merge 规则**：词表为每个 token 存了完整的字节串，所以解码是纯查表；merge 规则只在编码时用到。这形成了清晰的"训练期数据结构（merges）"与"推理期数据结构（vocab）"分离。 – **复杂度问题（代码里明确标注）**：`encode` 对**所有** merge 都遍历一遍序列，复杂度约 O(num_merges × 序列长度)。作业 1 要求你只应用"真正相关的" merge（例如序列里已不存在的对直接跳过）、支持 special token（如 `<|endoftext|>`）、加入 GPT-2 风格的预分词 regex，并把整体做快。 – **为什么需要预分词与词尾标记**：经典 Sennrich 版 BPE（以及你给的示例）会先按词切分并用 `</w>` 标记词尾，防止跨词合并；讲义这里的字节级版本则直接合并原始字节，更简单，也和 GPT 系分词器一致。

与作业的联系：这是**作业 1 第 2 节（BPE 分词器）**的绝对基础。你要扩展的正是这套逻辑：在大语料上学习 merge、构建含 special token 的词表、做预分词（GPT-2 regex）、实现高效编码器。讲义明确列出了作业 1 要求的四项升级：(1) 只遍历有意义的 merge；(2) special token；(3) 预分词；(4) 提速。

关键要点

1. 分词是 LM 流水线的关键第一步：它决定词表大小、序列长度以及罕见词的处理方式，并且必须完美往返（`decode(encode(s)) == s`）。
2. 字符/字节/词三种分词器都次优：字符词表巨大且压缩率低；字节词表小但压缩率恰好为 1；词级压缩率好但词表无界且存在未登录词（UNK）问题。
3. BPE 是简单、数据驱动、被广泛使用的启发式算法：从字节出发，反复合并最高频相邻对，从而在词表大小与压缩率之间取得平衡。
4. 本课的一切都是关于**效率**：分词之所以重要，是因为它缩短序列（attention 是平方复杂度！）并能自适应分配模型容量。
5. 一个好的分词方案应当：① 让模型在"有意义的块（chunks）"上操作；② 让块是可变的，从而把更多容量分配给输入中有信息量的部分。

常见陷阱

- **没有正确处理 Unicode**：多字节字符必须以 UTF-8 字节形式编解码；忽略它会让非 ASCII 文本无法往返。
- **不做往返测试**：永远要断言 `decode(encode(s)) == s`，并覆盖 emoji、CJK、特殊空白等边界情况。
- **合并顺序错乱**：编码必须按训练顺序应用 merge，否则结果与学到的词表不匹配。
- **编码复杂度接近 O(n^2)**：朴素的 `encode`（对所有 merge 遍历整条序列）在真实语料上慢到不可用；作业 1 期望更聪明的做法。

- **字节解码失败**：token 序列未必构成合法 UTF-8；生产环境要用 `errors="replace"`。
- **忽视压缩率**：压缩率 1.0 意味着序列很长，而 attention 是平方复杂度——序列长度是第一位的成本。

复习题

1. **问：** 为什么字节分词器的压缩率恰好是 1？这为什么是问题？
2. **答：** 每个字节恰好映射成一个 token，所以"字节/token" = 1。这意味着 1000 字节的文档变成 1000 个 token；由于 attention 对序列长度是平方复杂度，模型开销会急剧膨胀——你希望大约 4 倍压缩（约 250 个 token）。
3. **问：** 在 `train_bpe` 里，为什么 `decode` 用词表实现而不是用 merge 规则？
4. **答：** 词表把每个 token 下标映射到它代表的完整字节串，所以解码就是直接查表；merge 规则描述的是"token 是如何被构造出来的"，只有编码新文本时才需要。此外，用词表解码能处理任意 token 序列，无需知道合并历史。
5. **问：** 如果 BPE 从不合并（词尾标记，下一个词的首字符）这样的跨词对——即在原始字节上做合并而不提供任何词边界信息——会发生什么？
6. **答：** 分词器可能学到跨越词边界的 token，使 token 的语义性变差，甚至把不同上下文混在一起；预分词加边界标记就是用来防止这种情况的。

第 2 讲：PyTorch 与资源核算（FLOPs、显存、算术强度）

日期：4 月 1 日（周三，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_02.py`

概览

这是一讲"系统思维"课：在你谈"如何在固定资源下训练最好的模型"之前，必须先学会**核算（accounting）**——一次计算消耗的显存与算力。本讲覆盖 PyTorch 张量基础（数据类型、显存占用）、用 `einops` 写出可读的张量运算、FLOPs 计数，以及最关键的**算术强度 / roofline 分析**（你是 compute-bound 还是 memory-bound？），最后把核算应用到训练循环、梯度累积与激活重计算上。

核心概念与定义

- **张量（Tensor）**：深度学习里存储一切的基本单元——数据、参数、梯度、优化器状态、激活值。它有**秩（rank，即维度数）**；Transformer 里常见秩为 4 的张量，形状为 (B=32, S=16, H=16, D=64)（batch、序列位置、head、每头维度）。
- *类比*：张量就像一张固定轴数的电子表格；秩 4 的张量像一座按"批次、位置、注意力头、特征"四个方向组织的仓库，每层货架上都是一张表格。
- **精度类型（dtype）与权衡**：fp32（4 字节，默认）、fp16（2 字节，但动态范围小——`torch.tensor([1e-8], dtype=torch.float16)` 会**下溢为 0**！）、**bf16**（2 字节，动态范围与 fp32 相同但分辨率更差——1e-8 不会下溢）、fp8（E4M3/E5M2，H100 支持）、fp4（NVFP4，仅 4 比特，Blackwell）。

- *类比*：fp16 像一把只标厘米的尺子——便宜，但量不出一根头发的粗细；bf16 像一把量程极大、但刻度略粗的尺子；fp32 是精密千分尺。用 fp16/bf16 训练时，小数值塌缩成 0 会造成"训练不稳定"。
- **混合精度训练（mixed precision）**：参数/激活/梯度用 bf16，优化器状态用 fp32（因为要在很多步上累积，需要精度）。PyTorch 的 `torch.amp.autocast` 会自动处理。
- **FLOPs 与 FLOP/s**（发音相同、极易混淆）：FLOPs = 浮点运算次数（衡量做了多少"工作量"，例如 GPT-3 约 3.14e23）；FLOP/s = 每秒浮点运算次数（衡量硬件速度）。
- **MFU（Model FLOPs Utilization，模型浮点利用率）**：实际 FLOP/s ÷ 峰值（承诺）FLOP/s。≥0.5 就算相当不错；它永远达不到 1，因为受限于显存带宽、kernel 效率等。
- **算术强度（arithmetic intensity）**：某次计算的 FLOPs ÷ 搬运的字节数。**加速器强度（accelerator intensity）**：硬件峰值 FLOP/s ÷ 显存带宽（H100 约 295 FLOP/byte）。若负载的算术强度 < 加速器强度 → **memory-bound（访存受限）**；若大于 → **compute-bound（计算受限）**。
- *类比*：工厂（计算单元）由一条传送带（显存带宽）供料。带子送料不够快，工厂就闲置——你是"带子受限"（memory-bound）；带子送料过剩而工厂本身慢，则是"工厂受限"（compute-bound）。
- **6ND 法则**：训练一个 N 参数模型、喂 D 个 token，总计算量约 6·N·D FLOPs（前向 2ND，反向 4ND）。**反向传播是前向的 2 倍。**
- **Roofline 模型**：把算术强度（横轴）与达到的 FLOP/s（纵轴）画在一起；拐点就是加速器强度；MFU = min(1, 算术强度 / 加速器强度)。
- **梯度累积（gradient accumulation）**：为了用"大逻辑 batch"而不承担大 batch 的显存，在多个 micro-batch 上分别算梯度并累加（不清零），最后统一更新一次。
- **激活重计算（activation checkpointing / gradient checkpointing / rematerialization）**：只保存部分层的激活，其余在反向时**重新计算**。显存-算力权衡：每层都存是 O(L) 显存、零重算；完全不存是 O(1) 显存但 $O(L^2)$ 重算；每隔 $\sqrt{L}$ 层存一次则 $O(\sqrt{L})$ 显存、O(L) 重算。

代码示例：用 einops 写出可读的张量数学

代码（Python）：

```
from einops import rearrange, einsum, reduce
import torch

x = torch.ones(2, 3, 4)  # batch seq hidden
y = torch.ones(2, 3, 4)  # batch seq hidden

# 传统写法（很容易把 -2、-1 搞混）：
z = x @ y.transpose(-2, -1)  # batch seq seq

# einops 写法：
z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")

# 用 '...' 表示对任意个前导维度做广播：
z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")

# reduce：对最后一个维度求和
y_sum = reduce(x, "... hidden -> ...", "sum")

# rearrange：把被压平的维度拆成（heads, hidden1）
w = torch.ones(4, 4)
x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
```

代码做了什么： 1. `einsum` 是"带维度命名"的广义矩阵乘法：同时在两个操作数中出现、但不出现在输出里的维度会被**求和消去（contracted）**。第一个例子按 batch 元素计算 $x @ y^T$（正是 attention 打分矩阵的模式）。 2. `reduce` 对命名维度做聚合（sum/mean/max/min）。 3. `rearrange` 只改变形状、不改变数据，包括拆分/合并带括号的维度（`(heads hidden1)`）。

实现深挖： – **为什么要给维度起名**：`x @ y.transpose(-2, -1)` 语义不透明——到底哪个维度被消去？einops 把"收缩模式"明确写进字符串，并在运行时检查形状错误，而讲义认为手写 transpose 极易出错。生产代码中，普通 `torch.matmul` / `torch.bmm` 通常比 einops 的 einsum 更快；所以实践建议是"原型期用 einops 保证清晰"，而作业 1 的 handout 正是用这种 einops 风格来表述 Transformer 前向计算的。 – **为什么要有 `...`（省略号）**：它让同一个表达式既能处理带 batch 的情况、也能处理不带 batch 的情况，对任意个前导维度广播——写"维度无关"的层时非常方便。

与作业的联系：作业 1 的 Transformer 前向计算正是用这套记号表述的（handout 里 attention 与 RMSNorm 都给出了 einops 写法）。作业 2 的 FlashAttention kernel 也需要以命名维度（B、H、S、D）的思维来做 tiling。

代码示例：线性层与训练步的 FLOPs 计数

代码（Python）：

```
B, D, K = 1024, 256, 64   # batch、输入维度、输出维度
x = torch.ones(B, D)
w = torch.randn(D, K)
y = x @ w

# 每个 (i, j, k) 三元组对应一次乘法 + 一次加法:
actual_num_flops = 2 * B * D * K

# 两层 MLP:
# 前向:  h1 = x @ w1            -> 2*B*D*D FLOPs
#         h2 = h1 @ w2           -> 2*B*D*D FLOPs
# 反向:  h1.grad = h2.grad @ w2^T  -> 2*B*D*D
#         w2.grad = h2.grad^T @ h1  -> 2*B*D*D
# 每层合计: 2 (前向) + 4 (反向) = 6 * B * D * D
```

代码做了什么： – 统计矩阵乘法（`2 * M * N * K`）的 FLOPs，并说明一层的反向传播恰好是前向的 2 倍（两次矩阵乘法：一次算输入梯度、一次算权重梯度），从而导出著名的 **6ND 法则**。

实现深挖： – **为什么是 2·B·D·K**：每个输出元素是一个长度为 K 的点积，约含 K 次乘法 + (K−1) 次加法 ≈ 2K FLOPs，乘以输出元素个数（讲义按"每个 (i,j,k) 三元组一次乘加"计为 2·B·D·K）。真正重要的是**比例**：反向 = 2 × 前向。 – **为什么它对 Transformer 只是近似**：6ND 对 MLP 精确，对短上下文的 Transformer 是很好的近似（长上下文下 attention 会额外贡献不可忽略的一项）。 – **为什么要实测而非只看规格**：峰值 FLOP/s 依赖精度与稀疏性（H100：含稀疏 1979 TFLOP/s，不含则减半）；实际吞吐要用"FLOPs ÷ 时间"测出来，再算出 MFU。

与作业的联系：作业 1 的**资源核算**部分要求你针对给定配置，算出 Transformer 每个组件的 FLOPs（embedding、attention 的 QK^T、softmax、attention·V、MLP 的 up/gate/down 投影、LM head）——正是这种 2·M·N·K 计数。显存核算（bf16 下 2 字节参数 + 2 字节梯度 + 8 字节 AdamW 优化器状态 = 每参数 12 字节）同样属于作业 1，并且是作业 2 分布式显存规划的基础。

代码示例：从零实现 AdaGrad 与训练循环

代码（Python）：

```
class AdaGrad(torch.optim.Optimizer):
    def __init__(self, params, lr=0.01):
        super().__init__(params, dict(lr=lr))
    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                state = self.state[p]
                grad = p.grad.data
                g2 = state.get("g2", torch.zeros_like(grad))  # 梯度平方的累积和
                g2 += torch.square(grad)
                state["g2"] = g2
                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)

# 标准训练循环：
for t in range(num_train_steps):
    x, y = get_batch()
    pred_y = model(x).mean()
    loss = F.mse_loss(pred_y, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

代码做了什么： 1. `AdaGrad` 为每个参数维护"梯度平方的累积和" `g2`，并用 `sqrt(g2 + eps)` 去除梯度——过去的梯度越大，当前有效步长越小（逐坐标自适应学习率）。 2. 训练循环完成标准动作：采样 batch → 前向 → 算损失 → 反向 → 优化器更新 → 清零梯度。

实现深挖： – **为什么要自己写优化器**：讲义用 AdaGrad 作为铺垫，串起优化器家族谱系：momentum = SGD + 梯度的指数平均；AdaGrad = SGD + 梯度平方的（累积）平均；RMSProp = 把 AdaGrad 的梯度平方改成指数平均；**Adam = RMSProp + momentum**。作业 1 要求从零实现 **AdamW**——你写的类基本上就是上面这个加上一阶/二阶矩估计与权重衰减。 – **为什么优化器状态用 fp32**：讲义原话是"习惯上使用 fp32 以保证稳定性（要在很多步上累积幂次平均）"——这正是混合精度的由来：参数/梯度 bf16，优化器状态 fp32（Adam 两个矩共 8 字节/参数；AdaGrad 一个 4 字节/参数）。 – **为什么 `zero_grad(set_to_none=True)`**：置为 None 比清零更快释放显存。

与作业的联系：作业 1：实现交叉熵损失、AdamW，以及带 checkpoint 保存/加载的训练循环——正是上面这个模式。显存表（参数 2B + 梯度 2B + 优化器状态 4–8B + 激活）就是你在资源核算里要报告的；作业 2 在这个循环之上构建分布式版本（DDP/FSDP）。

代码示例：梯度累积与激活重计算

代码（Python）：

```
# 梯度累积：用 256 的 micro-batch 模拟 4096 的大 batch
for micro_step in range(accumulation_steps):
    x, y = get_micro_batch()
    loss = loss_fn(model(x), y) / accumulation_steps  # 损失要缩放
    loss.backward()           # 累积进 .grad（不清零）
optimizer.step()              # 整个逻辑 batch 只更新一次
optimizer.zero_grad(set_to_none=True)

# 激活重计算：
for layer in self.layers:
    x = torch.utils.checkpoint.checkpoint(layer, x)  # 反向时重算
```

代码做了什么： 第一段把一次优化器更新摊到多个 micro-batch 上，用"一个 micro-batch 的激活显存"换到与大批量相同的梯度统计量。第二段用 `torch.utils.checkpoint.checkpoint` 包裹每层：前向丢弃中间激活，反向时重新计算。

实现深挖： – **为什么要梯度累积**：激活显存随 batch 大小线性增长；逻辑 batch 为 64×1024 维 × 16 层时激活需要 2·64·1024·16 字节，容易爆显存。切成 256 的 micro-batch 可把激活显存降低 4 倍。 – **为什么重计算是"用算力换显存"**：全部保存是 O(L) 显存、零重算；完全不存是 O(1) 显存但 $O(L^2)$ 算力（每层都从头重算）；每隔 $\sqrt{L}$ 层存一次则在两者之间取得平衡。 – **为什么注意损失缩放**：损失要除以累积步数，否则等价于把学习率乘以累积步数。

与作业的联系：作业 2 的**激活重计算**任务（为 TransformerBlock 实现重计算，并用显存 hook 验证）正是这个例子的直系后代——handout 里同样给出了 `pack_hook/unpack_hook` 插桩来测量被保存张量的显存。

关键要点

1. 一切都是张量：参数、梯度、激活、优化器状态——每种的显存 = 元素个数 × 单元素字节数（bf16 为 2B，fp32 为 4B）。
2. 6ND 法则：训练成本约等于"每参数每 token 6 个 FLOPs"（前向 2 + 反向 4）；反向是前向的 2 倍。
3. 用 roofline 思维：矩阵乘法是 compute-bound（算术强度约 n/3），逐元素运算（ReLU/GELU）与点积是 memory-bound——所以"孤立地看，ReLU 并不比 GELU 快"，而推理（矩阵–向量乘）必然是 memory-bound。
4. 显存技巧很重要：混合精度（bf16 + fp32 优化器状态）、梯度累积、激活重计算，让你能塞下更大的 batch 或模型。
5. einops 让张量数学可读可调；MFU ⩾ 0.5 已算不错，而且计时一定要配合 `torch.cuda.synchronize()`。

常见陷阱

- **fp16 下溢**：1e-8 这类值会塌成 0 导致训练不稳定；优先用 bf16，或用 fp32 做累加。
- **benchmark 忘记 `torch.cuda.synchronize()`**：CUDA 是异步的，不同步测到的只是 kernel 启动开销，不是 kernel 时间。应使用 CUDA events。

- **混淆 FLOPs（工作量）与 FLOP/s（速度）**：两者读音相同但是不同量；另外峰值 FLOP/s 取决于精度与稀疏性。
- **梯度累积时忘记缩放损失**：等价于改变了有效学习率。
- **核算显存时忽略激活**："8 张 H100 能训多大模型"的纸面推算只是**上界**——激活取决于 batch 与序列长度，可能成为主导项。
- **盲目使用 `-2, -1` 转置**：attention/MLP 代码里的维度错乱是经典 bug 来源；要么命名维度（einops），要么在注释里写清形状。

复习题

1. **问：** 某 GPU 峰值 1000 TFLOP/s、带宽 3.35 TB/s。某负载搬运 1 GB、做 1 TFLOP。它是 memory-bound 还是 compute-bound?
2. **答：** 加速器强度 ≈ 1000e12 / 3.35e12 ≈ 298 FLOP/byte；负载强度 = 1e12 / 1e9 = 1000 FLOP/byte > 298 → compute-bound。
3. **问：** 为什么线性层的反向传播是前向的 2 倍?
4. **答：** 需要算两个梯度：传给前面层的输入梯度，以及本层权重梯度——各自都是与前后向同量级的矩阵乘法，因此前向 1 次 + 反向 2 次 ≈ 每层 $6 \cdot B \cdot D^2$，即前向 2ND、反向 4ND。
5. **问：** 为什么推理的算术强度低，而训练不低?
6. **答：** 训练处理大的批量矩阵乘法（B≫1，compute-bound）。推理一次只解码一个 token（B=1）：每步都要读全部参数（矩阵-向量乘），强度约等于 1，远低于加速器强度——因此是 memory-bound。

第 3 讲：模型架构与超参数

日期：4 月 6 日（周一，Spring 2026） | 讲师：Tatsu Hashimoto | 材料： `lecture_03.pdf`

概览

本讲回答两个问题：*大型语言模型有哪些共同点？哪些地方在变？* 先回顾"原始 Transformer"的设计选择（post-norm + LayerNorm、正弦位置编码、ReLU 前馈），再对比"简洁的现代变体"（pre-norm、RoPE、SwiGLU、去掉 bias），随后系统梳理几十个已发布模型在归一化、激活函数、位置编码以及超参数（FFN 比例、head 维度、长宽比、词表大小、正则化）上的经验共识，最后讲稳定性技巧（z-loss、QK-norm、logit soft-capping）。

核心概念与定义

- **Pre-norm 与 post-norm**：把 LayerNorm 放在子层**之前**（pre-norm），从而不让它落在主残差信号通路上。几乎所有现代 LM 都是 pre-norm（BERT 是 post-norm）；OPT-350M 是个有趣的反例。
- *类比*：pre-norm 像把滤水器装在水龙头上（用水时才净化，主管道保持干净）；post-norm 像把滤水器装在蓄水池出口，所有水回流时都得穿过它。

- *原因*：更好的梯度传播、更少的梯度尖峰、大规模下的稳定性，并允许更大的学习率；最初宣称的好处是"可以去掉 warmup"。
- **LayerNorm 与 RMSNorm**：LayerNorm 在隐藏维度上减均值、除方差；RMSNorm 只用均方根做缩放（`y = x / sqrt(mean(x²)+ε) * γ`），不减均值、没有 bias。GPT-3/OPT/GPT-J 用 LayerNorm；LLaMA/PaLM/Chinchilla/T5 用 RMSNorm。
- *为什么用 RMSNorm*：运算更少（不算均值）、参数更少（没有 bias）；两者的 FLOPs 本来就微不足道，但**FLOPs ≠ 运行时间**——数据搬运才是关键，RMSNorm 搬的字节更少，因此实测墙钟时间更快（Narang 等，2020）。
- **门控线性单元（*GLU）**：把 `FFN(x) = activation(xW1) W2` 换成门控版本，例如 `SwiGLU(x) = (swish(xW1) ⊙ (xV)) W2`，多出一个门控投影 V。GeGLU（高斯误差门控）与 SwiGLU 是标准选择；门控 FFN 的中间维度取约 2/3。证据显示收益稳定（Shazeer 2020；Narang 等 2020）。2023 年后的模型大多用 SwiGLU。
- *类比*：门控单元像夜店门口的保安：一个投影逐元素决定"内容"投影能通过多少。
- **串行层与并行层**：标准块先 attention 再 MLP（串行）；"并行"块（GPT-J、PaLM）把两者并列计算后相加。并行层可共享 LayerNorm、融合矩阵乘法，但现代模型大多是串行。
- **位置编码（position embeddings）**：正弦式（加 sin/cos，原始 Transformer）、绝对可学习式（GPT-1/2/3、OPT）、相对式（T5、Gopher），以及 **RoPE**（旋转位置编码，GPT-J/PaLM/LLaMA 及绝大多数 2024+ 模型）。
- **RoPE（旋转位置编码）**：把 query/key 的坐标成对旋转一个与位置成正比的角，使 attention 分数只依赖**相对**位置：`⟨f(x,i), f(y,j)⟩ = g(x, y, i-j)`。
- *类比*：把每个 token 的向量想成一根钟表指针；RoPE 把指针旋转一个代表位置的角。两根指针旋转后的点积只取决于**角度差**（相对位置），与钟表的绝对读数无关。
- **超参数共识**：FFN 维度 ≈ 4 × 模型维度（GLU 变体约 8/3，实际多用 2.5–2.7）；head_dim × num_heads ≈ model_dim（比例大多围绕 1）；长宽比 model_dim/layers ≈ 100–200；词表大小：单语 30–50K，多语 100–250K。
- **正则化**：新模型在预训练中基本不用 dropout（数据量太大、只过一遍、难以记住），只依赖 weight decay——而在 LLM 中 weight decay 更多是影响优化动力学（与学习率调度耦合），而非控制过拟合（Andriushchenko 等 2023）。
- **稳定性技巧**：
- **z-loss**：加一项惩罚 log-sum-exp 过大的损失，防止 logits 漂移；PaLM、Baichuan 2、DCLM、OLMo 2/3 使用。
- **QK-norm**：在 attention softmax 之前对 query/key 做归一化（RMSNorm/LayerNorm），让 attention logits 有界；DCLM、OLMo 2、Gemma 2、Qwen 3、Chameleon 使用。
- **Logit soft-capping**：用 `tanh` 把 logits 截断到上限。
- **GQA/MQA 回顾**：减少 key/value head 数量以削减 KV cache、提升推理性能（详见第 10 讲）。
- **交错 attention（interleaved attention）**：例如 Cohere Command A 每第 4 层用 full attention、其余用局部 attention；LLaMA 4、Gemma 3/4、OLMo 3 交错使用滑窗（SWA）与 full attention。

代码示例：RMSNorm（作业 1 的公式）

讲义幻灯片定义了要实现什么，作业 1 handout 给出了形式化描述。给定激活向量 `a ∈ R^d_model`，RMSNorm 逐元素重缩放：

代码（Python）：

```
import torch
from torch import nn

class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))  # 可学习的逐维增益 γ

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (..., dim)
        rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return x / rms * self.weight
```

代码做了什么： 计算最后一维的均方根，用它（加 ε）去除原张量，再乘以可学习的增益。不减均值、没有 bias——正是现代配方。

实现深挖： – **为什么不用减均值和 bias**：运算更少、需要搬运的参数更少；经验上与 LayerNorm"一样好"，却节省墙钟时间（FLOPs 由矩阵乘法主导，但**数据搬运**不是）。 – **为什么要 ε**：当均方根极小时保证数值稳定。 – **为什么它属于"现代变体"清单**：pre-norm 位置 + RMSNorm + RoPE + SwiGLU + 无 bias 正是作业 1 要实现的内容——讲义明确发问"我们为什么这样选？你应该怎么选？"（答案：经验共识 + 稳定性 + 效率）。

与作业的联系：作业 1 第 3 节：实现 `RMSNorm`，接口与上面一致（它也会被用在你的 attention 与 FFN 块里）。作业 2 的 **RMSNorm 融合 Triton kernel** 则把同一个算子写成单个 GPU kernel——它是 memory-bound 的逐元素运算，融合收益显著。

代码示例：RoPE（概念实现）

代码（Python）：

```
def precompute_rope_cache(seq_len: int, head_dim: int, base: float = 10000.0):
    # positions: [seq_len]
    positions = torch.arange(seq_len)
    # frequencies: [head_dim // 2] (等比数列)
    freqs = 1.0 / (base ** (torch.arange(0, head_dim, 2) / head_dim))
    # angles: [seq_len, head_dim // 2]
    angles = positions[:, None] * freqs[None, :]
    cos = torch.cos(angles)
    sin = torch.sin(angles)
    return cos, sin  # 预计算一次，所有层/头复用

def apply_rope(x, cos, sin):
    # x: [..., seq, head_dim]；成对旋转坐标
    x1, x2 = x[..., : x.shape[-1] // 2], x[..., x.shape[-1] // 2 :]
    # 复数式旋转：(x1 + i x2) * (cos + i sin)
    return torch.cat([x1 * cos - x2 * sin, x1 * sin + x2 * cos], dim=-1)
```

代码做了什么： 先为所有位置构建 cos/sin 缓存（采用经典的等比频率序列），再把 query/key 的每一对坐标 `(x1, x2)` 按位置相关的角度旋转——即复数乘法 `(x1 + i·x2)(cos + i·sin)`。

实现深挖： – **为什么成对旋转**：每次旋转二维坐标对是 RoPE 的标准构造（动机来自复数）；Gemma 4 的变体只旋转前 2 个坐标。关键性质是：`⟨RoPE(x,i), RoPE(y,j)⟩ = g(x,y,i−j)`——内积（attention 分数）只依赖相对位置，且**没有交叉项**（这正与加法式正弦编码不同）。 – **为什么要预计算**：cos/sin 表只取决于 (seq_len, head_dim)，算一次、每次 attention 直接索引即可。 – **为什么这在 GQA/MLA 里很微妙**：RoPE 与 MLA 式 KV 压缩不兼容（见第 4 讲），所以 DeepSeek 额外保留了若干"不旋转"的隐层 key 维度——即使你只实现普通 RoPE，这个实现细节也值得知道。

与作业的联系：作业 1 第 3 节：实现 RoPE 并在多头注意力中对 Q、K 应用（在算 attention 分数之前或之中）。旋转的轴搞错（每头、每位置的对应关系）是经典调试点。

关键要点

1. 架构共识：pre-norm（非残差式）、RMSNorm（或 LayerNorm）、SwiGLU 门控、串行层、无 bias、RoPE——这个"现代变体"就是你作业 1 要实现的东西。
2. FLOPs ≠ 运行时间：数据搬运才是关键；去掉 bias、使用 RMSNorm 即使不改变 FLOPs 也能节省墙钟时间。
3. 超参数共识：FFN ≈ 4× 模型维度（GLU 为 8/3×）、head_dim×heads ≈ model_dim、长宽比 100–200、词表 30–50K（单语）。
4. 大规模下的正则化主要关乎优化动力学而非过拟合（dropout 基本消失；weight decay 与学习率调度耦合）。
5. 稳定性技巧（z-loss、QK-norm、soft-capping）之所以存在，是因为"带指数的 softmax"正是训练爆炸的地方。

常见陷阱

- **post-norm 配大规模训练**：post-norm 把归一化放进残差通路，大规模下容易不稳定；现代做法是 pre-norm（可选再加一个非残差的 post-norm，如 Grok/Gemma 2/OLMo 2）。
- **GLU 用了错误的 FFN 比例**：SwiGLU 若仍用 4× 模型维度，参数量会多出约 1.5 倍；应当用约 8/3×。
- **RoPE 应用到错误维度、或忘记逐头旋转**：attention 会静默退化；务必与参考实现比对。
- **盲目照搬词表大小**：32K 词表适合英文，但多语模型需要 100–250K。
- **忽视 softmax 稳定性**：没有 z-loss/QK-norm，logits 可能漂移，大训练直接发散。

复习题

1. **问：** 现代模型为什么用 pre-norm 而不用 post-norm?
2. **答：** pre-norm 让 LayerNorm 不落在主残差通路上，改善梯度流动、减少尖峰，从而在大规模下更稳定并支持更大的学习率（最初宣称的好处是省掉 LR warmup）。
3. **问：** 某模型 model_dim=4096、采用 SwiGLU，FFN 维度大致应取多少？为什么?
4. **答：** 约 8/3 × 4096 ≈ 10,923（实际模型多取 2.5–2.7 倍）。门控变体之所以只要 ReLU FFN 的 2/3 维度，是因为门控在增加参数的同时提升了表达能力。
5. **问：** 是什么性质让 RoPE 在"使用绝对位置索引"的同时具有相对性?
6. **答：** 旋转让两个位置编码向量的内积只依赖角度**差**，即 `⟨f(x,i), f(y,j)⟩ = g(x,y,i−j)`——绝对位置被消掉。

第 4 讲：Attention 替代方案与混合专家（MoE）

日期：4 月 8 日（周三，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_04.pdf`

概览

Attention 的复杂度对序列长度是平方级的，在长上下文下会成为主导成本。本讲讲两类应对方案：(1) **attention 替代方案**——线性注意力、状态空间混合模型（Mamba-2、Gated DeltaNet）、稀疏注意力（DSA）；(2) **混合专家（MoE）**——用一个路由器把大 FFN 换成许多专家 FFN，从而把"参数量"与"每 token 的 FLOPs"解耦。内容包括路由方法、训练目标（辅助均衡损失）、系统实现、upcycling，以及 DeepSeek MoE v1→v3 的演进。

核心概念与定义

- **Attention 的成本**：`Attn(Q,K,V) = softmax(QKᵀ)V` 中，$QK^\top$ 需 $O(n^2 \cdot d_k)$、attention·V 需 $O(n^2 \cdot d_v)$。上下文窗口越大，这一项越主导。
- *类比*：full attention 像派对里每个人都和每个人交谈——n^2 次对话；线性注意力则像每个人只对记录员说一句摘要、由记录员统一广播——约 $2 \cdot n \cdot d$ 次交流。

- **线性注意力（linear attention）**：若把 softmax 换成恒等核，则 `QKᵀV = Q(KᵀV)`，成本从 O(n²d) 降到 O(2·n·d_v·d_k)——对序列长度线性。
- *循环形式*：`S_t = S_{t-1} + k_t v_tᵀ`，`y_t = q_tᵀ S_t`——这看起来就是 RNN！这种**对偶性（duality）**让你用并行（二次）形式训练、用串行（线性）形式推理。把 `S_{t-1}` 乘以权重 γ 就得到 RetNet。
- *类比*：一个上课认真记笔记的学生（状态 S_t）随时能回答问题，而不必为每个新问题把之前所有讲义（全部历史）重读一遍。
- **Mamba-2**：加入逐位置门控：`S_t = γ_t S_{t-1} + k_t v_tᵀ`，其中 `γ_t = f(x_t)`——门控让线性注意力表达力更强，同时保留对偶性。
- **Gated Delta Net（GDN）**：再加输入门与选择性状态擦除：`S_t = γ_t(I − β_t k_t k_tᵀ) S_{t-1} + β_t k_t v_tᵀ`。`β=0` 表示"不写入"；擦除项 `(I − β_t k_t k_tᵀ)` 会忘掉与当前 key 同方向的内容。与 fast weight programmer / test-time training 关系密切。Qwen 3.5/Qwen Next 使用 3:1 的 GDN/attention 混合。
- **混合架构（hybrids）**：不做"全 attention"或"全线性"，而是交错：Minimax M1（7:1 线性:full）、Nemotron 3（约 3:1 Mamba:attention）、Qwen 3.5（3:1 GDN:attention）。受控消融显示小混合比例下损失几乎不退化，而推理收益巨大（常量大小的状态 vs O(n) 的 KV cache）。
- **稀疏自适应（DSA，DeepSeek Sparse Attention）**：不再关注所有 token，而是用一个轻量 **indexer** 选出 top-k 个 token 参与 attention；可在稠密短上下文预训练之后"事后适配"（DeepSeek v3.2、GLM-5）。
- **混合专家（MoE）**：用许多 FFN（"专家"）加一个路由器（router）取代单个大 FFN，每个 token 只走 top-k 个专家。总参数量随专家数增长，但 FLOPs 基本不变（每 token 只激活 k 个专家）。
- *类比*：不是让一位全科医生看所有病人，而是开一家有 256 位专科医生的诊所，由分诊护士（router）把每位病人分给最相关的 2–8 位专家。诊所人手很多（参数多），但每位病人只见少数几位（FLOPs 不变）。
- *MoE 为何流行*：同等 FLOPs 下参数更多 → 损失更低；单位算力下训练更快；与稠密模型相比很有竞争力；天然可跨设备并行（每个专家可放在一张卡上）。实例：Mixtral（8 专家、top-2）、Grok、DBRX（16、top-4）、Qwen（60、top-4）、DeepSeek v3（256 专家、激活 8 + 1 共享）、Llama 4 Maverick（128 路由 + top-1 + 共享）。
- **路由方法**：token 选择专家的 top-k（主流：Switch k=1，GShard/Grok/Mixtral k=2，Qwen/DBRX k=4，DeepSeek k=7–8）、哈希路由（基线）、RL 学习路由（早期工作，Bengio 2013）、线性分配路由（Clark '22）。
- *路由打分方式*：逻辑回归门控（DeepSeek v1–2、Grok、Qwen）vs 在 top-k 之后做 softmax（Mixtral、DBRX、DeepSeek v3）。
- *近期变体*：细粒度专家（很多小专家）+ 少量常开的共享专家（DeepSeek、Qwen；最初来自 DeepSpeed MoE）。消融大体显示细粒度专家有帮助；共享专家的收益存在分歧（OlMoE 认为没有）。
- **MoE 训练——不可微问题**：稀疏路由决策不可微。可选方案：(1) 用 RL 优化门控策略（可行但方差大，未被广泛采用）；(2) 随机扰动（Shazeer 2017 的高斯噪声、Switch Transformer 的乘性抖动）；(3) **启发**

式均衡损失（auxiliary balance loss）——实践标准：例如 Switch 的负载均衡损失会把过热的专家降权；DeepSeek v1–2 同时做逐专家与逐设备均衡；DeepSeek v3 用**无辅助损失的均衡（aux-loss-free）**——通过在线调整每个专家的偏置实现。

- **MoE 的系统侧**：专家可跨设备并行（expert parallelism），每个 FFN 能放进一张卡；稀疏矩阵乘需要专门 kernel（如 MegaBlocks）；Nemotron 3 通过把激活降维来减少通信量。
- **MoE 的稳定性/微调问题**：router logits 建议用 fp32 + z-loss；稀疏 MoE 在小数据上微调容易过拟合（Zoph 等：只微调非 MoE 的 MLP；DeepSeek：用大量数据，1.4M 条 SFT）。
- **Upcycling（升级复用）**：用预训练的稠密模型初始化 MoE（拆分/复制专家权重）。实例：MiniCPM-MoE（来自 MiniCPM）、Qwen-MoE（来自 Qwen 1.8B）。
- **DeepSeek MoE 谱系**：v1（16B/激活 2.8B：共享 2 + 细粒度 64，辅助损失均衡）、v2（236B/激活 21B：共享 2 + 细粒度 160、激活 6，通信均衡损失，top-M 设备路由）、v3（671B/激活 37B：共享 1 + 细粒度 256、激活 8，sigmoid+softmax top-k + top-M，无辅助损失 + 序列级辅助损失）。
- **DeepSeek v3 的配套技巧**：MLA（多头隐层注意力，把 KV 压到低维隐向量 c，见第 10 讲）与 MTP（多 token 预测：用小的轻量头预测多步）。

代码示例：线性注意力的循环形式（概念）

代码（Python）：

```
# 并行（训练）形式：   Y = Q(K^T V)    -- O(n^2 d)，只有两次矩阵乘法
# Q: [n, dk], K: [n, dk], V: [n, dv]
KV = K.transpose(-2, -1) @ V          # [dk, dv]  -- 所有 key/value 外积之和
Y = Q @ KV                            # [n, dv]

# 循环（推理）形式：          -- 每 token O(n)，状态 O(1)
S = zeros(dk, dv)                     # 状态
for t in range(n):
    S = S + K[t][:, None] * V[t][None, :]   # S_t = S_{t-1} + k_t v_t^T
    y[t] = Q[t] @ S                         # y_t = q_t^T S_t
```

代码做了什么： 展示线性注意力的两种等价计算：训练用的并行矩阵形式，与推理用的顺序状态更新形式（永不回看旧 token）。

实现深挖： – **为什么对偶性重要**：训练用并行形式以充分利用 GPU 矩阵乘法；推理用循环形式，每 token 显存 O(1)（KV cache 不增长）。这正是状态空间模型实用的"训练并行、推理串行"技巧。 – **为什么加门控（Mamba-2/GDN）**：最朴素的递推无法遗忘；逐位置门控 γ_t、输入门 β_t 与擦除项 $(I - \beta\ k\ k^\top)$ 赋予选择性记忆，这在语言任务上经验性收益很大。

与作业的联系：这些内容对作业 1–5 属于概念性背景（作业 2 你实现的是完整 attention 的 FlashAttention-2），但这里培养的算术强度推理（第 2、10 讲）正是 GQA/MLA 取舍与 FlashAttention tiling 的理论依据。

代码示例：top-k 路由（概念 MoE router）

代码（Python）：

```
import torch
import torch.nn.functional as F

def topk_route(x, router_weight, num_experts, k):
    # x: [num_tokens, d_model]
    logits = x @ router_weight                # [num_tokens, num_experts]
    topk_logits, topk_idx = torch.topk(logits, k, dim=-1)   # 选中哪些专家
    # 只在被选中的专家上做 softmax（Mixtral 风格）
    probs = F.softmax(topk_logits, dim=-1)
    return topk_idx, probs                    # 把 token 分发到专家
```

代码做了什么： 计算每个 token 对各专家的 logits，取 top-k，并只在被选中的专家上归一化（Mixtral/DBRX/DeepSeek-v3 的做法）。

实现深挖： – **为什么用 top-k 而不是 k=1 的 argmax**：k>1 给路由器更细的粒度并平滑梯度；Switch（k=1）更简单但脆弱。DeepSeek v3 在 256 个专家中激活 8 个。 – **为什么要有共享专家**：常开的共享专家负责捕获通用模式，让路由专家更专注；DeepSeek/Qwen 配 1–4 个共享专家。 – **为什么"top-k 之后才 softmax"**：只对选中的专家归一化，使路由权重与未选中专家的 logits 无关——这是与 DeepSeek v1–2 逻辑门控路由器有意的设计差异。

与作业的联系：必修作业不要求实现 MoE（第 8 讲会从概念上讲 expert parallelism），但作业 2 的分布式训练（all-to-all token 分发）正是 MoE 需要的系统基础；而"在预算下最小化损失"的排行榜思维与 DeepSeek 靠消融做选择的方式一致。

关键要点

1. Attention 的 $O(n^2)$ 成本可用线性/状态空间替代方案（线性注意力、Mamba-2、Gated DeltaNet）来攻击——实践中多以**混合**形式使用（如 3:1），用少许精度换取长上下文推理的巨大成本下降。
2. 线性注意力具有"训练并行/推理递推"的对偶性；门控是让递推具备表达力的关键。
3. MoE 把参数量与 FLOPs 解耦：同等算力下容量更大，代价是复杂的路由、均衡损失与系统开销。
4. top-k "token 选专家"路由 + 启发式均衡损失是实践共识；RL 路由理论优雅但方差太大。
5. 现代 MoE 采用细粒度专家 + 共享专家、无辅助损失均衡、fp32/z-loss 稳定路由器；从稠密 checkpoint 做 upcycling 是获得 MoE 的廉价途径。

常见陷阱

- **负载不均衡**：没有均衡损失时少数专家吃掉所有 token——容量浪费，更糟的是设备闲置。
- **路由器不稳定**：fp16 的 router logits 会爆炸；用 fp32 + z-loss（Zoph 等 2022）。
- **批次级 token 丢弃**：路由在 batch 级别丢 token，意味着**别人的 query 可能把你的 token 丢掉**——这是额外的随机性来源。
- **微调时 MoE 过拟合**：稀疏模型在小 SFT 集上容易过拟合；要么加数据，要么冻结/降权路由专家。
- **MLA/RoPE 的 KV cache 盲区**：RoPE 与 MLA 缓存冲突；需要保留非旋转的 key 维度（DeepSeek 的 64 维技巧）。

复习题

1. **问：** 为什么 `Q(KᵀV)` 能改变 attention 的成本？代价是什么？
2. **答：** 矩阵乘法满足结合律：`(QKᵀ)V = Q(KᵀV)`。左边 $O(n^2 \cdot d)$，右边 $O(n \cdot d^2)$。代价是：只有当 attention 核为**恒等**（没有 softmax）时才成立，这正是线性注意力的假设——softmax attention 无法这样因式分解。
3. **问：** 线性注意力的"对偶性"是什么？为什么在工程上重要？
4. **答：** 同一个计算既能写成并行形式（$Q(K^TV)$，适合 GPU 训练），也能写成循环形式（$S_t = S_{t-1} + k_t v_t^T$，推理时状态 $O(1)$）。训练用一种、推理用另一种。
5. **问：** 一个 256 专家、激活 8 专家的 MoE 为何能"参数更多但 FLOPs 不变"？
6. **答：** 每个 token 只经过 k=8 个专家，因此每 token 的矩阵乘 FLOPs 大致相当于"激活规模"的稠密模型；其余 248 个专家的参数仍占显存（提供容量/知识），但对该 token 不产生计算。

第 5 讲：GPU

日期：4 月 13 日（周一，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_05.pdf`

概览

本讲为 GPU "去神秘化"：它和 CPU 有何不同、内部构造（SM、warp、内存层次），以及最关键的——*为什么 GPU 会变慢* 与 *如何写出快算法*：低精度计算、算子融合、重计算、显存合并访问（coalescing）、分块（tiling）。最后以 FlashAttention 作为综合案例拆解：KQV 矩阵乘的分块 + 在线（telescoping）softmax 技巧。

核心概念与定义

- **GPU 与 CPU 的区别**：CPU 为"少数快线程"优化（延迟优先）；GPU 为"海量线程"优化（吞吐优先）。GPU 有大量小型 ALU、对分支支持弱、内存层次深。
- *类比*：CPU 是几位手艺精湛的工匠，每人做得极快；GPU 是上千名简单工人的流水线，总产量惊人——但每一步所有人都必须执行同一条指令（SIMT）。
- **硬件结构**：SM（流式多处理器）内部含大量 SP（流式处理器）来执行线程；一个**线程块（thread block）** 跑在一个 SM 上并拥有自己的共享内存；线程以**warp**（32 个连续编号的线程）为单位锁步执行（SIMT——单指令多线程）。
- **内存层次**：寄存器（最快）→ 共享内存/L1（在 SM 内部，比 DRAM 快约 8 倍但每字节成本高约 100 倍）→ L2（片上）→ HBM/global memory（GPU 旁的 DRAM 芯片）。**算力（FLOPs）增长快于显存带宽**——"内存墙（memory wall）"——因此"如何持续喂饱计算单元"是核心问题。
- **Tensor Core**：专用矩阵乘电路（Volta 起），使矩阵乘比其他浮点运算快 10 倍以上。TPU 思路类似：轻量控制 + 大而快的矩阵乘单元 + 快内存，但核心数更少更大、没有 warp 概念（只有 block 模型）。

- **Roofline 模型**：横轴为算术强度（FLOPs/byte），纵轴为达到的性能（FLOP/s）。拐点 = 加速器强度 = 峰值 FLOP/s ÷ 带宽。拐点左侧是 memory-bound（性能随强度线性上升）；右侧是 compute-bound（贴在峰值上）。
- **让 GPU 变快的六种手段**（摘自幻灯片）：
- **低精度计算**：比特更少 = 搬运字节更少，提升算术强度（fp32 的 ReLU：8 字节/FLOP → fp16：4 字节/FLOP）。Tensor core 加速低/混合精度。前沿方向：FP8（E4M3/E5M2）、MXFP8（分块缩放、E8M0 缩放因子）、NVFP4。
- **算子融合（operator fusion）**：把多个逐元素算子合成一个 kernel，避免数据在 HBM 之间来回搬运（例如 `sin²x + cos²x` 从 5 个 kernel 变成 1 个）。*类比*：由仓库传送带供料的工厂——不要把半成品每步都退回仓库；一次走完全部工序。
- **重计算（activation checkpointing）**：不保存全部激活，反向时重算。常常是最优选择：3 个叠加 sigmoid 用重计算后显存访问降到 5/8。
- **显存合并（memory coalescing）**：DRAM 以突发（burst，128 字节事务）读取；当一个 warp 的 32 个线程落在同一个突发内时访问被合并。对行主序矩阵而言，线程沿"行"方向移动是**不合并**的——经典性能陷阱。
- **分块（tiling）**（最重要）：把输出矩阵切成 tile，把 A/B 的 tile 一次载入共享内存，被多个输出元素复用，并使访问合并。未分块的矩阵乘会从 global memory 读每个输入 N 次；分块后从 global memory 只读 N/T 次、从共享内存读 T 次——HBM 流量降低 T 倍。
- **避免控制发散（control divergence）**（不是访存问题）：warp 内线程执行同一指令，条件分支会串行化（先走 A 路径再走 B 路径）——数据相关分支的隐形成本。
- **Wave quantization（波量化）**：若线程块数量不能整除 SM 数量，最后一个波次会部分闲置（A100 有 108 个 SM；120 个 tile → 108 + 12）。这解释了"神秘"的周期性性能凹陷（1792→1793 的矩阵乘之谜）。
- **FlashAttention 拆解**：
- *第一步*：KQV 矩阵乘的分块（blocked GEMM）——把 A/B tile 通过共享内存搬运。
- *第二步*：增量（在线）softmax——为了逐 tile 归一化，需维护滑动最大值并用 telescoping 修正，从而在不物化完整 $S = QK^T$ 矩阵的前提下得到精确的 softmax 分母。
- 反向传播：逐 tile 重算（不保存 attention 矩阵）。

代码示例：在线 softmax（FlashAttention 的核心）

代码（Python）：

```
import torch, math

def online_softmax_attention(Q, K, V, block_size=2):
    # Q,K,V: [n, d]（单头）；按 tile 处理打分矩阵的行
    n = Q.shape[0]
    acc = torch.zeros(n, V.shape[1])          # 加权和累加器
    m = torch.full((n,), -float("inf"))       # 滑动行最大值
    l = torch.zeros(n)                        # 滑动 exp 之和
    for j in range(0, n, block_size):
        S = Q @ K[j:j+block_size].T           # 打分 tile: [n, block]
        m_new = torch.maximum(m, S.max(dim=1).values)
        alpha = torch.exp(m - m_new)          # 重新缩放旧累加器
        P = torch.exp(S - m_new[:, None])     # 未归一化的 tile 概率
        acc = acc * alpha[:, None] + P @ V[j:j+block_size]
        l = l * alpha + P.sum(dim=1)
        m = m_new
    return acc / l[:, None]                   # 最终归一化

# 与朴素实现对照：
def naive(Q, K, V):
    S = Q @ K.T
    P = torch.softmax(S, dim=-1)
    return P @ V
```

代码做了什么： 在不物化完整 `[n, n]` 打分矩阵的前提下计算 softmax attention。它按 K/V 的列 tile 迭代，为每一行维护三个滑动量：最大值 `m`、指数和 `l`、加权累加器 `acc`。当新 tile 带来更大的最大值时，旧累加器用 `exp(m_old - m_new)` 重新缩放（telescoping 修正），因此最终的 `acc / l` 是精确的。

实现深挖： – **为什么要维护滑动最大值并重缩放**：标准 softmax `exp(S - max(S))` 需要先拿到整行；在线技巧让你可以**流式**处理 tile——用 `exp(m_old - m_new)` 缩放累加器，保证每个贡献都相对当前最大值被正确加权。这正是 FlashAttention 的前向，并与 KQV 矩阵乘融合在一起。 – **为什么它带来 O(1)-block 显存**：只有累加器（[n, d]）和 P 的 tile（[n, block]）活在寄存器/共享内存中；完整的 S 和 P 从不写入 HBM。 – **为什么反向要重算**：在反向重算 S 和 P 的 tile 就不必保存它们，代价是多一次 QK^T 类计算——即幻灯片里"用算力换显存"的取舍。

与作业的联系：作业 2 的核心任务是用 **Triton 实现 FlashAttention-2**（前向 *和* 反向），其中就包含这里的在线 softmax + 分块逻辑，再加上 mask 与 bias 处理。听课时的"分块 + 在线 softmax"是概念蓝图；作业里的 Triton 技巧（tl.dot、block pointer、反向重算）是机械实现。

关键要点

1. GPU 是高度并行的 SIMT 机器：32 线程的 warp 锁步执行、线程块跑在带共享内存的 SM 上，且内存层次中真正的稀缺资源是带宽（而不只是 FLOPs）。
2. 算力增长快于显存 → 必须最小化数据搬运：融合算子、合并访问、通过共享内存分块，并（有时）用重算代替保存。
3. 低精度（fp16/bf16/fp8）提升算术强度并解锁 tensor core；roofline 模型告诉你处于 memory-bound 还是 compute-bound。

4. 分块 + 在线 softmax = FlashAttention：把"看起来必然平方复杂度"的算子变成访存高效、全融合 kernel 的经典范例。
5. 性能充满量化效应（波量化、对齐、bank conflict）——benchmark 与 profiling 必不可少，而像 1792→1793 这样的小改动可能带来非直观的大幅波动。

常见陷阱

- **共享内存 bank conflict**：32 个 bank、每周期每 bank 一次访问；跨步访问模式（例如读矩阵的列）会造成 32 路串行化。用 padding/swizzling 缓解。
- **未合并的 HBM 访问**：线程索引必须映射到连续地址（128 字节事务）；行主序下沿列方向遍历是经典杀手。
- **波量化**：网格尺寸尽量整除 SM 数量，避免最后一个波次闲置。
- **warp 发散**：数据相关分支（例如 ReLU kernel 里的 `if x < 0`）会让两条路径都串行执行——理论无害，实践昂贵。
- **寄存器膨胀导致低占用率**：单线程使用超过约 160 个寄存器会减少 SM 能调度的 warp 数；thread coarsening（一个线程处理多个元素）有时是解药、有时是病因。
- **以为 FLOPs 等于运行时间**：同一个操作因融合与数据搬运差异，墙钟时间可能天差地别（第 3 讲的 RMSNorm 例子）。

复习题

1. **问：** 在同一块硬件上，为什么矩阵–向量乘是 memory-bound，而矩阵–矩阵乘是 compute-bound?
2. **答：** 两者都要读 $O(n^2)$ 字节的矩阵，但矩阵–向量乘只做 $O(n^2)$ FLOPs（强度约 1），矩阵乘做 $O(n^3)$ FLOPs（强度约 $n/3$）。$n=1024$ 时矩阵乘强度约 341 ≫ H100 的约 295（compute-bound），而矩阵–向量乘强度约 $1 \ll 295$（memory-bound）。
3. **问：** 在线 softmax 如何在流式处理 tile 的同时保持结果精确?
4. **答：** 它维护滑动最大值 m，并在最大值增大时把已累加的项乘以 exp(m_old − m_new)。这在代数上等价于"事后一次性减去最终最大值"——一种 telescoping 修正——因此最终累加器等于真实的 softmax 加权和。
5. **问：** 为什么分块能把矩阵乘的 HBM 流量降低 T 倍（T 为 tile 大小）？
6. **答：** 每个输入元素只需在每个它参与的 tile 中载入共享内存一次（N/T 次，而不是 N 次），而在 tile 内部从高速共享内存读 T 次。全局显存读次数从每元素 O(N) 降到 O(N/T)。

第 6 讲：Kernel 与 Triton

日期：4 月 15 日（周三，Spring 2026）| 讲师：Percy Liang | 材料：`lecture_06.py` | 截止：作业 1 到期、作业 2 发布

概览

这是一讲"动手写 kernel"的课：先用 benchmark 与 profiling 找出瓶颈，再用 **Triton** 写自定义 kernel 消除瓶颈。讲义由易到难开发四个 kernel——GeLU（逐元素）、softmax（行内归约）、row-sum（行超过一个 block 的归约）、matmul+ReLU（用共享内存分块）——并把第 5 讲的 GPU 编程模型（thread → thread block → grid）、占用率、bank conflict、合并访问与波量化落到具体代码上。

核心概念与定义

- **Kernel**：在 GPU 上运行的函数。用 PyTorch 时，每个基础算子都会启动一个标准 kernel；写自定义 kernel（CUDA/Triton/CUTLASS/ThunderKittens）可以融合与分块，让"GPU 起飞（go brrr）"。
- **GPU 硬件表（讲义数据）**：A100：108 SM、192KB L1+共享、40MB L2、80GB HBM、2TB/s；H100：132 SM、256KB、50MB L2、80GB、3.35TB/s；B200：148 SM、256KB、96–126MB L2、192GB、8TB/s。寄存器带宽是 HBM 带宽的 4–20 倍——所以"把数据留在寄存器里"。
- **编程模型**：*thread*（在小片数据上执行）→ *thread block / CTA*（共享共享内存的一组线程，被调度到一个 SM 上）→ *grid*（线程块的集合）。逐元素算子天然映射到线程；归约/矩阵乘需要线程块，因为线程之间必须通过共享内存通信。
- **Triton 的模型**：你描述的是**每个线程块**要做什么（而 CUDA 描述每个线程）：把 tile 从 global memory 载入共享内存、计算、写回。Triton 编译到 PTX（GPU 汇编）。
- **Warp**：32 个线程锁步执行；控制发散（warp 内 if/else）会串行执行；某个 warp 因访存阻塞时，SM 可以零成本切换到其他 warp。
- **占用率（occupancy）**：SM 上可同时驻留的 warp 数，受寄存器（每线程 0–255）、共享内存等限制。低占用率不一定坏，如果每个线程做更多事（thread coarsening）。例：128 线程 × 160 寄存器 = 每块 20480 寄存器 → 65536/20480 = 3 个块。
- **Bank conflict（共享内存）**：32 个 bank × 4 字节，每周期每 bank 一次访问。32 个线程撞同一个 bank（例如读矩阵的列）→ 32 路串行。Swizzling（行列异或）重排地址可避免冲突。
- **显存合并（HBM）**：warp 的 32 次访问若连续，则合并为一个 128 字节事务；完全合并 = 32 线程 × 4 字节一次事务。
- **波量化**：线程块按波次填入 SM；148 个 SM 上跑 160 个块 → 148 + 12（第二波大部分闲置）。解法：让块数整除 SM 数。
- **Benchmark 与 profiling**：benchmark 测端到端墙钟（用于比较实现、研究扩展性）；profiling 显示*哪些 kernel* 在执行、各花多久（PyTorch profiler、nsight）。kernel 名字本身就泄露实现细节：`cutlass3x_sm100_simt_sgemm_f32_..._64x64x16` = CUTLASS 库、Blackwell（sm100）、float32、64×64×16 tile。
- **Kernel 融合**：朴素 GeLU 会启动多个 kernel（多次 HBM 往返）；融合/builtin/torch.compile 版本只跑一个 kernel（一次读、一次写）——对 memory-bound 的逐元素运算收益巨大。

代码示例：benchmark 与 profiling 框架

代码（Python）：

```
def benchmark(run: Callable, num_warmups: int = 1, num_trials: int = 3) -> float:
    for _ in range(num_warmups):
        run()
    torch.cuda.synchronize()          # 关键：冲刷异步 CUDA 任务

    times: list[float] = []
    for trial in range(num_trials):
        start_event = torch.cuda.Event(enable_timing=True)
        end_event = torch.cuda.Event(enable_timing=True)
        start_event.record()          # GPU 侧时间戳
        run()
        end_event.record()
        torch.cuda.synchronize()
        times.append(start_event.elapsed_time(end_event))
    return mean(times)

def profile(run: Callable, num_warmups: int = 1):
    for _ in range(num_warmups):
        run()
    torch.cuda.synchronize()
    with torch.profiler.profile(activities=[ProfilerActivity.CUDA]) as prof:
        run()
        torch.cuda.synchronize()
    return prof.key_averages().table(sort_by="cuda_time_total", row_limit=10)
```

代码做了什么： benchmark 包装器先做 warmup（编译/JIT），用 CUDA events 做 GPU 级精确计时（避开 CPU 启动开销），并对多次试验取平均。profiler 则给出按 CUDA 时间排序的逐 kernel 表格。

实现深挖： – **为什么要 warmup + synchronize**：前几次启动可能触发编译；CUDA 是异步的，不同步就会测到启动延迟而非 kernel 时间。CUDA events 在 GPU 上打时间戳，排除 CPU 开销。 – **为什么要多次试验**：kernel 时间有抖动（时钟、显存状态）；平均可降噪。做扩展性研究时按维度扫（256→8192）：小矩阵受启动开销支配（时间近似恒定），大矩阵呈现立方增长。 – **为什么 profiling 关键**：朴素 vs builtin vs compiled GeLU 的对比揭示了*为什么*某个更快：profiler 显示是"许多 kernel 启动（未融合）"还是"一个 kernel"。

与作业的联系：作业 2 第一部分正是这件事：为你的作业 1 模型搭建 benchmark + profiling 框架（含 Nsight Compute 与 NVTX range），报告逐 kernel 运行时间，并回答"前向+反向哪个 kernel 占主导"。讲义中的 `run_operation1/2`、warmup、CUDA event 模式就是参考实现。

代码示例：Triton GeLU（逐元素）

代码（Python）：

```
import triton
import triton.language as tl

def triton_gelu(x: torch.Tensor):
    assert x.is_cuda and x.is_contiguous()
    y = torch.empty_like(x)
    num_elements = x.numel()
    BLOCK_SIZE = 1024
    num_blocks = triton.cdiv(num_elements, BLOCK_SIZE)   # 向上取整除法
    triton_gelu_kernel[(num_blocks,)](x, y, num_elements, BLOCK_SIZE=BLOCK_SIZE)
    return y

@triton.jit
def triton_gelu_kernel(x_ptr, y_ptr, num_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)          # 当前是哪个块
    start = pid * BLOCK_SIZE
    offsets = start + tl.arange(0, BLOCK_SIZE)   # 本块负责的元素下标
    mask = offsets < num_elements                # 不要越界读写
    x = tl.load(x_ptr + offsets, mask=mask)      # 从 HBM 读
    # tanh(a) = (exp(2a) - 1) / (exp(2a) + 1)，因为 tl.tanh 不存在
    a = 0.79788456 * (x + 0.044715 * x * x * x)
    exp = tl.exp(2 * a)
    tanh = (exp - 1) / (exp + 1)
    y = 0.5 * x * (1 + tanh)
    tl.store(y_ptr + offsets, y, mask=mask)      # 写回 HBM
```

代码做了什么： 每个 1024 元素一个线程块；每个块算出自己的索引范围、带边界掩码地载入、计算 GeLU 的 tanh 近似（因为 Triton 没有 `tl.tanh`，用 exp 重新实现），再写回——全在一个 kernel 内：一次 HBM 读、一次 HBM 写。

实现深挖： – **为什么要 mask**：`num_elements` 未必整除 BLOCK_SIZE；掩码避免越界访问（否则是静默的数据损坏 bug）。 – **为什么 `tl.constexpr`**：编译期常量 → Triton 会特化/展开；网格大小是运行期量。`triton.cdiv` 是向上取整除法，保证最后一个块覆盖尾部。 – **为什么自己实现 tanh**：Triton 语言算子集有限；`(e^{2a}-1)/(e^{2a}+1)` 是标准替代写法。这很好地体现了"用 Triton 写 kernel"的取舍：控制力不如 CUDA，但样板代码少得多。 – **为什么一个线程处理 8 个元素**：生成的 PTX 显示 thread coarsening——Triton/编译器做了向量化，让一个线程处理多个元素，提高指令级并行与访存吞吐。

与作业的联系：作业 2 要求实现**融合的 RMSNorm Triton kernel**，用的正是这个模式（分块逐元素 + mask + 单次读/写）。读 PTX 的练习（ld.global/st.global、%ctaid.x、%tid.x）就是验证 kernel 实际行为的方式。

代码示例：Triton softmax（行内归约）与 row-sum（分片循环）

代码（Python）：

```
@triton.jit
def triton_softmax_kernel(x_ptr, y_ptr, x_row_stride, y_row_stride, num_cols, BLOCK_SIZE: t
    assert num_cols <= BLOCK_SIZE
    row_idx = tl.program_id(0)                      # 一行一个块
    col_offsets = tl.arange(0, BLOCK_SIZE)
    x_ptrs = x_ptr + row_idx * x_row_stride + col_offsets
    x_row = tl.load(x_ptrs, mask=col_offsets < num_cols, other=float("-inf"))
    x_row = x_row - tl.max(x_row, axis=0)          # 减去行最大值（数值稳定）
    numerator = tl.exp(x_row)
    denominator = tl.sum(numerator, axis=0)
    y_row = numerator / denominator
    tl.store(y_ptr + row_idx * y_row_stride + col_offsets, y_row, mask=col_offsets < num_co

@triton.jit
def row_sum_kernel(x_ptr, out_ptr, N, BLOCK_SIZE: tl.constexpr):
    row = tl.program_id(0)
    acc = tl.zeros([BLOCK_SIZE], dtype=tl.float32)   # 每线程一个累加器
    for start in range(0, N, BLOCK_SIZE):            # 遍历列 tile
        cols = start + tl.arange(0, BLOCK_SIZE)
        x = tl.load(x_ptr + row * N + cols, mask=cols < N, other=0.0)
        acc += x
    result = tl.sum(acc, axis=0)                     # 线程间最终归约
    tl.store(out_ptr + row, result)
```

代码做了什么： softmax kernel 每行一个线程块：载入该行（带掩码、用 `-inf` 填充，使填充列的 exp(−inf)=0）、减最大值、取指数、求和、归一化、写回——单 kernel 行 softmax。row-sum kernel 处理*比一个块还长*的行：每个线程跨列 tile 累加，最后用 `tl.sum` 归约各线程的局部和。

实现深挖： – **为什么 softmax 用 `other=float("-inf")`**：填充位置必须在求和中等价于 exp(−∞)=0，且不能影响最大值。row-sum 则用 `other=0.0`。 – **为什么需要循环**：`assert num_cols <= BLOCK_SIZE` 是"行能放进一个块"的假设；当行有 4096 列而 BLOCK_SIZE 为 1024 时，就要按 tile 迭代累加——这是"初级分块"（对归约做分块），也是矩阵乘分块的铺垫。 – **为什么 `tl.sum(acc, axis=0)`**：循环结束后，BLOCK_SIZE 个线程各持有覆盖自己那部分列的部分和；块级归约（共享内存/ warp shuffle，Triton 内部处理）得到标量行和。 – **成本核算**：朴素的 PyTorch softmax 约需 5MN 次读 + 3MN 次写（max、减、exp、sum、除）；融合后的 Triton kernel 只需 MN 读 + MN 写——最多减少约 4 倍显存事务，这对 memory-bound 的 softmax 至关重要。

与作业的联系：这是作业 2 的 FlashAttention-2 的结构模板：分块 + 带掩码载入 + 在线 softmax 累加（第 5 讲的技巧，这里用 `other=-inf` 掩码实现）。理解这个归约循环，才能理解 attention 中那个 `O` 累加器（形状 [BLOCK_M, head_dim] 的滑动加权和）为什么这样写。

代码示例：Triton matmul + ReLU（共享内存分块）

代码（Python）：

```
@triton.jit
def matmul_relu_kernel(
    a_ptr, b_ptr, c_ptr, M, N, K,
    stride_am, stride_ak, stride_bk, stride_bn, stride_cm, stride_cn,
    BLOCK_M: tl.constexpr, BLOCK_N: tl.constexpr, BLOCK_K: tl.constexpr,
):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)
    indices_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    indices_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    indices_k = tl.arange(0, BLOCK_K)

    # A、B tile 的指针网格
    a_ptrs = a_ptr + indices_m[:, None] * stride_am + indices_k[None, :] * stride_ak
    b_ptrs = b_ptr + indices_k[:, None] * stride_bk + indices_n[None, :] * stride_bn

    acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
    for k in range(0, K, BLOCK_K):                          # 沿 K 维遍历 tile
        a = tl.load(a_ptrs, mask=(indices_m[:, None] < M) & (indices_k[None, :] + k < K), o
        b = tl.load(b_ptrs, mask=(indices_k[:, None] + k < K) & (indices_n[None, :] < N), o
        acc += tl.dot(a, b)                                 # 对 tile 做 tensor core 矩阵乘
        a_ptrs += BLOCK_K * stride_ak                       # 推进到下一个 tile
        b_ptrs += BLOCK_K * stride_bk

    acc = tl.maximum(acc, 0.0)                              # 融合 ReLU!
    c_ptrs = c_ptr + indices_m[:, None] * stride_cm + indices_n[None, :] * stride_cn
    tl.store(c_ptrs, acc, mask=(indices_m[:, None] < M) & (indices_n[None, :] < N))
```

代码做了什么： 把 C 划分为 BLOCK_M × BLOCK_N 的输出 tile（每个 tile 一个线程块，二维网格）。每个块沿 K 维以 BLOCK_K 为步长循环：载入 A tile 与 B tile（带边界掩码），把 `tl.dot(a, b)` 累加进输出累加器，最后在 kernel 内先做 ReLU 再写回。

实现深挖： – **为什么要分块**：朴素矩阵乘对每个 (m,n) 都要从 HBM 读 A[m,k] 和 B[k,n]——O(M·K·N) 次读，算术强度 O(1)。把 tile 载入共享内存、在每个 K 分片上复用给整个输出 tile，强度提升到 O(tile_size)；讲义"理想化"版本（全部放共享内存）可达 O(N)。 – **为什么每次载入/写回都要 mask**：网格按 `triton.cdiv(M, BLOCK_M) × triton.cdiv(N, BLOCK_N)` 大小构造，边缘 tile 会越界 M/N/K；（用 `other=0.0`）掩码让越界贡献为 0——这对矩阵乘是数学上正确的填充。 – **为什么用 `tl.dot`**：它会下沉为 tensor core 指令（第 5 讲"矩阵乘比其他浮点运算快 10 倍以上"的硬件）。Triton 会替你选择 MMA 布局；在 CUDA 里你得手动管理 fragment。 – **为什么融合 ReLU**：在写回前做 `tl.maximum(acc, 0.0)` 避免第二次 HBM 往返（读 C、激活、写 C）——融合原则用于矩阵乘。FlashAttention 把 softmax 融进 attention 矩阵乘也是同一手法。 – **为什么用带步长的指针**：矩阵在内存中是线性化的；`index = row*stride_row + col*stride_col`。用 stride（而不是硬编码形状）让 kernel 支持非连续布局，也让同一个 kernel 服务于转置操作数。

与作业的联系：作业 2 的 FlashAttention-2 Triton kernel 就是同一结构（BLOCK_M × BLOCK_N 输出 tile、K 维循环配 `tl.dot`、掩码），再加上在线 softmax、mask/bias 处理，以及重算注意力分数的反向传播。FlashAttention 的前向"确实就是 KQV 矩阵乘的分块"（第 5 讲）——softmax 是额外的那部分。

关键要点

1. 配方是：benchmark → profile → 改 → 再 benchmark。benchmark 给端到端时间；profiling 告诉你哪些 kernel 占主导以及它们叫什么名字。
2. 理解硬件（SM、warp、占用率、bank conflict、合并访问、波量化）——即使代码正确，它仍决定性能。
3. Triton 的思维单位是线程块而非线程：从 HBM 载入 tile → 在共享内存/寄存器里计算 → 写回 HBM。把逐元素算子融进 kernel 以避免往返。
4. 四类典型 kernel：逐元素（GeLU）、行归约（softmax）、分片归约（row sum）、分块矩阵乘（matmul+ReLU）——真实 kernel 都是它们的组合。
5. 融合与分块是把 memory-bound 代码变成 compute-bound 代码的两大杠杆；HBM 流量下降时 MFU 就上升。

常见陷阱

- **忘记掩码**：越界的读/写会造成静默损坏或崩溃；掩码的 `other=` 填充值必须对当前算子是数学中性的。
- **benchmark 里漏掉 `torch.cuda.synchronize()`**：你会测到异步启动开销而不是 kernel 时间。
- **填充值用错**：softmax 用 `-inf`（使 exp→0），matmul/sum 用 `0.0`——用错会在边界上给出错误结果。
- **tile 布局导致 bank conflict**：朴素的共享内存布局会在同一 bank 上串行；按真实 FlashAttention 实现那样做 swizzling（行列异或）。
- **网格尺寸不整除 SM 数**：波量化会让 SM 闲置；需要调块数。
- **非连续输入**：kernel 往往断言 `is_contiguous()`；直接传入转置张量而不处理 stride 会破坏合并访问或正确性。
- **精度漂移**：即使输入是 bf16，也要用 fp32 累加（`acc = tl.zeros(..., dtype=tl.float32)`）；作业也强调结果要在容差内匹配 PyTorch 参考实现。

复习题

1. **问：** 融合后的 Triton GeLU 与朴素 PyTorch 表达式 `0.5*x*(1+tanh(...))` 数学上相同，为什么快这么多?
2. **答：** 朴素版本会启动很多 kernel（乘、加、tanh、加、乘），每个都要把整个张量读写一遍 HBM。融合 kernel 只做一次读、在寄存器里算完、一次写——对逐元素运算而言 HBM 流量（瓶颈所在）降低约 4–5 倍。
3. **问：** 在矩阵乘 kernel 中，为什么用 `other=0.0` 掩码载入 A 能在 M 不整除 BLOCK_M 时保持正确?
4. **答：** 填充行的点积贡献为 0，所以越界行的累加器是 0；最后的带掩码写回根本不写这些行。0 是加法单位元，使填充在数学上无害。
5. **问：** 什么是"波量化"? 如果 kernel 在 N=1793 时性能骤降，你会怎么修?

6. **答：** 线程块按波次调度到 SM；若 tile 数不整除 SM 数，最后一个波次会部分闲置。修法：调整 tile 大小/块数使网格整除 SM 数，或对问题做 padding 使 tile 对齐。

第 7 讲：多 GPU 并行

日期：4 月 20 日（周一，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_07.py`

概览

上周讲的是**单卡内部**的并行（融合、分块），本周讲**跨 GPU** 的并行：集合通信操作（broadcast、scatter、gather、reduce、all-gather、reduce-scatter、all-reduce、all-to-all）、硬件连接（NVLink/NVSwitch vs Infiniband vs Ethernet、RDMA），以及用 `torch.distributed` 在深层 MLP 上从零实现三种经典策略——**数据并行**（切 batch）、**张量并行**（切宽度）、**流水线并行**（切深度）。

核心概念与定义

- **统一主题**：计算（ALU）离数据太远。单卡内靠融合/分块减少 HBM 访问；跨卡靠复制/分片减少网络流量。层次结构：L1/共享内存（最快）→ HBM → NVLink/NVSwitch（节点内）→ Infiniband/Ethernet（节点间，最慢）。
- **为什么要多 GPU?** (1) 参数 + 优化器状态 + 梯度 + 激活放不进单卡；(2) 更多 GPU = 更多 FLOPs = 训练更快。
- **集合通信操作**（1980 年代并行计算文献中的经典概念；描述的是"跨设备的通信**模式**"，而非点对点消息）：
- **Rank** = 设备编号（0..world_size−1）；**world size** = 设备总数。
- **Broadcast**：从 rank 0 复制到所有 rank（例：rank 0 加载 checkpoint 后广播）。
- **Scatter**：把 rank 0 上的一个张量分给各 rank（理解 reduce-scatter 的踏脚石）。
- **Gather**：把各 rank 的片段收集到 rank 0（理解 all-gather 的踏脚石）。
- **Reduce**：用某个算子（sum/min/max）把各 rank 的数据合并到 rank 0。
- **All-gather**：收集到**所有** rank（用途：每个 rank 持有参数分片，前向时聚合出完整参数）。
- **Reduce-scatter**：先按维度做归约、再把结果散开（用途：反向之后汇总各数据分片的梯度，但把存储分摊出去）。
- **All-reduce** = reduce-scatter + all-gather（用途：汇总梯度同时保持完整参数副本——即普通 DDP）。
- **All-to-all**：每个 rank 向其他每个 rank 发送一份数据（用途：MoE 的 token 路由；均衡切分时它看起来就是一次转置）。
- *记忆技巧*：reduce = 结合/交换运算；scatter 是 gather 的逆；all = 目的地是所有设备。
- **硬件**：PCIe（家用：242 GB/s）、Ethernet（约 200 MB/s，需经过 CPU）、NVLink→NVSwitch（B200：1.8 TB/s）、Infiniband（约 0.05 TB/s，经 HCA/NIC）。**RDMA** 让一张 GPU 直接读写另一张 GPU 的显存而不惊动 CPU（Infiniband 支持；标准 Ethernet 不支持；RoCE 是"以太网 + RDMA"，

Meta 在用）。NCCL 把集合操作翻译成底层包、探测拓扑并启动收发 kernel。GB200 NVL72 把 72 张 GPU 放在同一个 NVLink 域内。

- **数据并行（DDP）**：每个 rank 拿 batch 的一个切片；每个 rank 持有**完整参数副本**；本地反向之后 all-reduce(AVG) 梯度，使各 rank 保持同步。
- **张量并行**：每个 rank 持有**每层权重的一个切片**（例如 MLP 的列切片 W_i）；所有 rank 处理完整 batch；每层结束后 all-gather 部分激活并拼接。因为每层都要通信，需要极快的互联（NVLink）。
- **流水线并行**：每个 rank 持有**一部分层**（一个 stage）；激活从 rank 0 → 1 → … 流动，用 micro-batch 填满流水线（缩小"气泡"）。能容忍慢互联（点对点、激活大小的通信），但 batch 不大时气泡会拖累性能。
- **流水线气泡**：stage 数为 n、micro-batch 数为 m 时，闲置比例 ≈ (n−1)/m——"所以我们需要很大的 batch！"
- **本讲未覆盖（明确列出）**：通信/计算重叠、attention 相关的并行、序列/专家并行，以及"下次课"的 FSDP/ZeRO（用 all-gather + reduce-scatter 避免持有全部参数）。

代码示例：`torch.distributed` 中的集合通信

代码（Python）：

```
import torch.distributed as dist

def collective_operations_main(rank: int, world_size: int):
    setup(rank, world_size)  # init_process_group("nccl"/"gloo", ...)

    data = tensor([0., 1, 2, 3], device=f"cuda:{rank}") + rank
    dist.all_reduce(tensor=data, op=dist.ReduceOp.SUM, async_op=False)  # 原地修改!
    # 之后: 每个 rank 都拿到"所有 rank 向量之和"

    input = torch.arange(world_size, dtype=torch.float32) + rank   # 每 rank 一个 [world_siz
    output = torch.empty(1)
    dist.reduce_scatter_tensor(output=output, input=input, op=dist.ReduceOp.SUM)
    # output[rank] = 各 rank 第 rank 列之和

    input = output
    output = torch.empty(world_size)
    dist.all_gather_into_tensor(output_tensor=output, input_tensor=input)
    # output = [第 0 列之和, 第 1 列之和, ...] (每个 rank 都拿到)

    cleanup()  # destroy_process_group()
```

代码做了什么： 在 `world_size` 个独立进程（通过 `mp.spawn`）中运行同一函数；每个进程依次执行 all-reduce、reduce-scatter、all-gather，演示了"all-reduce = reduce-scatter + all-gather"（两者输出一致）。

实现深挖： – **为什么是原地操作**：`dist.all_reduce(tensor=data, ...)` 会就地修改 `data`（既是输入也是输出），避免额外分配——这一点常让人意外。 – **为什么区分 NCCL 与 gloo**：NCCL 是 GPU 后端（会最

优利用 NVLink/Infiniband)；gloo 用于 CPU。讲义的 `setup` 依据 `torch.cuda.is_available()` 选择。 – **为什么 `async_op=False`**：同步集合操作会阻塞到完成；要做重叠就得用 `async_op=True` 并收集 handle——作业 2 明确要求实现异步重叠。 – **为什么 world_size=4 时用 spawn**：每个 rank 是独立 OS 进程、各占一张 GPU；通过 `MASTER_ADDR/MASTER_PORT` 由 rank 0 协调。

与作业的联系：作业 2 的分布式部分——DDP、优化器状态分片、FSDP——都建立在这些原语之上：梯度 all-reduce（DDP)、reduce-scatter（状态分片)、all-gather + reduce-scatter（FSDP)。讲义测量带宽的代码（all-reduce 的 `sent_bytes = size_bytes * 2 * (world_size-1)`）就是你 benchmark 自己实现时要用的模式。

代码示例：数据并行（最简 DDP）

代码（Python)：

```
def data_parallelism_main(rank, world_size, data, num_layers, num_steps):
    setup(rank, world_size)
    batch_size = data.size(0)
    local_batch_size = int_divide(batch_size, world_size)     # 切分 batch
    data = data[rank*local_batch_size:(rank+1)*local_batch_size].to(f"cuda:{rank}")

    params = [get_init_params(num_dim, num_dim, rank) for _ in range(num_layers)]
    optimizer = torch.optim.AdamW(params, lr=1e-3)            # 每 rank 自己一份

    for step in range(num_steps):
        x = data
        for param in params:
            x = x @ param
            x = F.gelu(x)
        loss = x.square().mean()

        loss.backward()

        # 与单卡训练唯一的区别:
        for param in params:
            dist.all_reduce(tensor=param.grad, op=dist.ReduceOp.AVG, async_op=False)

        optimizer.step()
    cleanup()
```

代码做了什么： 每个 rank 用自己的 batch 切片算损失（各 rank 的 loss 不同)，然后在 step 之前用 all-reduce(AVG) 把梯度跨 rank 平均——于是所有 rank 的参数演化完全一致。

实现深挖： – **为什么用 AVG 而不是 SUM**：每个 rank 的梯度是其本地 batch 的均值；取平均才能保持对"全局 batch 梯度"的无偏估计。 – **为什么说这是"朴素"DDP**：每步一次阻塞 all-reduce，通信量 2×#params。讲义指出"下次：FSDP/ZeRO——用 all-gather 和 reduce-scatter 避免在显存里保存全部参数"。每 rank 显存：参数 + 梯度 + 优化器状态（bf16/AdamW 下 12+ 字节/参数)。 – **为什么 MLP 有代表性**："MLP 是 Transformer 的计算瓶颈"——这些模式可以直接迁移。

与作业的联系：作业 2：实现带反向 hook 与异步通信的分布式数据并行（讲义的阻塞 all-reduce 是正确性基线，作业要求重叠）。这就是"DDP 必须做什么"的参考——其余都是工程细节。

代码示例：张量并行（前向）

代码（Python）：

```
def tensor_parallelism_main(rank, world_size, data, num_layers):
    setup(rank, world_size)
    data = data.to(f"cuda:{rank}")            # 所有 rank 都有完整 batch
    batch_size, num_dim = data.shape
    local_num_dim = int_divide(num_dim, world_size)   # 切宽度

    params = [get_init_params(num_dim, local_num_dim, rank) for _ in range(num_layers)]

    x = data
    for layer in range(num_layers):
        x = x @ params[layer]                 # 只用本 rank 的列切片
        x = F.gelu(x)

        activations = [torch.empty(batch_size, local_num_dim, device=f"cuda:{rank}")
                       for _ in range(world_size)]
        dist.all_gather(tensor_list=activations, tensor=x, async_op=False)  # 收集各切片
        x = torch.cat(activations, dim=1)     # 还原成完整宽度
    cleanup()
```

代码做了什么： 每个 rank 持有每层权重的一个列切片；完整 batch 依次通过各 rank 的切片，每层结束后把部分激活 all-gather 并拼接，得到下一层所需的完整宽度张量。

实现深挖： – **为什么每层都要 all-gather**：下一层的矩阵乘需要**完整**激活；张量并行用"每层一次 all-reduce（8·b·s·h·(n−1)/n 每层）"换来权重的分片能力——这正是它必须依赖 NVLink 级带宽、通常只在节点内（≤8 卡）使用的原因。 – **为什么反向是镜像关系**：前向里 f 是恒等、g 是 all-reduce；反向里 f 变成 all-reduce、g 变成恒等——讲义把它作为"课后练习"，而第 8 讲幻灯片给出了模式：QKV/up-projection 做列并行，attention 输出/down-projection 做行并行。 – **为什么显存线性扩展**：每个 rank 只存每个权重矩阵的 1/world_size，因此参数显存随设备数线性下降。

与作业的联系：作业 2 的 FSDP 部分是这种显存分片思路的表亲；张量并行本身不在必修作业内（属第 8 讲内容），但你为 FSDP 写的 all-gather 机制用的是同一个原语。

代码示例：流水线并行（带 micro-batch）

代码（Python）：

```
def pipeline_parallelism_main(rank, world_size, data, num_layers, num_micro_batches):
    setup(rank, world_size)
    data = data.to(f"cuda:{rank}")
    batch_size, num_dim = data.shape

    local_num_layers = int_divide(num_layers, world_size)   # 切深度
    local_params = [get_init_params(num_dim, num_dim, rank) for _ in range(local_num_layers

    micro_batch_size = int_divide(batch_size, num_micro_batches)
    if rank == 0:
        micro_batches = data.chunk(chunks=num_micro_batches, dim=0)   # 数据源
    else:
        micro_batches = [torch.empty(micro_batch_size, num_dim, device=f"cuda:{rank}")
                         for _ in range(num_micro_batches)]

    for x in micro_batches:
        if rank - 1 >= 0:
            dist.recv(tensor=x, src=rank - 1)              # 从上一 stage 收激活
        for param in local_params:                          # 算我负责的层
            x = x @ param
            x = F.gelu(x)
        if rank + 1 < world_size:
            dist.send(tensor=x, dst=rank + 1)               # 传给下一个 stage
    cleanup()
```

代码做了什么： rank 0 持有数据；每个 rank 计算自己 stage 的层，并把激活发给下一个 rank。micro-batch 让 rank 1 可以在 rank 0 还在处理 micro-batch 1 时就开始算 micro-batch 0 的后半段——从而填满流水线气泡。

实现深挖： – **为什么要 micro-batch**：没有它，同一时刻只有一个 rank 在工作（利用率 1/n）。有 m 个 micro-batch 时气泡比例约 (n−1)/m——"所以需要大 batch！" – **为什么用点对点 send/recv**：流水线通信是"激活大小"且只发生在相邻 stage 之间——开销小到足以跑在慢速互联（节点间）上，这就是为什么流水线并行常跨机器、而张量并行留在节点内。 – **为什么这里不做重叠**：讲义明确不重叠通信与计算（"未处理：通过重叠通信/计算消除流水线气泡"）——工程实现里是带异步发送的 1F1B 调度。

与作业的联系：流水线并行不在必修作业中实现（作业 2 是 DDP → FSDP），但理解气泡数学能解释：为什么作业 2 的 FSDP 一定要做通信重叠，以及为什么讲义说"流水线能容忍慢互联，但需要花功夫减小气泡"。

关键要点

1. 切分模型的方式很多：数据（batch）、张量/专家（宽度）、流水线（深度）、序列（长度）——各自的通信模式与硬件要求不同。
2. 原语词表——broadcast/scatter/gather/reduce 及其 "all-" 变体——是分布式训练的共同语言；all-reduce = reduce-scatter + all-gather，而这个分解正是 ZeRO/FSDP 得以成立的原因。
3. 数据并行（DDP）：all-reduce 梯度、复制参数——简单，但通信 2×#params 且显存不随设备数下降。
4. 张量并行：切分权重、每层 all-gather 激活——需要 NVLink；流水线并行：切分层、点对点传激活——能容忍慢网络但要付气泡代价。

5. 需要最小化的是**通信带宽**（与第 5/6 讲最小化 HBM 带宽是同一个原则）；并用计算/通信重叠把它藏起来。

常见陷阱

- **阻塞式集合操作串行化流水线**：到处 `async_op=False` 会扼杀吞吐；要重叠并保存 handle。
- **数据切分不均匀**：`int_divide` 断言 `a % b == 0`——不均衡切分破坏 DDP 梯度平均的语义。
- **benchmark 忘记 `dist.barrier()`**：各 rank 抢跑，计时里混入 straggler 偏斜。计时的集合操作两侧都要 barrier + synchronize。
- **all-gather 形状不匹配**：tensor list 必须预先分配精确输出形状；`all_gather_into_tensor` 可免去手工管理。
- **原地 all-reduce 的意外**：输入张量会被覆盖——如需保留归约前的值要先拷贝。
- **在慢链路上用重通信的集合操作**：例如跨节点用以太网做逐层 all-reduce（张量并行）会完全支配运行时间；策略要与互联能力匹配。
- **跨 rank 的随机种子控制**：`get_init_params` 手动设了种子；不加控制会导致某些策略下初始化不一致（张量并行要求各权重切片语义一致，DDP 要求各 rank 初始参数相同）。可复现性很重要。

复习题

1. **问：** 为什么流水线并行能容忍慢互联，而张量并行不能?
2. **答：** 流水线只在相邻 stage 之间传"激活"——每个 micro-batch O(b·s·h) 的点对点通信，与模型宽度无关。张量并行**每一层**都要 all-reduce 激活大小的张量——每层约 8·b·s·h·(n−1)/n，大致是流水线的 8 倍——所以它必须靠 NVLink 级带宽，否则通信将成为瓶颈。
3. **问：** 在 DDP 中，为什么梯度要取平均（AVG）而不是求和?
4. **答：** 每个 rank 计算的梯度是其本地数据切片上的均值；跨 rank 求与会把有效学习率放大 world_size 倍。取平均才能复现全局 batch 的梯度。
5. **问：** 什么是流水线气泡？micro-batch 如何缩小它?
6. **答：** 流水线一次通过的开头和结尾，各 stage 会在第一/最后一个 micro-batch 排空时闲置——闲置比例约 (n_stages−1)/m（m 为 micro-batch 数）。更多 micro-batch（更大的逻辑 batch）能填满流水线、摊薄气泡。

第 8 讲：并行基础（系统细节）

日期：4 月 22 日（周三，Spring 2026）| 讲师：Tatsu Hashimoto | 材料：`lecture_08.pdf`（第 7 讲的并列 PDF 位于私有仓库，标注为受限资源）

概览

第 7 讲搭出了最简分布式代码，本讲给出系统层深挖：网络基础（为什么不能把所有设备全连起来）、并行策略全景——朴素 DDP、ZeRO 第 1–3 级（FSDP）、流水线、张量、序列/上下文、专家并行——并为每种策略做显存与通信量核算，最后总结真实大规模训练（DeepSeek、Llama 3 405B、Gemma 2、Mixtral、Qwen 3、Nemotron）如何组合它们（"3D/4D 并行"）。

核心概念与定义

- **为什么要多 GPU**：单卡扩展同时受算力（世界最快超算也不过 exaflops 级）与显存（大模型装不下）限制。并行把显存**和**算力一起摊到多卡/多机。节点内用高速互联，节点间走网络。
- **网络拓扑**：TPU 用环形网格（toroidal mesh，便宜，非常适合张量并行）；GPU 用 all-to-all/树形拓扑（更适合非结构化通信，如专家并行）。TPU8i/8t 正向树形/交换网络演进（为 MoE）。**为什么不把所有东西都连起来？** 成本——域大小与物理限制。
- **朴素 DDP 的显存核算**：每个参数约需 16 字节：2（bf16 参数）+ 2（bf16 梯度）+ 4（fp32 主权重）+ 4 + 4（Adam 一阶/二阶矩）——即"权重的 5 份副本"问题。这就是 DDP 显存不随设备数扩展的原因。
- **ZeRO（Zero Redundancy Optimizer）各级**——把冗余副本分片：
- **第 1 级——优化器状态分片**：把一阶/二阶矩分散到各 GPU。步骤：算出完整本地梯度 → **reduce-scatter** 梯度（每个 rank 拿到自己负责的切片）→ 只更新自己那片参数 → **all-gather** 更新后的参数。通信 = 2×#params（与 DDP 的 all-reduce 相同！），显存 = (4 + K/N_gpu)×#params。*"ZeRO 第 1 级是免费的（在带宽受限区间）——显存收益白拿。"*
- **第 2 级——再加梯度分片**：梯度也分片；反向过程中一旦某个梯度被归约完就立即释放（从不物化完整梯度向量）。
- **第 3 级 = FSDP——连参数都分片**：参数在前向/反向时按需 all-gather，用完即释放。通信 = 3×#params（DDP 的 1.5 倍），但纯 bf16 训练下每 rank 显存降到 12/8 字节每参数。关键技巧：**增量式通信/计算重叠**——在计算 W0 的同时去 all-gather W1、W2，把通信开销藏起来。
- *类比*：DDP 像每个图书馆都藏一整套全书的副本；ZeRO-3 像图书馆联盟——每个分馆只保留一个书架，谁需要就把书调来（all-gather）、用完就还回去（释放）。
- **模型并行**（分片参数，通信**激活**——而 ZeRO-3 通信的是参数）：
- **流水线并行（逐层）**：朴素的逐层并行利用率极差（每张 GPU 只有 1/n 时间在工作）。micro-batch 解决它；气泡 ≈ (n_stages−1)/n_micro。通信特性好（点对点、激活大小），用于节点间；性能高度依赖 batch 大小。"Zero-bubble"变体把反向拆成"激活反向传播"与"权重梯度计算"两部分。
- **张量并行**：沿宽度切分矩阵乘。前向：f = 恒等，g = all-reduce（把部分和相加）；反向：f = all-reduce，g = 恒等。QKV/up-projection 做列切分；attention 输出/down-projection 做行切分；norm/router 复制。通信：每层 8·b·s·h·(n−1)/n（all-reduce），对比流水线的 b·s·h 点对点。用在互联快的地方（节点内，≤8 卡）。优点：没有气泡、复杂度低、不需要大 batch。
- **序列/上下文并行**：把**序列**维度切开，用于逐点运算（LayerNorm、dropout）与长上下文（ring attention），让激活显存随设备数下降。

- **专家并行**：把专家切到不同设备，用 all-to-all 路由 token（仅用于 MoE 的 MLP 部分）。需要每个专家有足够多的 token 才高效。
- **激活显存**：隐藏的主导项——即使参数分片做得很完美，激活仍可能压垮显存（例如包含 dropout 的二次 attention 项约 5·a·s·h，可通过重计算消除；LayerNorm/dropout 的 10·s·b·h 项可通过序列并行消除）。
- **组合策略——"3D 并行"经验法则**：(1) 在模型装得下之前：节点内做张量/专家并行，跨机器做流水线（或按带宽情况用 ZeRO-3）；(2) 剩下的一路用数据并行扩；若 batch 太小，就用梯度累积，以更大 batch 换更高通信效率。示例（Narayanan 等 2021）：先 TP=8，再用 PP 让模型装下，DP 随规模增大而缩小（DP：32→32→32→24→15→9→6）。
- **真实配方**：DeepSeek v3：ZeRO-1 + TP=1 + EP=64 + PP=16；Llama 3 405B：DP=128、TP=8、PP=16；Gemma 2：ZeRO-3 + MP(TP+SP) + DP=768；Mixtral 8x22B（Megatron）：TP/PP/CP/EP = 4/4/1/8；Nemotron 3 120B：TP=2、EP=64、CP=64；Qwen 3：EP=32、TP=2、PP=8。

代码示例：ZeRO-1（优化器状态分片）概念实现

代码（Python）：

```
# 每个训练步，world_size 台设备，每台只负责 params[my_slice]：
# 第 1 步：每台设备在本地 batch 上算出完整梯度
loss.backward()                          # 每 rank 的 param.grad 都是完整大小

# 第 2 步：reduce-scatter 梯度 -> 每个 rank 只保留自己那一份切片
grad_slices = [torch.empty_like(grad_chunk) for ...]
dist.reduce_scatter_tensor(output=my_grad_slice, input=full_grad,
                           op=dist.ReduceOp.SUM)

# 第 3 步：每台设备只用自己那片的梯度 + 状态，更新自己负责的参数
for i in my_param_indices:
    state[i].m1 += ...                   # fp32 矩只存在于本 rank
    state[i].m2 += ...
    params[i] -= lr * update(my_grad_slice_i, state[i])

# 第 4 步：all-gather 更新后的参数，让每台设备都有完整模型
dist.all_gather_into_tensor(output=full_params, input=my_param_slice)
```

代码做了什么： 勾画 ZeRO-1 的循环：完整本地梯度 → reduce-scatter → 在本地更新一部分参数 → all-gather 刚更新好的参数。

实现深挖： – **为什么通信量仍然是 2×#params**：reduce-scatter 送 #params，all-gather 送 #params——恰好等于一次 all-reduce 的数据量。在带宽受限区间，ZeRO-1 相对 DDP 是"免费的"，同时把优化器状态显存降低 N_gpu 倍。 – **为什么"先更新再聚合"而不是"先聚合再更新"**：每个 rank 只需要自己那片的梯度与矩，因此在 all-gather 之前，更新是完全并行的。 – **为什么这是作业 2"优化器状态分片"的基础**：作业正是要求你实现 reduce-scatter 梯度、本地 AdamW 更新、all-gather 参数——算法完全相同。

与作业的联系：作业 2 任务：(4) 分布式数据并行（all-reduce）；(5) 优化器状态分片（reduce-scatter + all-gather，如上）；(6) FSDP（连参数都分片，按需 all-gather 并重叠）。讲义"ZeRO 第 3 级是 3×#param

——1.5 倍通信开销，但不算差"的分析，正是你 FSDP 实现与实验报告要复现的内容。

代码示例：FSDP 风格的分片前向（概念）

代码（Python）：

```
# 每个 rank 只保存每个权重 W_l 的一个分片。
# 要用第 l 层时，先聚合出完整权重，用完释放：
def fsdp_forward(x, layers, rank, world_size):
    for layer in layers:
        # 1. all-gather 本层的权重分片 -> 每个 rank 拿到完整 W
        full_W = all_gather(layer.weight_shard[rank])
        # 2. 计算（可与下一次 all-gather 重叠）
        x = x @ full_W
        x = F.gelu(x)
        # 3. 释放 full_W（只保留分片）
    return x
```

代码做了什么： 展示 FSDP 的"按需物化参数"：只在需要时聚合某一层的权重，算完即丢——因此显存峰值只包含"一层完整权重"，而非整个模型。

实现深挖： – **为什么重叠是关键**：如果聚合是阻塞的，FSDP 会比 DDP 更慢；通过在当前矩阵乘执行时**同时**发起下一次 all-gather（增量式通信/计算），通信开销被掩盖——例如 `(W1W0 + W2W0)x = y` 在算 W0 时就把 W1、W2 聚好。 – **为什么通信量是 3×#params**：两次 all-gather（前向与反向的参数物化）+ 一次 reduce-scatter（梯度）。讲义指出这是 DDP 流量的 1.5 倍，但换来显存随设备数线性下降。 – **为什么"概念上非常简单——写个 FSDP block wrapper 就行"**：魔法在于把每个 module 包起来，让它的参数透明地分片/聚合；作业 2 要求的正是这样一个包裹 `torch.nn.Module` 的 `FSDP` 类。

与作业的联系：作业 2 的 FSDP 任务（全分片数据并行训练，含前向/反向的 gather 与梯度 reduce-scatter，并与 DDP 对比 benchmark）就是这个设计。"装得下吗？"表格（在 8×A100-80G、每参数 12 字节下：基线 6.67B → ZeRO-1 16B → ZeRO-2 24.6B → ZeRO-3 53.3B）就是你在报告中要复现的显存算术。

关键要点

1. 朴素 DDP 显存效率低（约 16 字节/参数）；ZeRO 第 1→2→3 级依次分片优化器状态、梯度、参数——第 3 级只多 1.5 倍通信，却换来线性显存扩展。
2. ZeRO-1 是"免费的"：通信量与 DDP 相同、显存严格更优——"所以你不如总是开着它"。
3. 模型并行是"分片参数、通信激活"：流水线（深度、点对点、气泡）、张量（宽度、每层 all-reduce、需 NVLink）、序列（长度）、专家（MoE 路由、all-to-all）。
4. 真实训练把上述手段全部组合：节点内 TP ⩽ 8、跨节点 PP、其余用 DP、MoE 层用 EP、长上下文用 CP——并且处处做通信/计算重叠。
5. 显存是动态的：激活常常超过参数；重计算与序列并行是主要杠杆。

常见陷阱

- **显存受限却仍用 DDP**：每 rank 复制整个模型；应改用 ZeRO-1/2（几乎免费）或 FSDP（1.5 倍通信）。
- **不做通信与计算重叠**：无重叠的 FSDP 在墙钟时间上严格劣于 DDP；它的全部意义就是把 gather 延迟藏起来。
- **阻塞式流水线发送**：没有 1F1B 式调度与异步操作，流水线会停摆。
- **在慢链路上做 TP**：跨以太网的逐层 all-reduce 会摧毁吞吐；TP 应限制在节点内。
- **忽略激活显存**：参数分片做到完美仍可能因为激活而 OOM；要用序列并行 + 重计算。
- **天真地组合 DP 与 EP**：DP 通常与 EP 共享副本（因此 EP < DP），而 DP 与 TP 组合不当会降低利用率。

复习题

1. **问：** 为什么说 ZeRO-1 相对 DDP 是"免费的"?
2. **答：** 它的通信量是 2×#params——与 DDP 那一次 all-reduce 相同——因为 reduce-scatter + all-gather 的总搬运字节数与 all-reduce 相同。但显存从 (4+K)×#params 降到 (4+K/N_gpu)×#params。带宽代价相同，显存严格更优。
3. **问：** FSDP 聚集什么、什么时候聚集？为什么 3×#params"不算差"?
4. **答：** FSDP 在前向和反向中按层按需 all-gather 参数分片（2×#params），并对梯度做 reduce-scatter（1×#params）：合计 3×#params，是 DDP 的 2×#params 的 1.5 倍——但它把**所有**显存都分片了（参数、梯度、优化器状态，配合序列并行还包括激活）。
5. **问：** 为什么张量并行比流水线并行需要更快的互联?
6. **答：** TP 每层都要通信（激活大小的张量做 all-reduce，每层 $8 \cdot b \cdot s \cdot h \cdot (n-1)/n$）；流水线只在 stage 之间通信（每 micro-batch 一次 $b \cdot s \cdot h$ 的点对点）。TP 的逐层、近似全连接式流量需要 NVLink；流水线的稀疏点对点可以跑在 Infiniband 上。

第 9 讲：扩展律基础（Scaling Laws — Basics）

日期：4 月 27 日（周一，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_09.pdf` *| 截止：作业 2 到期*

概览

如果你拿到 10000 张 B200 用一个月，*到底该训哪个模型？* 本讲引入**扩展律（scaling laws）**——把损失与数据量、模型规模、算力联系起来的简单、可预测的幂律规则——从而可以在小模型上调参并外推到大规模。内容包括：历史脉络（从 1993 年的样本复杂度研究到 Hestness 2017）、*为什么*会出现幂律的理论（估计误差、内在维度）、经典的 Kaplan 与 Chinchilla 结果（含著名的 N 与 D 之争及其分歧原因）、critical batch size、muP，以及实用的"扩展律设计流程"。

核心概念与定义

- **扩展律**：把某种资源（数据量 n、参数量 N、算力 C）映射到损失/误差的简洁公式，例如 `Loss ≈ C·n^(−α)`。在对数–对数坐标下呈直线："无标度（scale-free）"或幂律行为。
- *类比*：就像衡量"多修几条车道能让通勤快多少"——存在干净的幂律趋势（车道翻倍带来固定比例的提速），于是你可以用 2/4/8 车道的实测结果预测 16 车道的效果。
- **为什么是幂律？（理论）**：估计误差多项式衰减。玩具例子：用 n 个 i.i.d. 样本估计均值，$E[(\hat{\mu}-\mu)^2] = \sigma^2/n$——这就是斜率 −1 的扩展律。d 维非参数回归的误差约 n^(−1/d)：**斜率依赖维度**。因此扩展律指数与数据的（内在）维度相关——这是活跃研究领域（Bahri 2021），但内在维度的估计方法并不可靠。
- **经验事实**：损失与数据量在 log–log 下呈线性，跨语言模型/机器翻译/语音都成立；数据**构成**影响截距（偏移）而非斜率（distribution-shift 扩展律，Hashimoto 2021）——说明数据多样性很重要。观测到的斜率与经典 1/n 预测不同——一个"谜团"。
- **数据重复（repetition）**：重复使用有限数据会降低其价值；有效数据量 D' < 唯一 token 数——因此数据选择应当**随规模自适应**。
- **用扩展律做模型工程**：架构选择（大规模下 Transformer 优于 LSTM）、优化器（Adam vs SGD）、深度/宽度、batch size、学习率——都可以从小模型实验得出。**重要警示**：下游任务的扩展性通常比预训练损失**更难预测**。
- **Critical batch size（临界 batch size）**：出现收益递减之前的最小 batch；做法是拟合 S_min/E_min 曲线（约为朴素最优步数/样本数的 2 倍；据称与"梯度协方差迹 / 梯度范数平方"的比例有关）。目标损失越小，临界 batch 越大。
- **muP（maximal update parametrization，最大更新参数化）**：宽度感知的初始化 + 学习率缩放，使最优超参数能跨模型规模迁移（细节见第 11 讲）。
- **N 与 D 之争**：给定算力 C = 6ND，该训更大的模型（N）还是更多 token（D）？
- **Kaplan 等 2020**：N_opt ∝ C^0.73，D_opt ∝ C^0.27——**每参数 token 数随算力下降**（模型更大、数据相对更少）。
- **Chinchilla（Hoffmann 等 2022）**：N_opt ∝ C^0.5，D_opt ∝ C^0.5——**算力最优**约为 D = 20N（70B 参数应对应约 1.4T token）。
- *分歧原因*：Kaplan 的计数问题（是否剔除最后一层、小预算下 warmup 过高）、非嵌入参数 vs 全参数的取舍、小规模非线性。一项"数据取证"式再分析（Besiroglu 等 2024）发现 Chinchilla 的方法 3 本身也有缺陷，恢复原始数据重新拟合后与它的方法 1/2 一致。
- **Chinchilla 的三种拟合方法**：(1) 取所有训练曲线的最小值（下包络）；(2) **IsoFLOPs**（固定算力 C_i、扫模型规模、取最小损失；〈C_i, N_opt〉构成幂律）；(3) 在"规模×数据"网格上做联合最小二乘拟合。
- **训练最优 ≠ 部署最优**：Chinchilla 优化的是**训练**算力，但真实算力开销主要在**推理**上——所以模型越来越被"过度训练（over-trained）"：GPT-3：2 token/参数，Chinchilla：20，LLaMA-65B：22，Llama 2 70B：29，Mistral 7B：110，Llama 3 70B：215。模型被用得越多，越值得为前期成本多训。
- **扩展律设计流程**：(1) 先训练几个小模型；(2) 建立扩展律（例如 Adam vs SGD）；(3) 依据扩展律的预测选择超参数——*"大模型上超参数的影响可以在训练之前就被预测出来！"*
- **IsoFLOPs 无处不在**：该方法可迁移到扩散模型、MoE（稀疏度扩展律）等。

代码示例：IsoFLOPs 扩展律拟合（作业 3 的核心）

代码（Python）：

```
import numpy as np

# 数据：对每个算力预算 C_i，训练不同规模的模型 N_ij
# （token 数 D_ij = C_i / (6 * N_ij)），记录最终损失 L_ij。
# runs = [(C_i, N_ij, L_ij), ...]

def isoflop_optima(runs):
    optima = []  # (C_i, N_opt(C_i), D_opt(C_i))
    for C_i in sorted(set(r for r, _, _ in runs)):
        subset = [(n, l) for r, n, l in runs if r == C_i]
        N_opt, L_min = min(subset, key=lambda nl: nl[1])   # 该 isoflop 曲线上的最小损失
        D_opt = C_i / (6 * N_opt)                          # 6ND 法则
        optima.append((C_i, N_opt, D_opt))
    return optima

def fit_power_law(xs, ys):
    # log y = log a + b log x  ->  在对数空间做线性回归
    logx, logy = np.log(np.array(xs, dtype=float)), np.log(np.array(ys, dtype=float))
    b, loga = np.polyfit(logx, logy, 1)
    return np.exp(loga), b

# 例：拟合 N_opt = a * C^b 与 D_opt = c * C^d
optima = isoflop_optima(runs)
a, b = fit_power_law([o[0] for o in optima], [o[1] for o in optima])
c, d = fit_power_law([o[0] for o in optima], [o[2] for o in optima])
# 对大预算 C_target 做预测：
N_target = a * C_target ** b
D_target = c * C_target ** d
```

代码做了什么： 对每个固定 FLOPs 预算，找出损失最小的模型规模（即 IsoFLOPs 最优点），用 C = 6ND 反推 token 数，然后在对数空间拟合 N_opt(C) 与 D_opt(C) 两条幂律，并外推到大预算。

实现深挖： – **为什么要在 isoflop 曲线上取最小值**：固定算力时，过小的模型拟合不了数据、过大的模型来不及走足够多步——损失曲线是凸的，其最小值就是该预算下的算力最优配置。 – **为什么在对数空间回归**：幂律在对数空间是直线；`np.polyfit(logx, logy, 1)` 的斜率就是指数。注意：对数空间拟合对"小损失点"的加权与线性空间不同，这是一个已知的微妙点。 – **为什么 C = 6ND 是基础**：第 2 讲的 6ND 法则把 (N, D) 换算成算力——这是所有扩展律计算的支柱。

与作业的联系：作业 3 *就是* 这件事：你拿到一个训练 API（超参数 → 验证损失），用 12 个 B200 小时的预算拟合扩展律，然后提交对 48 B200 小时运行的算力最优超参数预测与最终损失预测。报告必须写清你的 IsoFLOPs（或联合拟合）方法；排行榜按你预测模型的**实测**损失评分。讲义的"Chinchilla 方法 2 = IsoFLOPs"是推荐起点；作业也允许 Kaplan 风格或 muP 思路。

代码示例：critical batch size（概念）

代码（Python）：

```
# 对某个目标损失，扫 batch size；记录达到目标所需的步数 S 与样本数 E。
# 理论（McCandlish 等）：曲线大致满足
#   S(B) = S_min * (1 + B_crit / B)          # 步数随 batch 变化
#   E(B) = S(B) * B                           # 消耗的样本数
# 拟合 S_min 与 B_crit；选使步数与样本数平衡的 B*。
def fit_critical_batch(B_sweep, S_measured):
    # 解 S_min、B_crit：S(B) = S_min * (1 + B_crit / B)
    # （对 1/B 与 S 做最小二乘）
    import numpy as np
    invB = np.array([1.0 / b for b in B_sweep])
    S = np.array(S_measured, dtype=float)
    S_min, S_min_Bcrit = np.polyfit(invB, S, 1)   # S = S_min + (S_min*B_crit) * (1/B)
    B_crit = S_min_Bcrit / S_min
    return S_min, B_crit
```

代码做了什么： 把"batch size → 达标步数"曲线拟合成直线，从而提取临界 batch size B_crit——超过它之后，加大 batch 的收益迅速递减（大致是步数/遍历次数最优点的 2 倍）。

实现深挖： – **为什么会有收益递减**：超过 B_crit 后，batch 翻倍并不能让达标步数减半，等于浪费样本。目标损失越低，临界 batch 越大。 – **为什么它对作业 1/3 重要**：batch size 是一阶训练超参数；扩展性分析（DeepSeek、StepFun，见第 11 讲）都建立在"拟合最优 batch 随规模变化"之上。

与作业的联系：作业 1 的训练循环必须支持可配置 batch（以及梯度累积）；作业 3 的 API 把 batch size 作为输入超参数——你构建的扩展律拟合应当把 batch/LR 当作可调旋钮，正如本讲的临界 batch 分析所暗示的。

关键要点

1. 损失对数据/模型/算力在 log–log 下呈现干净的幂律——足以"小规模调参、大规模外推"（"扩展即预测"）。
2. 理论解释：估计误差多项式衰减（均值估计斜率 −1；d 维非参数回归斜率 −1/d），把扩展指数与数据维度联系起来——但语言模型观测到的斜率仍是一个部分未解之谜。
3. 算力最优训练（Chinchilla）约等于每参数 20 个 token；但部署现实（推理算力占主导）推动模型重度过度训练（多到每参数 200+ token）。
4. IsoFLOPs 是主力方法：固定算力、扫规模、取最小、拟合幂律——可迁移到 MoE、扩散模型，以及你的作业 3。
5. 超参数选择（优化器、深度、架构、batch）都能从小模型扩展律**预测**出来，再花大算力——这正是全课效率思维的体现。

常见陷阱

- **盲目外推**：扩展律本质是下界；脱离拟合区间（例如数据重复、不同架构）就会"失效"。
- **参数计数口径不一致**：含/不含 embedding 参数、是否剔除最后一层——这些差异曾让 Kaplan 与 Chinchilla 相差巨大。要明确 N 的定义。
- **把训练最优当成部署最优**：Chinchilla 的 20 token/参数针对训练算力；若推理占主导，就要过度训练。

- **在对数空间草率拟合**：log 空间最小二乘对各点的权重并不均匀；两种拟合都报出来并检查残差。
- **忽略 batch/LR 的联合调参**：在固定 batch 下拟合的扩展律未必可迁移；batch 与 LR 都对规模敏感。
- **把下游指标等同于预训练损失**：能力型指标（MMLU 等）的扩展性远不如损失可预测——切勿过度宣称。

复习题

1. **问：** Kaplan 与 Chinchilla 为什么在最优 N/D 比例上分歧？实践启示是什么？
2. **答：** Kaplan 得到 N_opt ∝ C^0.73（每参数 token 数随算力下降）；Chinchilla 得到 N_opt ∝ C^0.5、约 20 token/参数。分歧来源：参数计数口径（非嵌入 vs 全参数、最后一层）、小预算下 warmup 过高、拟合中的小规模非线性（外加方法 3 本身有缺陷）。实践启示：现代实践相对 Kaplan、常常也相对 Chinchilla 更加"过度训练"，因为推理算力占主导。
3. **问：** 用四步说清 IsoFLOPs 的流程。
4. **答：** (1) 选定一组算力预算 C_i；(2) 对每个预算训练若干规模 N_ij 的模型，token 数 D_ij = C_i/6N_ij；(3) 每个预算取损失最小的规模，得到 (C_i, N_opt)；(4) 在对数空间拟合 N_opt ∝ C^a、D_opt ∝ C^b 并外推到目标预算。
5. **问：** 为什么某模型的扩展律斜率可能与经典的 1/n 不同？
6. **答：** 经典参数化估计给出 1/n（斜率 −1）；d 维非参数学习给出 n^(−1/d)，斜率反映数据的内在维度——神经网络语言模型呈现的斜率两者都不完全符合，这正是促使内在维度理论（Bahri 2021）出现的"谜团"。

第 10 讲：推理（Inference）

日期：4 月 29 日（周三，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_10.py` *| 截止：作业 2 到期、作业 3 发布*

概览

推理才是模型真正被使用的地方，而它的特性与训练截然不同：**memory-bound（访存受限）且动态（dynamic）**（请求到达和结束的时间各不相同）。本讲推导 prefill 与 generation 两个阶段中 MLP 与 attention 层的算术强度，计算 Llama 2 13B 在 H100 上的理论延迟与吞吐，然后系统梳理加速手段：削减 KV cache（GQA、MLA、CLA、局部/滑窗 attention、DeepSeek v4 的 CSA/DSA/HCA）、量化（QAT/PTQ、AWQ）、剪枝 + 蒸馏、投机采样（无损！），以及面向动态负载的系统优化（continuous batching、PagedAttention）。

核心概念与定义

- **为什么推理效率重要**：训练是一次性成本，推理会被重复无数次（OpenAI 每天处理约 8.6T token）。Agent 让它更严重：内部轨迹可以无限增长。**生成的 token 数 = 花掉的算力**。

- **指标**：**TTFT**（time-to-first-token，首 token 延迟，由 prefill 决定）、**latency**（单条查询的秒/token，面向交互）、**throughput**（多查询的 token/秒，面向批处理）。
- **两个阶段**：**prefill**（把整个 prompt 并行处理，像训练一样——compute-bound）与 **generation/decode**（一次一个 token——memory-bound）。关键不对称性：*检查比生成快*（prefill 能一次算完所有位置）。
- **KV cache**：避免在每个生成步为整段历史重算 key/value。对每个序列（B）、token（S）、层（L）、头（K），存一个 H 维向量。朴素推理生成 T 个 token 需 $O(T^3)$ FLOPs；有 KV cache 后降为 $O(T^2)$。
- **算术强度核算**（bf16，每值 2 字节）：
- MLP 每步：FLOPs = 6·B·T·D·F；字节 = 4·B·T·D + 4·B·T·F + 6·D·F → 强度 ≈ B·T。Prefill（B·T 大）compute-bound；generation（T=1）强度 ≈ B——需要很多并发请求（batching）才能维持 compute-bound。
- Attention 每步：FLOPs = 4·B·S·T·D；字节 = 4·B·S·D + 4·B·T·D → 强度 = S·T/(S+T)。Prefill（T=S）为 S/2（不错）；generation（T=1）< 1——**靠 batching 也无法改善**，因为每个序列有自己的 KV cache（Q、K、V 都依赖 B），而 MLP 权重是共享的。
- 总结：prefill 是 compute-bound，generation 是 memory-bound（每步都要读全部参数 + KV cache）。
- **延迟/吞吐模型**：latency ≈ memory / bandwidth（读全部参数 + KV cache），throughput = B / latency。batch 越大：延迟越差（要读写的 KV cache 更大）、吞吐越好（分摊参数读取成本）——这是根本性权衡。此外：复制 M 份模型可让吞吐线性提升；TTFT 是 prefill 现象（TTFT 用小 batch，生成吞吐用大 batch）。
- **削减 KV cache**（memory-bound ⇒ cache 更小 ⇒ 更快）：
- **GQA（grouped-query attention）**：N 个 query 头，但只有 K 个 key/value 头（K < N）；MHA 为 K=N，MQA 为 K=1。把 KV cache 缩小 N/K 倍且几乎不损精度（Ainslie 2023）。Llama 2 13B 从 K:40 改到 K:8：单批延迟变差，但吞吐提升且能装进内存。
- **MLA（multi-head latent attention，DeepSeek v2）**：不存 K、V，而存压缩隐向量 c_t = W_c h_t（C 维）；需要时上投影为 K = W_K c、V = W_V c。DeepSeek v2：N·H = 16384 → C = 512（+ 64 维 RoPE，共 576）。MLA 在更低成本下甚至略优于 MHA。与 RoPE 不兼容：需额外保留非旋转的 key 维度。
- **CLA（cross-layer attention）**：跨层共享 KV（正如 GQA 跨头共享）；改善"精度 vs KV 大小"的帕累托前沿。
- **局部（滑窗）attention**：只关注一个窗口（Longformer、Mistral）；KV cache 与序列长度无关；有效上下文随层数线性增长；有时损精度 → 把局部与全局 attention 交错（混合层）。
- **DeepSeek v4 attention**（1M 上下文）：Compressed Sparse Attention（CSA，每 m 个 token 压成 1 个）、DeepSeek Sparse Attention（DSA，选 top-k）、Heavily Compressed Attention（HCA）。
- 其他：线性注意力 / 状态空间模型（Mamba-2、GatedDeltaNet）、扩散语言模型。
- **量化（quantization）**：比特更少 = 字节更少 = 更快（memory-bound）。fp32（训练）→ bf16（推理默认）→ fp8/int8 → int4/nvfp4。**QAT**（训练中量化，昂贵）；**PTQ**（训练后量化，便宜：在样本数据上校

准 scale/zero-point；GPTQ 用 Hessian 信息修正）；**AWQ**（激活感知：依据激活幅度把 0.1–1% 的重要权重保留高精度；fp16→int3 得到 4 倍显存下降、3.2 倍加速）。

- **剪枝 + 蒸馏**：(1) 在约 1024 条校准样本上识别重要的 {层, head, 隐维度}；(2) 去掉不重要的部分得到更小模型；(3) 用原模型向剪枝模型蒸馏（NVIDIA 的 pruning-KD 循环）。
- **投机采样（speculative sampling，无损）**：廉价**草稿模型** p 提议 γ 个 token；**目标模型** q 并行（prefill 速度）为它们打分；按修正的拒绝采样接受/拒绝。两个关键性质：(1) 至少生成一个 token（否则拒绝采样会无限循环）；(2) **保证是 q 的精确采样**——用两个符号 {A,B} 举例证明：若 p(A)>q(A)，残差 max(q−p,0) 修正后 P[采到 A]=q(A)、P[采到 B]=q(B)。扩展：Medusa（并行多头）、EAGLE（用目标模型特征做草稿）。
- *类比*：让一位手快的实习生先起草一段话；教授只读一遍（很快——检查比写作快），然后决定批准或修改；最终文本在统计上与教授独自写作完全相同。
- **Continuous batching（Orca）**：迭代级调度——新请求随到随加入批次，而不是等静态 batch 里所有序列都结束。**Selective batching**：attention 逐序列单独处理（长度参差），非 attention 运算把所有序列拼成一个 [Σs, H] 张量。
- **PagedAttention（vLLM）**：给 KV cache 做操作系统的"虚拟内存分页"——把每个序列的 KV 切成不连续的固定大小 block；消除内部/外部碎片；支持跨序列共享前缀 block（系统提示、同一 prompt 多采样）并用写时复制（copy-on-write）。vLLM 的其他优化：融合 block-attention kernel、FlashAttention/FlashDecoding、CUDA graphs。
- *类比*：静态 KV 分配像给每个进程预留"按最坏情况算"的连续内存（造成碎片）；分页就是虚拟内存——按需映射 block、共享只读页、写时复制。

代码示例：attention 的算术强度（关键推导）

代码（Python）：

```
# B 批、S 个历史 token、T 个待生成 token、D 模型维度；bf16 => 每值 2 字节
flops = 4*B*S*T*D                          # QK^T: 2*B*S*T*D  +  softmax@V: 2*B*S*T*D
bytes = 4*B*S*D + 4*B*T*D                  # 读 Q,K,V；写 Y

intensity = (S*T) / (S + T)                # 化简后的算术强度

# Prefill: T = S  =>  强度 = S/2           (不错——compute-bound)
prefill_intensity = S / 2

# Generation: T = 1  =>  强度 = S / (S + 1) < 1    (很差——memory-bound)
generate_intensity = S / (S + 1)
```

代码做了什么： 统计 attention 矩阵乘的 FLOPs 与 HBM 字节数，得出算术强度为 S·T/(S+T)——prefill 时是 S/2，generation 时小于 1——并且**与 B 无关**。

实现深挖： – **为什么与 B 无关**：attention 中的 Q/K/V 都是逐序列的（B 同时放大分子与分母，相互抵消）；而 MLP 的权重在 batch 间共享，所以 B 会提升强度。这正是"batching 救不了生成阶段的 attention"的原因——KV cache 是每个序列独有的。 – **为什么强度 <1 是致命的**：H100 的加速器强度约 295

FLOP/byte；生成阶段 attention 约每 1 个 FLOP 就要搬 1 字节——tensor core 几乎完全闲置。 – **为什么这解释了 GQA/MLA 的必要性**：每减少一个字节的 KV cache，都会直接降低生成延迟（latency ∝ 每步读取的显存）。

与作业的联系：作业 1 的资源核算要求给出每个组件的 FLOP/字节数；作业 2 的 FlashAttention 与 benchmark 工作针对的是同一组公式的训练侧。讲义的理论延迟/吞吐模型（`compute_transformer_performance_stats`：`num_params = 2VD + 3LDF + 2L·(2DNH + 2DKH)`，`kv_cache_size = 4·S·K·H·L` 字节）正是你用来 sanity check 作业 2 实测数字的模板。

代码示例：投机采样（无损解码）

代码（Python）：

```
def speculative_sample(draft_logits, target_logits, draft_next, rng):
    # draft_logits / target_logits: 下一个位置的 [vocab] 分布
    p = softmax(draft_logits)              # 草稿分布
    q = softmax(target_logits)             # 目标分布
    x = draft_next                         # 从草稿中抽出的候选 token
    u = rng.uniform(0, 1)
    if u < min(1, q[x] / p[x]):            # 以 q(x)/p(x) 的概率接受
        return x, True
    # 拒绝：从残差分布（归一化后）重采样
    residual = torch.clamp(q - p, min=0)
    x2 = sample(residual / residual.sum())
    return x2, False
```

代码做了什么： 从草稿模型抽取候选；以概率 q(x)/p(x) 接受（重要性加权）；被拒时从归一化的残差 max(q−p, 0) 重采样。这是"被改造过的拒绝采样"，保证总能给出一个来自 q 的有效样本。

实现深挖： – **为什么它是精确的**：接受（概率 min(1, q/p)）与残差重采样的混合恰好复现 q。讲义的双符号证明：P[采到 A] = p(A)·(q(A)/p(A)) + p(B)·1·0 = q(A)；P[采到 B] = p(B)·1 + p(A)·(1−q(A)/p(A))·1 = q(B)。 – **为什么它快**：草稿模型（如 8B）以 memory-bound 速度生成 γ 个 token；目标模型（如 70B）**并行**为这 γ 个 token 打分（prefill 式，compute-bound）——利用的正是"检查与生成之间的不对称性"。 – **为什么"至少生成一个"**：朴素拒绝采样可能永远拒绝；该修正保证前进，同时用残差修正保持分布精确。 – **如何让草稿更好**：向目标模型蒸馏草稿（提高接受率）、Medusa（并行草稿头）、EAGLE（用目标模型特征条件化草稿）。

与作业的联系：必修作业不实现它，但作业 5 的 RL 训练循环用的是同一套"通过高速推理服务器（vLLM）做 rollout"的机制——`cs336_alignment/vllm_utils.py` 中的 vLLM 接口正是本讲推理栈的工程落地。

代码示例：延迟/吞吐模型（Llama 2 13B on H100）

代码（Python）：

```
def compute_transformer_performance_stats(config):
    # 参数量（embedding + 3 个 MLP 矩阵 + 每层 attention 的 QKV/O）
    num_params = 2*V*D + D*F*3*L + (2*D*N*H + 2*D*K*H)*L
    parameter_size = 2 * num_params                         # bf16

    # 每序列的 KV cache：S 个 token * K 头 * H 维 * L 层 * (K+V) * 2 字节
    kv_cache_size_per_seq = S * (K*H) * L * 2 * 2

    memory = B * kv_cache_size_per_seq + parameter_size   # 每步要读的总字节

    latency = memory / memory_bandwidth                     # 秒/token
    throughput = B / latency                                # token/秒
    return num_params, memory, latency, throughput

# Llama 2 13B 配置：S=1024, D=5120, F=13824, N=40, K=40, H=128, L=40, V=32000
# B=1:   latency ~ (26GB 参数 + 很小的 cache) / 3.35TB/s   (约 7.8ms/token)
# B=64：吞吐更好、延迟更差（KV cache 更大，要读更多）
# B=256：吞吐收益递减，而且装不进 80GB 的 H100!
```

代码做了什么： 计算参数量、显存（参数 + KV cache），以及在"显存带宽受限"下的延迟/吞吐，并代入 Llama 2 13B 在 batch 为 1/64/256 时的情形——展示延迟–吞吐权衡与显存上限。

实现深挖： – **为什么 latency = memory/bandwidth**：生成是 memory-bound；每一步都必须从 HBM 读取全部参数加上整个 KV cache。这是理论下界（假设完美重叠）——真实系统只会更差。 – **为什么 throughput = B/latency**：每步并行生成 B 条序列，故 token/秒 = B × 每秒步数。 – **为什么削减 KV cache 一举两得**：显存更小 → 每步读取时间更短（延迟降低），同时能塞进更大的 batch（吞吐提升）。

与作业的联系： `TransformerPerformanceStats` 这套符号化核算（讲义用 sympy）是可复用的模板，可用于作业 1 的资源核算，以及核对作业 2 的 profiling 数字（例如"为什么生成是 memory-bound？"——你的 attention kernel benchmark 应当反映这一点）。

关键要点

1. 推理分两种：prefill（compute-bound，像训练）与 generation（memory-bound，一次一个 token）——而生成阶段的 attention 是**batching 无法解决**的 memory-bound（强度 <1，与 B 无关）。
2. KV cache 是核心资源：latency ∝ 每步读取的（参数 + KV cache）；用 GQA、MLA、CLA、局部 attention 或状态空间混合模型削减它。
3. 存在无损加速：投机采样可证明是精确的（修正拒绝采样），利用"检查比生成快"。
4. 有损加速：量化（QAT/PTQ/AWQ）与剪枝 + 蒸馏，用精度换显存与速度。
5. 动态负载需要系统技巧：continuous batching（迭代级调度、selective batching）与 PagedAttention（分页、前缀共享、写时复制）——思路直接借自操作系统。

常见陷阱

- **显存预算里忽略 KV cache**：长上下文 + 大 batch 时，OOM 的往往是 KV cache（而不是参数），而且它随并发数增长。

- **memory-bound 时还用 MHA**：高 batch 下 GQA/MLA 几乎是白送的收益；MHA 的精度优势很小。
- **朴素量化**：逐张量 scale 会丢掉离群通道的信息；应使用分块 scale（AWQ、MXFP8）并在真实激活范围上校准。
- **以为投机解码会改变分布**：它必须精确——若实现改动了采样，就不再是"目标模型"的输出。
- **静态 batching**：等所有请求结束会浪费 GPU（一条慢序列拖住所有人）；要用 continuous batching。
- **KV 分配碎片化**：为每个请求预留最大长度会造成内部/外部碎片和 HBM 浪费——要分页。
- **忘记 TTFT 与吞吐的区分**：为交互流量优化吞吐会伤害用户可见延迟；分阶段调 batch（prefill 小、generation 大）。

复习题

1. **问：** 为什么 batching 无法解决生成阶段 attention 的 memory-bound 问题?
2. **答：** attention 的算术强度 S·T/(S+T) 中没有 B 项：Q、K、V 都是逐序列的，B 同时放大 FLOPs 与字节数并相互抵消。MLP 不同——权重在 B 间共享，因此 batching 把强度提升到约 B·T。生成阶段 attention 无论 batch 多大都保持 <1 的强度。
3. **问：** 投机采样如何保证样本精确来自目标模型 q?
4. **答：** 它是被改造过的拒绝采样：以 min(1, q(x)/p(x)) 的概率接受草稿 x；被拒时从归一化的残差 max(q−p, 0) 重采样。混合代数（双符号情形已给出证明）恰好复现 q，而"至少接受一个 token"的修正保证过程终止。
5. **问：** Llama 2 13B 把 K 从 40 个头降到 8 个头（GQA）——什么变了、什么没变?
6. **答：** KV cache 缩小 5 倍（每步延迟下降、可容纳更大 batch、吞吐上升）。query 头仍为 40（attention 的表达力基本保持）；据 Ainslie 等 2023，精度损失很小甚至可忽略。

第 11 讲：扩展律案例与细节（Scaling — Case Study and Details）

日期：5 月 4 日（周一，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_11.pdf` *| 截止：作业 3 到期*

概览

本讲把理论落到实践：拆解**公开且细节充分**的扩展配方——**MiniCPM**（用 muP + WSD 学习率 + Chinchilla 分析训出小而强模型）与 **DeepSeek**（用小规模实验拟合 batch/LR、用 IsoFLOPs 定模型规模）；此外还有 StepFun 关于 LR/batch 扩展的大规模实证研究、优化器扩展（含 Muon），以及 **muP** 的深入剖析——它是什么、如何推导、对什么鲁棒、对什么不鲁棒。

核心概念与定义

- **实践中的扩展**：2022 年后很少有模型公开扩展细节；MiniCPM 与 DeepSeek 是两个难得的、有严谨公开分析的例外。

- **MiniCPM 配方（2024）**：1–2.5B 模型，打败大多数 2B 并追平不少 7B 模型。
- **用 muP 稳定扩展**：`scale_emb=12, scale_depth=1.4, init_std=0.1, lr=0.01`——有了 muP，最优学习率在不同宽度下大致恒定。
- **固定长宽比（aspect ratio）**，整体放大规模（最大实验模型与实际模型差距约 5 倍）。
- **最优 batch**：用 3 个规模（9m/30m/170m）画损失 vs (batch, 数据量)；最优 batch 随损失下降呈多项式增长（Kaplan 2020 式分析）。
- **WSD 学习率**（warmup–stable–decay）：让 Chinchilla 分析变便宜——可以在 stable 阶段末尾**重启**一个新 token 预算的训练，而不必从头再训。这把拟合扩展律的 $O(n^2)$ 成本降到可承受范围。
- 用 Chinchilla 方法 1（下包络）与方法 3（联合拟合），得到很高的数据:模型比例。
- **WSD（warmup-stable-decay）学习率调度**：把调度拆为预热、稳定、衰减三段；损失主要在衰减段快速下降（约占训练的 10%）。效果与 cosine 相当，却支持重启。
- **DeepSeek 配方（2024）**：不用 muP——直接从小规模实验估计最优 batch/LR（保留"接近最优"、与最小损失相差 0.25% 以内的运行）；WSD 式学习率（两次各 10% 的衰减）；用 Chinchilla 方法 2（直接的 IsoFLOPs）定模型规模；拟合出的扩展模型能准确预测最终模型的损失。
- **近期配方**（细节更少）：Qwen（LR/batch 拟合）、Kimi K2（MoE 稀疏度扩展律）、Hunyuan（MoE 的 IsoFLOPs——最优数据:激活参数比 96:1）、LLaMA 3（IsoFLOPs，39:1，算力到下游指标的扩展）、MiniMax-01（架构扩展 + Chinchilla 方法 1）。
- **StepFun 扩展律研究**：纯实证地在多个规模上网格搜索 (LR, batch)：
- 损失对 batch/LR 是**凸的**——最优点能干净地识别。
- 扩展趋势：batch 主要取决于数据量；固定 M 时最优 LR 随 D 增大而上升（但换成 WSD 后这一结论较脆弱）。
- 可推广到 MoE 与其他数据集（有前提）。
- **优化器扩展的问题**：(1) 不同优化器需要不同超参数，可能还有不同的最优扩展规则；(2) 显著的规模依赖性——"永远要检查相对于算力与 Chinchilla 比例的扩展性——它们常是性能对比中的主要混淆因素"；(3) 建立扩展性本身并不容易——看起来漂亮的扩展曲线可能突然爆炸（AdamC + sqrt-batch LR 缩放的例子）。
- **Muon**：面向**矩阵值参数**的优化器，用 Newton–Schultz 迭代近似正交化更新：$B_t = U\Sigma V^\top \to UV^\top$。在大规模上有效（nanoGPT speedrun、Kimi K2）；收益难以精确度量。
- **muP（maximum update parametrization）深入**：
- *两条断言*：随宽度 n_l 增大，(A1) 初始化时激活保持 $\Theta(1)$；(A2) 走一步梯度后激活的变化量为 $\Theta(1)$。
- *推导梗概*：对深层线性网络 $h_l = W_l h_{l-1}$，$W \sim N(0, \sigma^2)$，取 $\sigma = \Theta(1/\sqrt{n_{l-1}}\cdot\min(1, n_l/n_{l-1}))$ 可使 $\|h_l\|^2 = \Theta(n_l)$。对更新量，$\Delta W_l = -\eta\nabla_{h_l}\ell \cdot h_{l-1}^\top$（秩一外积）；要求 $\Delta\ell = O(1)$ 就给出 SGD 下的 $\eta = \Theta(n_l/n_{l-1})$，以及 Adam 下的相应缩放（$\Delta W\cdot\sqrt{n_{l-1}} = \Theta(\eta_l)$，对 Adam 即约 $1/\sqrt{n_{l-1}}$）。
- *标准参数化（SP）vs muP*：SP 用 init $1/\sqrt{n_{l-1}}$、LR $\Theta(1)$；muP 同时调整 init 与 LR（Adam 下 LR 还要乘以 n_{l-1} 的相应因子），使最优 LR 具备**宽度不变性**。

- *muP 对什么鲁棒*：SwiGLU/squared-ReLU 激活、大/小 batch、zero-attention 初始化、部分异类优化器（Lion）——大体都还行。
- *什么会破坏 muP*：**RMSNorm 的可学习增益**（理论前提被破坏，但去掉增益几乎不损性能）、**基于梯度符号的异类优化器**、以及**强 weight decay（0.1）**——"也许这是唯一显著的 muP 失效情形"。
- *结论*：muP 总体有用——SP 明显更不稳定；muP 的参数化/初始化更容易调。

代码示例：WSD 学习率调度（让扩展律分析变便宜）

代码（Python）：

```
def lr_schedule(step, total_steps, warmup_steps, max_lr, decay_frac=0.1):
    # 阶段 1：预热（线性）
    if step < warmup_steps:
        return max_lr * (step + 1) / warmup_steps
    # 阶段 2：稳定（常数）
    decay_start = int(total_steps * (1 - decay_frac))
    if step < decay_start:
        return max_lr
    # 阶段 3：衰减（例如 cosine 或线性衰减到 max_lr 的 10%）
    t = (step - decay_start) / max(total_steps - decay_start, 1)
    return max_lr * (0.1 + 0.9 * 0.5 * (1 + math.cos(math.pi * t)))

# Chinchilla 分析的关键技巧：从稳定阶段重启。
# 若某次 200B token 的训练在 step S 结束稳定阶段，
# 你可以（用新的衰减）继续到 400B、800B……而不必从头重训。
```

代码做了什么： 实现三段式 WSD 调度，并突出其"可重启"性质——稳定阶段允许把一次运行分叉成多个 token 预算，成本很低。

实现深挖： – **为什么 WSD 能媲美 cosine**：经验上损失下降主要集中在衰减阶段；稳定阶段让模型保持"随时可衰减"的状态。DeepSeek 用两次各 10% 的衰减，MiniCPM 用一次衰减。 – **为什么这对扩展律至关重要**：Chinchilla 联合拟合需要很多 (N, D) 点上的损失，过去每个点都要从头训练一次。WSD 重启把一次运行变成多个数据点——把成本从 $O(n^2)$ 降到 $O(n)$。这正是作业 3 那 12 个 B200 小时预算逼迫你必须利用的经济学。

与作业的联系：作业 3 的训练 API 未必暴露 WSD，但其**方法论**——在有限运行预算下拟合扩展律、外推到 48 B200 小时——与 MiniCPM/DeepSeek 完全一致。你的报告应当像这些论文一样，写清搜索空间探索策略（IsoFLOPs、联合拟合，还是带 muP 假设）。

代码示例：muP 风格的初始化与学习率缩放（概念）

代码（Python）：

```
import math

def init_and_lr_scale(fan_in, fan_out, scheme="muP"):
    if scheme == "SP":                          # 标准参数化
        init_std = 1.0 / math.sqrt(fan_in)
        lr_scale = 1.0
    else:                                       # muP（简化版）
        # init: Theta(1/sqrt(fan_in) * min(1, fan_out/fan_in))
        init_std = (1.0 / math.sqrt(fan_in)) * min(1.0, fan_out / fan_in)
        lr_scale = fan_out / fan_in             # SGD 情形；Adam 用 lr * fan_in 的相应形式
    return init_std, lr_scale
```

代码做了什么： 对比标准参数化（init $1/\sqrt{\text{fan_in}}$、LR 1）与 muP（init $1/\sqrt{\text{fan_in}}\cdot\min(1, \text{fan_out}/\text{fan_in})$、LR $\text{fan_out}/\text{fan_in}$）——这两处改动让最优超参数具备宽度不变性。

实现深挖： – **为什么 init 这样设**：让激活保持 $\Theta(1)$，与宽度无关——由矩阵集中不等式推出（$\|W_l\| \to \sigma\cdot(\sqrt{n_{l-1}} + \sqrt{n_l})$）。 – **为什么 LR 这样设**：让每一步的**更新量**保持 $\Theta(1)$（条件 A2）：由 $\Delta W = -\eta\cdot$梯度外积、要求 $\Delta\ell = O(1)$，得到 $\eta \propto n_l/n_{l-1}$（SGD）；而 Adam 的逐坐标归一化会改变 LR 缩放（约 $1/\sqrt{n_{l-1}}$ 的形式）。 – **为什么有用**：超参数**跨规模迁移**是小规模扩展实验（作业 3！）可信的前提。讲义的 miniCPM 数值（scale_emb=12、scale_depth=1.4）就是实用旋钮取值。

与作业的联系：作业 3 的 handout 明确允许在扩展方法论中引入 muP 思路（G. Yang 等）；作业 1 的初始化（handout 中"某个缩放因子"）至少应当具备宽度感知。若你要在多个规模上训练模型，muP 就是"LR 可迁移"与"LR 必须重调"之间的差别。

关键要点

1. 两个公开配方：MiniCPM（muP + WSD + Chinchilla 1&3）与 DeepSeek（小规模 LR/batch 拟合 + IsoFLOPs + WSD）——两者都准确预测了最终模型的损失。
2. WSD 学习率是关键的使能技巧：效果媲美 cosine，而"可重启"性质让 Chinchilla 式拟合变得可负担（n 次 vs n^2 次训练）。
3. LR 与 batch 对规模敏感：每个规模上损失对 (LR, batch) 是凸的，但最优点随算力/数据移动——务必对照 Chinchilla 比例检查扩展性，它常是优化器/架构对比中的主要混淆因素。
4. muP 通过宽度感知的 init + LR 缩放使最优 LR 大致与宽度无关；它对很多现代组件鲁棒，但会被 RMSNorm 增益与强 weight decay 破坏。
5. 近期大厂（Qwen、Kimi K2、Hunyuan、LLaMA 3、MiniMax）都做某种形式的 IsoFLOPs/扩展分析——这个领域已把扩展律当成标准设计工具。

常见陷阱

- **$O(n^2)$ 的扩展律拟合**：每个 (N, D) 点都从头训练是负担不起的；要用 WSD 重启 / 稳定阶段分叉。
- **不用 muP 却信任 LR 迁移**：朴素宽度缩放会移动最优 LR；要么用 muP，要么像 DeepSeek 一样直接拟合。

- **混淆训练最优与推理最优**：回顾第 9 讲——部署算力主要是推理，所以要过度训练（每参数 token 数 ≫ 20）。
- **忘记 batch 的扩展**：超过临界 batch 后收益递减；batch 要与 LR 联合扫描。
- **过度套用 muP 结论**：RMSNorm 增益与强 weight decay 会破坏迁移——要验证而不是假定。
- **拟合出漂亮曲线却突然爆炸**：好看的扩展拟合可能掩盖不稳定（AdamC 的例子）；至少在一个更大规模上做验证。

复习题

1. **问：** WSD 调度如何让 Chinchilla 式分析更便宜?
2. **答：** cosine 调度需要为每个 (N, D) 点从头训练一次。WSD 的稳定阶段允许把一次运行分叉：用不同的衰减时机/token 预算继续训练，从而从一次预训练中得到多个损失点——把成本从点数的平方级降到接近线性。
3. **问：** muP 的两条断言与由此确定的旋钮是什么?
4. **答：** (A1) 初始化时激活保持 Θ(1)；(A2) 一步梯度后激活变化为 Θ(1)。这确定初始化尺度 σ = Θ(1/√fan_in·min(1, fan_out/fan_in)) 与 LR 尺度（SGD 为 fan_out/fan_in；Adam 的基础 LR 相应按 √fan_in 缩放）。
5. **问：** DeepSeek 没用 muP 却也成功了，为什么?
6. **答：** 它不去"假设"LR 可迁移，而是**实测**：跑大量小规模模型，保留"接近最优"（与最小损失相差 0.25% 以内）的运行，直接拟合 LR/batch 的扩展关系，再用 IsoFLOPs 定模型规模。实证得到的扩展关系可以替代基于参数化的迁移——代价是要多花一些小规模算力。

第 12 讲：评测（Evaluation）

日期：5 月 6 日（周三，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_12.py` *| 截止：作业 3 到期、作业 4 发布*

概览

在决定"用什么数据训练"之前，必须先决定"想要模型具备什么行为"，以及如何衡量它。本讲系统梳理评测版图：困惑度（同分布与 zero-shot）、考试型基准（MMLU、MMLU-Pro、GPQA、HLE）、对话型基准（Chatbot Arena、AlpacaEval、WildBench）、智能体型基准（SWE-bench、Terminal-Bench、CyBench、MLE-Bench）、纯推理基准（ARC-AGI）与安全基准（HarmBench、AIR-Bench）；随后讨论真实性与生态效度（GDPVal、MedHELM、Clio）、效度问题（训练-测试污染、数据集质量），以及"如何看待评测"（方法 vs 模型 vs 智能体）。

核心概念与定义

- **核心难题**：把**抽象构念（abstract construct）**——"好模型"——转化为**具体指标（concrete metric）**。不同人群对"好"的定义不同：榜单分数、性价比、人类偏好（Chatbot Arena）、真实用户采用

（OpenRouter）。

- *类比*：评测语言模型就像评价一家餐厅——米其林星级（基准测试）、性价比（成本调整后）、点评网站（人类偏好）、回头客（真实采用）测的是不同东西。
- **困惑度（perplexity）**：对模型 p 与数据集 D，`PPL = (1/p(D))^(1/|D|)`，衡量 p 给 D 分配的概率是否高。训练在最小化它；最直观的评测就是测测试集困惑度。
- *历史*：PTB、WikiText-103、One Billion Word（同分布时代；CNN+LSTM 把困惑度从 51.3 降到 30.0）。GPT-2 引入**zero-shot**（异分布）评测：在 WebText 上训练、在标准数据集上评估。
- *"困惑度就是一切"*：若 p 等于真实分布 t，则 p(solution|problem) 就能解决所有任务——压低困惑度终将"抵达 AGI"。*"困惑度又超过了所需"*：它惩罚每一个 token，包括无信息的 token（"Stanford was founded in **1885**" 中的 "founded"）；应改用条件困惑度 p(response|prompt)^(1/|response|)。有些基准其实是伪装的困惑度任务：LAMBADA（完形填空）、HellaSwag（句子补全）。
- *困惑度排行榜的警告*：你必须信任提交的模型给出的概率是合法的（归一化为 1）。
- **考试型基准**：科目与难度可控、答案无歧义、易于判分。
- **MMLU**：57 个学科、多选题、few-shot；尽管名字如此，它测的其实是**知识**而非"语言理解"。**MMLU-Pro**：剔除噪声题、选项从 4 个扩到 10 个、用 CoT 评测；准确率下降 16%–33%（不再饱和）。
- **GPQA**：研究生水平、"防谷歌"的题目，由博士承包者撰写；博士专家 65%，非专家用 Google 30 分钟 34%，GPT-4 39%。
- **HLE（Humanity's Last Exam）**：2500 道多模态题，出题者共享 50 万美元奖池并获共同作者署名，经前沿模型筛选与多轮评审。
- *局限*：无法反映真实使用（开放式、未必存在唯一正确答案）。
- **对话型基准**（开放式评测）：
- **Chatbot Arena**：随机用户向两个匿名模型提问并投票；用 **Elo** 排名，p(A 胜 B) = 1/(1+10^((Elo_B−Elo_A)/400))。特点：真实提示、动态、不必给所有模型喂同样提示（由人来评分）；但人群有偏、把风格与正确性混为一谈、易受谄媚（sycophancy）影响。
- **AlpacaEval**：805 条指令；以 GPT-4 为裁判、对比 GPT-4 的胜率；曾有长度偏差（LLM 裁判偏爱更长的回答）→ AlpacaEval 2.0 用回归去偏；与人类 Arena 相关性高。
- **WildBench**：从 100 万条真实人机对话中挑 1024 个例子；用 GPT-4-turbo 作为带清单（checklist，相当于"评判的 CoT"）的裁判。
- 关键经验：**相似回答之间的成对比较信噪比更高**；警惕人类与 LLM 裁判的偏差；明细清单/评分规则能提升可靠性。
- **智能体型基准**（评"模型做了什么"，而非"说了什么"）：Agent = LM + agent scaffold（规划、委派、记忆、上下文工程）。
- **SWE-bench**：12 个 Python 仓库上的 2294 个任务；给代码库 + issue 描述，提交 PR；用单元测试打分。
- **Terminal-Bench**：终端环境（简单、通用）；229 个众包任务。
- **CyBench**：40 个 CTF 任务；以"首次解出时间"衡量难度。

- **MLE-Bench**：75 个 Kaggle 竞赛（需要训练模型、处理数据）。
- **纯推理基准**：**ARC-AGI**——人类 100% 可解，但 AI 很难；每个任务都独一无二，记忆无用。ARC-AGI-1（2019）、ARC-AGI-2（2025，更多步推理）、ARC-AGI-3（2026，交互环境）。预训练语言模型没能推动它；推理模型（o1、o3）才真正起飞。
- **安全基准**：**HarmBench**（510 种违法或违反规范的有害行为）；**AIR-Bench**（基于监管框架：314 个风险类别、5694 条提示）；**越狱（jailbreaking）**——GCG 自动优化对抗提示，并可从开源模型迁移到闭源模型。安全是强语境相关的（政治、法律、社会规范因国家而异），风险也高度多样（幻觉、谄媚、协助犯罪、不平等）。
- **真实性与生态效度**：评测在多大程度上反映真实使用？**GDPVal**（OpenAI）：覆盖美国 GDP 前 9 大行业的 44 个职业，任务来自有约 14 年经验的专业人士；**MedHELM**：来自 29 位临床医生的 121 个临床任务（而非标准化考试）；**Clio**（Anthropic）：用 LM 分析真实用户数据。不幸的是，真实性与隐私有时互相冲突。
- **效度 / 污染**：
- 路径 1：从模型行为推断训练-测试重叠（利用数据点的可交换性）。
- 路径 2：推动报告规范（模型方应报告训练-测试重叠、置信区间）。
- 路径 3：使用最新评测（LiveCodeBench、UncheatableEval——抓取新网页）；但时间戳也不绝对可靠（存在转载）。
- 路径 4：使用私有评测（内部代码库、个人写作）——对困惑度最容易。
- 数据集质量：SWE-bench → SWE-bench Verified；各类基准的"Platinum"版本；智能体基准常见问题是测试用例不足（平凡 agent 也能通过）；Docent 用 LLM 检查 agent 轨迹来发现问题。
- **如何思考评测**：没有唯一正确的评测，取决于你要回答什么问题（采购决策、原始能力、收益与危害、开发反馈）。基础模型出现之前，我们评测的是**方法**（固定的训练-测试划分）；今天我们大多在评测**模型/系统**（规则宽松）——例外如 nanogpt speedrun（固定数据、测达到某验证损失所需时间）属于方法型。评测方法能激励算法创新，评测系统对下游用户更有用。无论如何，**必须明确游戏规则**。

代码示例：困惑度与条件困惑度

代码（Python）：

```
import torch

def perplexity(log_probs: torch.Tensor) -> torch.Tensor:
    # log_probs: [num_tokens] (log p(token_i | 历史))
    nll = -log_probs.sum()
    return torch.exp(nll / log_probs.numel())       # (1/p(D))^(1/|D|)

def conditional_perplexity(prompt_log_probs, response_log_probs):
    # 只对回答部分打分 (p(response | prompt)^(1/|response|))
    return torch.exp(-response_log_probs.sum() / response_log_probs.numel())
```

代码做了什么： 从逐 token 对数概率算困惑度（标准公式），并给出只对回答打分的条件变体——避免被"高度可预测的 prompt token"稀释。

实现深挖： – **为什么是 exp(平均 NLL)**：PPL = exp(−(1/T)Σ log p(x_t|x_<t))——几何平均意义下的"逆概率"。越低越好；在词表大小 V 上随机猜测约得 PPL ≈ V。 – **为什么需要条件困惑度**：对"Stanford was founded in ___"这类任务，"founded"、"in" 几乎确定、几乎不花代价；只对答案打分才能隔离出模型在**关键部分**的真实预测能力。 – **为什么这对排行榜重要**：若排行榜以 OpenWebText 困惑度为指标，就必须信任提交方给出的概率模型——讲义关于"验证概率合法性（归一化为 1）"的警告在此适用。

与作业的联系：作业 1 的排行榜就是一个**困惑度排行榜**（在 B200 上 45 分钟内最小化 OpenWebText 困惑度）——正是这里定义的指标。作业 4 的排行榜则在给定 token 预算下最小化困惑度（用下游 PPL 来评估数据处理的成效）。"评测方法 vs 评测模型"的区分，正是这类课程排行榜得以成立的原因。

代码示例：Elo 评分（Chatbot Arena 风格）

代码（Python）：

```
import numpy as np

def p_win(elo_a, elo_b):
    return 1.0 / (1.0 + 10 ** ((elo_b - elo_a) / 400.0))

def update_elo(elo, winner_idx, loser_idx, k=32):
    e = p_win(elo[winner_idx], elo[loser_idx])
    elo[winner_idx] += k * (1 - e)       # 冷门获胜得分更多
    elo[loser_idx] += k * (0 - (1 - e)) # 冷门落败失分更多
    return elo

# 拟合：遍历所有成对比较并同时更新双方的 Elo。
# （实践中：在比较矩阵上做最大似然拟合。）
```

代码做了什么： 实现 Elo 更新：胜者获得的分数正比于这场胜利有多"出人意料"（1 − 期望值），败者扣除相同分数。

实现深挖： – **为什么用 Elo**：它把成对偏好聚合为一个标量评分，其差值通过 logistic 曲线预测胜率。Arena 实际上是在所有比较上做最大似然拟合，这里展示在线更新只为直观。 – **为什么成对比较有力**：比较两个相似回答比绝对打分信噪比更高——这既是人类评测（Arena）也是 LLM 裁判评测（AlpacaEval/WildBench）的基础。

与作业的联系：评测方法论与作业 5 的可选 DPO 部分相关（AlpacaEval 是其评测数据集之一），也与作业 4"训练分类器过滤质量"相关（其训练数据的正负样本构造方式与 DCLM 一致——见第 13/14 讲）。

关键要点

1. 没有唯一正确的评测——选择能回答**你的问题**的指标，并明确你在评的是方法、模型还是智能体。
2. 困惑度依然是模型**研发**的主力（扩展曲线平滑），但要让不信任它的人信服，还需要贴近真实场景的基准。

3. 基准版图正走向更难的考试（MMLU→MMLU-Pro→GPQA→HLE）、真实用户偏好、智能体任务表现、纯推理（ARC-AGI）与安全。
4. 要警惕效度威胁：训练-测试污染、数据集质量问题、裁判偏差（尤其是长度偏好）、以及"针对评测过拟合"。
5. 评测塑造 AI 的发展——选择衡量什么本身就是有后果的设计决策（游戏规则很重要）。

常见陷阱

- **基准饱和**：模型在 MMLU 上触顶；要用 MMLU-Pro/GPQA/HLE 这类刻意保持难度的评测。
- **LLM 裁判的长度偏差**：基于 GPT-4 的裁判偏爱更长的回答；要做去偏（AlpacaEval 2.0 的回归）或使用清单式评判。
- **污染**：若你的预训练数据（作业 4！）包含基准样例，困惑度/准确率数字就失真——要过滤，或改用最新/私有评测。
- **人类评分中的谄媚与风格混淆**：Arena 投票测的是偏好，不是正确性。
- **评测智能体 ≠ 评测模型**：agent scaffold 会显著改变结果；要报告整个系统的配置。
- **排行榜上信任未经验证的概率**：确保提交的模型是合法的概率分布。

复习题

1. **问：** 为什么测试集困惑度更低的模型，对用户而言可能更差?
2. **答：** 困惑度奖励的是对所有 token 的概率（包括琐碎的 token），对分词方式敏感，且不反映偏好、安全、风格或任务成功率——而这些才是用户真正在意的。对话型与智能体型基准测的是不同的构念。
3. **问：** GPQA 的"防谷歌"是什么意思？为什么重要?
4. **答：** 题目极其专业，非专家即使能上网也答不出（非专家 34%，博士专家 65%）——因此该基准隔离的是真正的专家级知识，而不是检索能力。
5. **问：** 如何可靠地评测开放式回答?
6. **答：** 优先在相似回答之间做成对比较（信噪比更高）；用清单/评分规则配合 LLM 裁判（WildBench 式 CoT 评判）；用与人类（Arena）的相关性来验证裁判；警惕长度/谄媚偏差。

第 13 讲：数据 I — 来源与数据集

日期：5 月 11 日（周一，Spring 2026） | 讲师：Percy Liang | 材料： `lecture_13.py`

概览

训练语言模型时，"**数据**是最需要做对的东西"——而数据"不会从天而降"。本讲覆盖数据从哪来（爬虫、在线服务）、法律环境（版权、许可、合理使用、诉讼）、经典来源（Common Crawl、Wikipedia、GitHub、arXiv），并巡礼著名数据集：BooksCorpus、WebText/OpenWebText、CCNet、C4、GPT-3 数据混合、

The Pile、Gopher 的 MassiveText、LLaMA、RefinedWeb/FineWeb、Dolma、DCLM、Nemotron-CC、The Stack、CommonPile。

核心概念与定义

- **为什么数据是机密**：开放权重模型（Llama 3）会公开架构甚至训练流程，但基本不公开数据细节——原因：竞争格局与版权责任。"数据本质上是长尾问题，靠人力投入扩展（不像架构和系统）。"
- **训练的数据阶段**：**预训练**（大量较低质量的原始文本）→ **中训练（mid-training）**（高质量数据以增强能力）→ **后训练**（对话记录 / RL）。总体趋势是"从大量低质量数据走向少量高质量数据"。术语：base model（预训练 + 中训练后）、instruct/chat model（后训练后）。（OLMo 2 的例子：预训练 → Dolmino 中训练 → Tülu 后训练。）
- **原始来源**：网络 = 一组在线服务器；你需要**爬虫（crawler）**（从种子集发现网页、下载、遵守策略）。拿不到的部分：动态内容（应用、Discord）、需登录内容（Facebook、X、NYT）、robots.txt 禁止、Cloudflare 拦截、限速、ToS 禁止。"同意的衰退（decline of consent）"：常见数据集（C4、RefinedWeb、Dolma）中 URL 的限制随时间不断增加。**影子图书馆（shadow libraries）**（LibGen、Sci-Hub、Z-Library）：技术上属于网络，但法律上是盗版——LibGen 约 400 万本书（2019），Sci-Hub 约 8800 万篇论文（2022）。
- *类比*：爬取网络像在一片巨湖里捕鱼——湖就是"互联网"，但其中很多是私人水域（封闭平台）、保护区（robots.txt）或法律禁区（受版权保护的水域）。
- **版权基础**：保护"固定于任何有形表达媒介的原创作品"——也就是说**互联网上几乎所有内容都受版权保护**。保护针对**表达**而非**思想**（快排算法不受版权保护，但你的网站受保护）。保护期约 75 年；起诉前需登记（获得保护不需要）。使用受版权保护的作品有两条路：(1) **取得许可**（Creative Commons、商业授权——Google×Reddit、OpenAI×Shutterstock/StackExchange）；(2) **合理使用（fair use）**四要素：使用的目的与性质、作品性质、使用部分的数量与实质性、对原作品市场的影响。合理使用的例子：看完电影写摘要、重新实现某个算法、Google Books 展示片段（Authors Guild v. Google）。
- *对语言模型的特殊考量*：复制数据本身（训练的第一步）在技术上已构成侵权；训练应属转化性使用；模型应习得"思想"（巫师）而非"表达"（哈利·波特）；无论版权如何，LM 都可能冲击市场。**ToS 可施加超出版权的额外限制**（YouTube 的服务条款禁止下载，即使视频采用 CC 许可）。
- *诉讼*：NYT v. OpenAI（2023）；作者诉 Anthropic（2024）——2025 年简易判决认为**训练**使用原告作品属合理使用，但**盗版复制**不属于；Anthropic 最终以 15 亿美元和解。作者诉 Meta（2025）：在此案情形下用书籍训练属合理使用；盗版下载仍待审。目前结论：*训练在具体案件中被认定为合理使用；盗版明确违法；这一领域仍在快速演变*。
- **Common Crawl**：2007 年成立的非营利组织；约每月一次抓取，每次新增 30–50 亿网页；累计约 3000 亿页；2026 年 4 月的抓取包含 21.9 亿页（372.2 TB）。基于 Apache Nutch：种子 URL → 队列 → 下载 → 把超链接入队，并遵循选择/礼貌/重访策略。两种格式：**WARC**（原始 HTTP 响应，如 HTML）与 **WET**（转为文本——有损过程）。HTML→文本工具：trafilatura、resiliparse——而转换质量会可测量地影响下游任务精度（DCLM）。
- **Wikipedia**：6700 万条目、361 种语言；不做原创研究、以"关注度"为准；任何人都能编辑（破坏会被管理员回退）；定期 dump（无需爬取）。*投毒风险*：恶意编辑可在 dump 前注入并生效。

- **GitHub**：4.2 亿+ 仓库（2800 万公开）；代码仓库（走 git 协议）与元数据（issue/PR/评论，走 API 与 GitHub Archive）。大量重复（fork/复制）；只使用宽松许可。Software Heritage 聚合 GitHub/GitLab/Bitbucket/PyPI 等的仓库（2880 万个源文件）。
- **arXiv**：1991 年以来约 300 万篇论文；元数据（标题/摘要，CC0）+ PDF + 可选 LaTeX 源码；可从 S3 批量下载。
- **数据集谱系巡礼**（各自带过滤配方）：
- **BERT**：Wikipedia + BooksCorpus（从 Smashwords 抓取的 7000 本 0 元自助出版书，9.85 亿词——因违反 ToS 已被下架）；以文档（而非句子）为单位——对比 10 亿词基准。
- **GPT-2 WebText**：来自 karma ⩾ 3 的 Reddit 帖子外链页面（以"点赞"作质量代理）；800 万页、40GB。**OpenWebTextCorpus**：开源复现，用 Reddit 提交 URL + fastText 语言过滤 + 近重复去除。
- **CCNet**：Common Crawl + 段落去重（轻量归一化）+ fastText 语言识别 + **KenLM 5-gram "像不像 Wikipedia"** 过滤；用 CCNet(CC) 训练的 BERT 优于用 Wikipedia 训练的。
- **C4（Colossal Clean Crawled Corpus）**：取 2019 年 4 月的单次快照（1.4 万亿 token）；**手工启发式规则**：保留以标点结尾且 ⩾5 词的句子、丢弃少于 3 句的页面、去掉脏词表、含 '{'、"lorem ipsum"、"terms of use" 的页面；用 langdetect 保留英文（p ⩾ 0.99）→ 806 GB（1560 亿 token）。其 WebText 式变体（用 OpenWebText 外链页）改善了 GLUE/SQuAD。
- **GPT-3**：Common Crawl（已处理）+ WebText2 + Books1/2 + Wikipedia → 570GB（4000 亿 token）。CC 的处理方式是**质量分类器**（区分 {WebText, Wikipedia, Books} 与其余）+ 模糊去重。
- **The Pile**（EleutherAI）：22 个精选领域、825GB（约 2750 亿 token）：Pile-CC（WARC + jusText）、PubMed Central、arXiv（LaTeX）、Enron 邮件、Project Gutenberg、**Books3**（来自影子图书馆 Bibliotik 的 19.6 万本书——含 Stephen King 等，已因侵权下架）、StackExchange（Q&A，带元数据的 XML dump）。
- **Gopher 的 MassiveText**：MassiveWeb（英文、去重、基于规则的质量过滤——"80% 的词至少含一个字母"、用 Google SafeSearch 而非脏词表做毒性过滤）+ C4 + Books/News/GitHub/Wikipedia → 10.5TB（Gopher 只训练了其中 3000 亿 token，占 12%）。
- **LLaMA**：CommonCrawl 经 CCNet（分类"是否被 Wikipedia 引用"）、C4、GitHub（保留宽松许可 + 手工规则）、Wikipedia（20 种语言）、Gutenberg + Books3、arXiv（去掉注释/宏/参考文献）、Stack Exchange（按得分排序的前 28 个站点）→ 1.2T token。由 Together 的 RedPajama v1 复现；SlimPajama 是用 MinHashLSH 去重后的 627B 子集。
- **RefinedWeb**（Falcon）："网络数据就够了"——用 trafilatura 处理 WARC（而非 WET）、Gopher 规则、刻意不用 ML 过滤（避免引入偏差）、MinHash 模糊去重 → 发布 6000 亿（从 5T 中筛出）。**FineWeb**：95 次 CC dump、URL 过滤、语言识别（p(en) > 0.65）、Gopher+C4 规则、MinHash、邮箱与公网 IP 脱敏 → 15T token。
- **Dolma**（AI2）：Reddit（Pushshift）、PeS2o（4000 万篇论文）、C4、Gutenberg、Wikipedia；CC 部分用 fastText 语言识别、Gopher+C4 规则、Jigsaw 毒性分类器、Bloom filter 去重 → 3T token。
- **DCLM（DataComp-LM）**：DCLM-pool（240T token 的处理过的 CC）；DCLM-baseline 用**质量分类器**过滤——正样本（OpenHermes-2.5、ELI5，各 20 万）vs 负样本（RefinedWeb，20 万）训练

fastText 分类器，效果优于其他过滤方法 → 3.8T token。基于模型的过滤"正在成为常态"。

- **Nemotron-CC**：FineWebEdu/DCLM 过滤过于激进（删掉 90% 数据）；用集成分类器（把 Nemotron-340B 的"教育价值"打分蒸馏到小模型 + DCLM 分类器）；对低质量数据做**合成改写**、对高质量数据生成任务 → 6.3T token（其中高质量子集 1.1T）。参考：Llama 3 用 15T，Qwen3 用 36T。
- **The Stack**：1.37 亿仓库（按 GitHub Archive 名字 git clone，2015—2022）、510 亿文件（其中 50 亿唯一）；只保留宽松许可（go-license-detector）；MinHash 近重复去除 → 3.1TB。**Stack v2**：加入 issue/PR/评论、Software Heritage、文档站点；移除二进制/恶意软件/机器人活动；把低资源语言与 LLVM IR 配对；把 PR 的 diff 线性化以用于训练。
- **CommonPile**：8TB **仅宽松许可**的数据——能否只靠合法数据训出好模型？结果尚可，但"没有更多 token 很难竞争"。微妙之处：许可洗白（license laundering）、集合许可不延伸到单件作品、基于未授权数据训练的模型生成的合成数据法律地位不明。

代码示例：用分类器做质量过滤（DCLM/GPT-3 模式）

代码（Python）：

```
import numpy as np

# 已知：目标数据 T（"好"是什么样子）与海量原始数据 R。
# 1. 训练 fastText 风格分类器：T 作正样本、R 作负样本，
#    score(x) = p(好 | x)
# 2. 按分数（随机地）保留文档。

def keep_document_gpt3_style(score: float) -> bool:
    # GPT-3 按分数决定的概率保留（Pareto(9) 抽样 > 1 - score）
    return np.random.pareto(9) > 1 - score

def keep_document_threshold(score: float, thresh: float = 0.5) -> bool:
    return score >= thresh  # Dolma 风格：保留 p(英语) >= 0.5 的页面
```

代码做了什么： 展示两种保留策略：随机保留（GPT-3：高分文档几乎总保留、低分文档偶尔保留——一种软过滤）与硬阈值（Dolma）。

实现深挖： – **为什么要随机**：硬截断可能脆弱；GPT-3 的 Pareto(9) 抽样实现"以约等于 score 的概率保留"的平滑效果。现代流水线（DCLM）直接用分类器分数阈值。 – **为什么用 fastText**：过滤要跑在数百 TB 上——速度是硬要求（"极快"是筛选算法的必备性质）；fastText 线性分类器的推理速度比神经打分器快几个数量级。 – **为什么这套配方可推广**："目标数据 T vs 原始数据 R"的框架可覆盖语言识别（T = 英文页面）、质量（T = 精选/指令数据）、毒性（T = 干净评论）——第 14 讲的过滤讲座正是建立在这个框架上。

与作业的联系：作业 4 的核心任务就是这个配方：先把 Common Crawl 的 HTML 转文本（trafilatura），再用 (a) 给定的 NSFW/仇恨言论分类器、(b) Gopher/C4 式规则、(c) PII 移除来过滤，最后用 MinHash 去重（第 14 讲）。DCLM 的质量分类器思路解释了为什么作业直接给你预训练好的分类器，而不是让你从零做启发式规则。

关键要点

1. 数据不会从天而降：在线服务 → 抓取/dump → 处理后的数据；每一步都有技术约束（爬取、动态内容、登录）与法律约束（ToS、版权）。
2. 互联网上几乎所有内容都有版权；要么取得许可，要么主张合理使用（四要素）。目前的法律状态：具体案件中训练被认定为合理使用；盗版复制则不是。
3. 数据集谱系显示出持续"加重处理"的趋势：基于规则的启发式（C4、Gopher）→ ML 质量分类器（GPT-3、DCLM、Nemotron-CC）→ 合成改写（Nemotron-CC）→ 只用合法数据（CommonPile）。
4. 关键来源：Common Crawl（网页，WARC/WET）、Wikipedia（dump）、GitHub（git 协议 + archive）、arXiv（S3 dump）——各有各的获取方式。
5. 数据是语言模型之间的关键差异所在：公司严防死守；开放模型会公开除数据之外的一切。

常见陷阱

- **有 WARC 却用 WET**：HTML→文本的转换质量（trafilatura vs WET）会可测量地影响下游精度（DCLM）。
- **忽视 ToS**：即使内容是 CC 许可，若平台 ToS 禁止下载，也依然不可用（YouTube 的例子）。
- **去重与质量过滤的顺序/粒度不当**：粒度（C4 用 3 句片段）与顺序都很重要；从文档中间删掉片段会破坏连贯性。
- **使用影子图书馆的受版权数据（Books3）**：已被下架，且有诉讼风险——优先使用已许可/公共领域来源。
- **盲信任何单一来源**：连 Wikipedia 都可能在 dump 前被投毒；过滤与去重永远必要。
- **忘记 dump ≠ 在线服务**：Common Crawl/GitHub Archive 是快照，自带偏差与缺漏。

复习题

1. **问：** 合理使用的四个要素是什么？对 LLM 训练各自往哪个方向推？
2. **答：** (1) 目的与性质——转化性/教育性有利（训练可视为转化性使用）；(2) 作品性质——事实性优于创作性（书籍属创作性，不利）；(3) 使用数量——用片段优于用全作（训练用了全作）；(4) 市场影响——LM 可能替代作家。法院目前在具体案件中认定训练属合理使用。
3. **问：** GPT-3 的 Common Crawl 处理为什么优于 C4，尽管原始抓取相同？
4. **答：** GPT-3 使用**学习得到的质量分类器**（区分 WebText/Wikipedia/Books 与其余）加模糊去重，而 C4 用的是固定手工启发式（行/句/标点规则、脏词表）。基于分类器的过滤更能泛化"好数据"的模样——这也是 DCLM 后来形式化的主题。
5. **问：** WARC 与 WET 有何区别？为什么重要？
6. **答：** WARC 存原始 HTTP 响应（HTML），WET 存有损的 HTML→文本转换结果。下游精度取决于转换质量（DCLM 中 trafilatura 优于 WET），所以现代流水线多用更好的工具从 WARC 重新抽取文本（The Pile 用 jusText，RefinedWeb 用 trafilatura）。

第 14 讲：数据 II — 转换、过滤、去重、混合与合成数据

日期：5 月 13 日（周三，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_14.py`

概览

这是数据工程**算法核心**的一讲。覆盖流水线四个阶段——**转换（transformation）**（HTML/PDF → 文本）、**过滤（filtering）**（语言识别、质量、毒性，基于分类器）、**去重（deduplication）**（精确去重与哈希、MinHash、LSH）、**数据混合（data mixing）**（如何给各来源加权、epoch 陷阱、UniMax 上限、回归式混合、模拟 epoch）——最后讲**后训练 / 合成数据**（OpenThoughts、SWE-smith、SWE-Zero 等）。

核心概念与定义

- **转换**：原始数据不是文本，而是 HTML、PDF 或目录。HTML→文本：去掉 boilerplate（导航、广告）、抽正文、把表格/图片线性化（有损）。工具：trafilatura、resiliparse、jusText、lynx。质量很重要（DCLM）。FinePDFs：对 PDF 重新抓取 + OCR（RolmOCR/Docling）+ 清洗。
- **过滤——算法积木**：给定**目标数据 T** 与**原始数据 R**，找出与 T 相似的子集 $T' \subset R$。两步框架：(1) 基于 R 与 T 估计一个模型 → 得到打分函数；(2) 按分数保留样本。类型：**T 的生成式模型**（KenLM：score(x) = p_T(x)）或**分类器**（fastText：score(x) = p(T|x)）；按阈值（随机地）保留。
- *必备性质*：能从目标数据泛化（$T' \neq T$），并且**极快**（R 巨大）。
- *应用*：语言识别（fastText lid.176，176 种语言；Dolma 保留 p(en) ⩾ 0.5）、质量过滤、毒性过滤（Jigsaw Toxic Comments，6 个标签）。
- *基于模型 vs 基于规则*：C4/Gopher/RefinedWeb/FineWeb/Dolma 刻意不用模型过滤；GPT-3/LLaMA/DCLM 用——"正成为常态"。
- *案例*：OpenMathText（规则 + KenLM 困惑度 < 15000 + fastText 数学分类器 → 147 亿 token，效果超过 20 倍数据量的模型）；GPT-3（词特征线性分类器 + Pareto-9 随机保留）；LLaMA/RedPajama（正样本 = 被 Wikipedia **引用** 的页面）；phi-1（用 GPT-4 给 The Stack 的 Python 子集打"教育价值"标签 → 用 codegen 模型嵌入训练随机森林 → HumanEval 12.19% → 17.68%，且步数只有 1/3）；Dolma 的毒性过滤（Jigsaw 分类器）。
- *过滤的规模依赖*：并不存在唯一最优阈值——训练越久越需要更多（更低质量）数据；训练越短越需要更少（更高质量）数据。
- **去重（deduplication）**：精确重复（镜像站、fork）与近似重复（服务条款页面、模板化文本——某商品描述在 C4 中重复了 61036 次）。*为什么去重*：训练更高效（token 更少）并避免记忆（版权/隐私）。
- *设计空间*：(1) 以什么为"条目"（句子/段落/文档）；(2) 如何匹配（精确匹配、存在公共子条目、公共子条目比例）；(3) 采取什么动作（全删 / 只留一个）。
- *关键挑战*：比较条目与条目需要**线性时间**算法才能扩展到海量数据。
- **哈希（hashing）**：把条目映射为小的哈希值。密码学哈希（SHA-256）：抗碰撞、慢。非密码学哈希（MurmurHash、DJB2、CityHash）：快，用于哈希表。去重用 MurmurHash。

- **精确去重**：按哈希分组、每组留一个（MapReduce 风格，天然可并行）。C4：条目 = 3 句片段、精确匹配、只留一个——但从文档中间删除片段会破坏连贯性。
- **Jaccard 相似度**：J(A,B) = |A∩B| / |A∪B|；近似重复定义为 Jaccard ⩾ 阈值。
- **MinHash**：一种哈希方案，满足 **Pr[h(A) = h(B)] = Jaccard(A,B)**——这里你**希望**碰撞概率与相似度挂钩（与普通哈希相反！）。`minhash(S, seed) = min(mmh3.hash(x, seed) for x in S)`；用多个种子时，最小哈希相等的比例即为 Jaccard 的估计。
- *为什么成立*：随机哈希诱导出对元素的随机排列；集合的最小元素在其元素之间均匀分布，故 A 与 B 共享最小值当且仅当 A∪B 的全局最小元素属于 A∩B——概率为 |A∩B|/|A∪B|。
- **局部敏感哈希（LSH）**：把碰撞概率"锐化"成阈值判定。用 n = b·r 个哈希函数分成 b 个 band、每 band r 个：A 与 B 碰撞当且仅当**某个 band 内全部 r 个哈希都相等**。碰撞概率 P = 1 − (1 − s^r)^b——关于相似度 s 的 S 形曲线；相变点位于 s* = (1/b)^(1/r)。增大 r 会锐化曲线并把阈值右移（更难匹配）；增大 b 则左移（更容易）。真实配置（Lee 等 2021）：n=9000、b=20、r=450 → 阈值 ≈ (1/20)^(1/450) ≈ 0.993。在阈值处 P(碰撞) ≈ 1 − 1/e。
- **数据混合**：各来源的分布 p(s) 该怎么定？基线：凭感觉（手工）、均匀采样、按 token 数比例采样。两个直觉冲突：要给高质量来源加权，但每个来源都是有限的——在小的高质量来源上过度 epoch 会导致过拟合（例：10B token 的来源在 p=0.5、训练 1T token 时 = 50 个 epoch！）。
- **UniMax**：均匀采样 + 对每个来源的 epoch 数设**硬上限 C**：p(s)·train_tokens ⩽ C——用于多语模型的语言平衡。
- **回归式混合（RegMix）**：定义混合分布的分布（如 Dirichlet），训练小模型，回归"混合 → 损失"（线性/GBT），再优化；两个希望：(1) 回归在最优点附近准确，(2) 最优混合能迁移到大尺度。
- **模拟 epoch（simulated epoching）**：按相同比例对所有来源降采样，让小规模跑出与大规模相同的 epoch 结构——于是小规模拟合出的最优混合能迁移。
- **后训练 / 合成数据配方**：(1) 定义环境；(2) 定义任务/提示；(3) 用强教师模型收集回答。例子：OpenThoughts（用 QwQ-32B 造 120 万条；每提示采样 16 条有帮助；更强的模型不一定是更好的教师——QwQ-32B 优于 DeepSeek-R1；答案过滤没帮助；小而精的来源优于大而杂的来源）；SWE-smith（用 LM 往仓库注入 bug 来生成任务；128 个仓库产出 5 万个任务）；SWE-Zero（30 万条不依赖仓库特定执行的 agent 轨迹——强模型内部具备代码语义的"世界模型"；15 万个 GitHub PR；从 Qwen3-Coder-480B 蒸馏）；SWE-rebench（2.1 万个可交互的 Python SWE 任务）；SWE-ZERO-12M-trajectories（用 1.7B 的小 agent 把规模推到 1200 万条轨迹）。

代码示例：基于哈希的精确去重

代码（Python）：

```
import itertools, mmh3

items = ["Hello!", "hello", "hello there", "hello", "hi", "bye"]

# 按哈希分组、每组留一个（MapReduce 风格，可并行）
hash_items = itertools.groupby(sorted(items, key=mmh3.hash), key=mmh3.hash)
deduped_items = [next(group) for h, group in hash_items]
# -> "hello" 只出现一次；"Hello!" 与之不同（字节不同）
```

代码做了什么： 按 MurmurHash 值排序、把相同哈希分到一组、每组取第一个——线性时间的精确去重。

实现深挖： – **为什么用 MurmurHash**：快速的非密码学哈希——这里碰撞可以接受（我们要去重而非加密）；在 TB 级数据上用密码学哈希会慢得不必要。 – **为什么"排序后 groupby"**：与哈希分桶等价，但写成 MapReduce 友好的形式——该模式可在分片数据上跨 worker 并行（作业 4 正是用 `concurrent.futures` 处理 WET 文件）。 – **为什么 "Hello!" ≠ "hello"**：哈希作用于字节——大小写与标点差异会让近乎相同的文本得到不同哈希；这是精确去重无法解决的局限（所以需要 MinHash）。

与作业的联系：作业 4 的去重任务以段落/文档的精确哈希为基线，再用 MinHash 做近重复去除——这个片段就是起点，只是要扩展到在规模化语料上对分词后的文档操作。

代码示例：MinHash 与 LSH（作业 4 的核心）

代码（Python）：

```
import mmh3

def jaccard(A, B):
    return len(A & B) / len(A | B)

def minhash(S: set[str], seed: int) -> int:
    """MinHash：Pr[minhash(A) == minhash(B)] = Jaccard(A, B)。"""
    return min(mmh3.hash(x, seed) for x in S)

def get_prob_collision(sim, b, r):
    prob_match = sim ** r                    # 一个 band 内 r 个哈希全部相同
    return 1 - (1 - prob_match) ** b         # 某个 band 匹配

# 验证估计量：
A = {"1", "2", "3", "4"}; B = {"1", "2", "3", "5"}
true_j = jaccard(A, B)
n = 100
matches = [minhash(A, seed) == minhash(B, seed) for seed in range(n)]
assert abs(sum(matches)/n - true_j) < 0.01   # 估计值 ≈ 真实 Jaccard

# LSH：b 个 band、每 band r 个哈希；碰撞概率是一条陡峭的 S 形曲线
p80 = get_prob_collision(sim=0.8, b=10, r=10)   # 接近 1
p20 = get_prob_collision(sim=0.2, b=10, r=10)   # 接近 0
threshold = (1 / b) ** (1 / r)                  # 相变点位置
```

代码做了什么： 实现 MinHash（对集合在某个种子下的最小哈希），用实验验证 Jaccard 估计量，并计算 LSH 的 band 碰撞概率——展示把"相似度"变成"近似阈值判定"的 S 形曲线。

实现深挖： – **为什么取最小**：对随机哈希来说，A∪B 中每个元素成为最小值的概率相同；A 的最小值等于 B 的最小值，当且仅当全局最小元素落在 A∩B 中 → 概率 = |A∩B|/|A∪B| = Jaccard。 – **为什么用 band（b·r）**：单个哈希的碰撞概率就是 Jaccard 本身——太"软"。band 内的"与"（r 个都相同：s^r）加上跨 band 的"或"（b 个中任一：1−(1−s^r)^b）把它锐化成以 (1/b)^(1/r) 为中心的阶跃。 – **为什么参数重要**：n=9000、b=20、r=450（真实配置）瞄准相似度 ⩾ ~0.993 的近重复——你要找的是**几乎完全相同**的文档，而非大致相关的文档。调 (b, r) 就是权衡假阳性与假阴性。 – **为什么这是作业 4 的瓶颈**：在数百 GB 上做去重需要线性、近似、可并行的匹配——MinHash + LSH 正是如此（避免 $O(n^2)$ 的两两比较）。

与作业的联系：作业 4：在过滤后的语料上实现 MinHash 去重（条目 = 文档/段落，用 shingle token 的 Jaccard 匹配，每个近重复簇保留一个），并测量去重对困惑度的影响。讲义的 `get_prob_collision` 与阈值数学就是你在报告中论证 (b, r) 取值的依据。

代码示例：数据混合与 epoch 陷阱

代码（Python）：

```
def num_epochs(source_tokens: float, weight: float, train_tokens: float) -> float:
    return (weight * train_tokens) / source_tokens

# 陷阱：小规模高质量来源被反复读取
sources = {"low": 10e12, "high": 10e9}        # 10T vs 10B token
p = {"low": 0.5, "high": 0.5}                  # 天真的 50/50 混合
train = 1e12                                   # 训练 1T token
epochs = {s: num_epochs(sources[s], p[s], train) for s in sources}
# epochs["high"] == 50  -> 稀缺来源被过 50 遍：过拟合!

# UniMax：给每个来源的 epoch 数设硬上限
C = 2
p = {s: min(p[s], C * sources[s] / train) for s in sources}   # 之后需重新归一化

# 模拟 epoch：按相同比例对所有来源降采样
ratio = 10e9 / 1e12                            # 小规模运行 / 大规模运行
downsampled = {s: sources[s] * ratio for s in sources}   # 小规模的"等价"数据量
```

代码做了什么： 计算天真混合下每个来源的 epoch 数（暴露 50 个 epoch 的过拟合陷阱），应用 UniMax 式 epoch 上限，并展示模拟 epoch 的比例降采样。

实现深挖： – **为什么 epoch 数重要**：一个来源被看 50 遍就会被记住；在小规模上最优的混合（偏向稀缺高质量数据）在大规模上**不是**最优——这是会破坏"混合迁移"的规模依赖效应。 – **为什么模拟 epoch 能修复迁移**：把所有来源都降采样到小规模运行的 token 预算，让小规模实验看到与大规模运行相同的 **epoch 结构**；此时拟合出的最优点才能迁移（第 9 讲"让小规模看起来像大规模"的主题）。 – **为什么上限是务实解法（UniMax）**：给每个来源的 epoch 数设硬上限即可避免病态的过度重复，无需重新拟合——这是多语混合的标准技巧。

与作业的联系：作业 4 的最后一步是在 **token 预算**下混合各来源（排行榜：在给定 token 数下最小化困惑度）：epoch 陷阱与 UniMax 上限正是"每个来源该保留多少过滤后数据"的核心考量。作业 3 的扩展律推理用的是同一套"从小到大迁移"的逻辑。

关键要点

1. 数据流水线：转换（HTML→文本）→ 过滤（语言/质量/毒性的分类器）→ 去重（精确 + MinHash/LSH）→ 混合（权重、上限、模拟 epoch）。
2. 过滤 = "在原始数据中找出与目标相似的部分"：用生成式模型（KenLM）或分类器（fastText）打分；按阈值或随机保留；必须能泛化且极快。
3. 规模化去重需要线性时间的近似匹配：MurmurHash 做精确去重；MinHash（Pr[碰撞] = Jaccard）+ LSH band（阈值在 (1/b)^(1/r) 的 S 形曲线）做近重复。
4. 混合存在规模依赖：天真权重会过度 epoch 稀缺来源（50 epoch 陷阱）；用 UniMax 上限或模拟 epoch 让小规模最优点可迁移。
5. 后训练数据越来越合成化：教师模型 + 环境 + 过滤（OpenThoughts、SWE-Zero）——而更小但更高质量的来源常常优于更大更杂的来源。

常见陷阱

- **O(n²) 去重**：两两比较全部文档不可行；必须基于哈希（精确或 MinHash）。
- **LSH 参数选错**：(b, r) 决定阈值——b 太小/r 太大会漏掉近重复；相变点在 (1/b)^(1/r)，要按目标相似度来选。
- **把哈希碰撞当真值**：MurmurHash 会有碰撞；精确去重是"高精度"但非完美；MinHash 是对 Jaccard 的有方差估计——哈希个数（n）要足够。
- **过度 epoch 稀缺来源**：天真的按比例或按质量加权会记住小来源；要设上限或做模拟 epoch。
- **跨规模用固定过滤阈值**：最优阈值随训练预算变化（训练越久越应保留更多数据）。
- **去重破坏文档连贯性**：C4 式删除文档中间片段会产出不连贯文本——要检查去重单位对下游质量的影响。
- **分类器正负样本有偏**：你的质量过滤器会继承所选目标数据的偏差（例如 RefinedWeb 作负样本）。

复习题

1. **问：** 为什么 MinHash 的碰撞概率等于 Jaccard 相似度?
2. **答：** 随机哈希置换让 A∪B 中每个元素成为最小值的概率相同。A 与 B 的最小值相同，当且仅当全局最小元素落在 A∩B 中——概率为 |A∩B|/|A∪B| = Jaccard。
3. **问：** LSH 如何把"碰撞概率 = 相似度"变为硬阈值?
4. **答：** 用 b 个 band、每 band r 个哈希，只要**某个** band 内 r 个最小哈希全部相等就判为碰撞：P = 1−(1−s^r)^b。这是关于相似度 s 的 S 形曲线，陡峭的过渡位于 s *= (1/b)^(1/r)*——*高于 s* 的近重复几乎必然碰撞，远低于 s* 的几乎不碰撞。

5. **问：** 为什么在小规模上最优的混合在大规模上会失败？怎么修？
6. **答：** 随着训练 token 增多，稀缺的高质量来源会被过度 epoch（例子里是 50 个 epoch）——小规模最优点给了它们过高权重。修法：UniMax 对每来源 epoch 数设硬上限，或模拟 epoch（按比例降采样所有来源）让小规模复现大规模的 epoch 结构。

第 15 讲：中训练/后训练（SFT 与 RLHF）

日期：5 月 18 日（周一，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_15.pdf`

概览

预训练把你带到 GPT-3；本讲讲通向 instructGPT 的路径：先在指令数据上做**监督微调（SFT）**，再做 **RLHF**（基于人类反馈的强化学习，用 PPO 或 DPO）。内容包括指令数据的真实样貌（FLAN → Alpaca → OpenAssistant → Nemotron 智能体数据）、SFT 数据中"风格/长度/知识/安全"的微妙影响、从"模仿"转向"奖励优化"、成对反馈数据的采集，以及 RLHF 的两个主要陷阱：过度优化（overoptimization）与模式崩塌（mode collapse）。

核心概念与定义

- **G-V gap（生成-评价差距）**："人们并不总是写出自己偏好的东西"——模仿（SFT）是在拟合 $p(y|x)$，*但用户偏好的是一个需要优化*的奖励，而不是一个需要模仿的分布。这是 RLHF 的根本动机。
- **SFT 数据谱系**：FLAN（基准风格任务、言简意赅）→ Self-Instruct / Alpaca（LLM 生成的指令跟随数据；由 GPT-3.5/4 生成，52K 条）→ ShareGPT/Vicuna（真实人机对话）→ OpenAssistant（众包、细节丰富、知识密集）→ WizardLM → Tulu 3 → Nemotron（智能体式：工具调用、多轮、感知 AGENTS.md）。
- *差异在哪*：话多不多/长度、细节程度、工具使用、规模、安全。*影响到什么*：风格（人类与 GPT 裁判都表现出强烈的长度效应）、基准表现（多数不受风格影响）以及事实性。
- **知识抽取与对齐**：在模型不知道的"长尾知识"上微调会让它**幻觉**（Schulman 2023；Gekhman 等）——"即使 LM 的用例需要这些知识，你也可能不该在长尾知识上微调"。SFT 擅长**抽取**预训练已具备的行为，而非注入新行为；加入（事实上正确的）数据有时反而有害。
- **安全 SFT**：几千条样本就能教会拒绝行为（Llama 2）；约 500 条安全样本 + 500 条 Alpaca 风格样本就能让模型遵循安全准则。安全数据 = 从用户处提取的场景。
- **把 SFT 变回"预训练延续"**：把指令数据混进预训练（mid-training / 两阶段训练），最后再做一轮很短的指令微调——这样能在不灾难性遗忘的前提下扩展指令调优规模（miniCPM、jetMoE；"这是很多 LLM 公司的常识，但没被写进文档"）。
- **RLHF 数据**：成对反馈（chosen vs rejected）。来源：人类（众包——很难验证正确性、存在伦理问题、标注人群分布会改变模型行为）、专家标注（昂贵且在增长）、以及 **LM 生成的反馈**（GPT-4 的一致率接近人类标注者之间的一致率，系统级排序相关性近乎完美）——Zephyr（UltraFeedback）、Tulu 3、OLMo 都在用。此外还有自训练（Constitutional AI：批评 + 修改）。

- *长度效应*：RLHF 系统性地产生更长的回答（人类与 AI 反馈都会奖励长回答）——这是显著且常常不受欢迎的副作用。
- **PPO（Proximal Policy Optimization）**——最初的 RLHF 算法（InstructGPT）：
- *谱系*：策略梯度（方差太大：∇E[R] = E[R∇log p]）→ TRPO（在当前策略附近线性化、设信任域）→ PPO（把重要性比裁剪到 ±ε）。
- *在 LM 中*：动作 = token；奖励稀疏且在序列末尾（序列级）；需要一个价值模型 + 奖励模型；对参考策略有逐 token 的 KL 惩罚；用广义优势估计（GAE）——在 bandit 设定下 γ=λ=1 即可，即"reward-to-go 减去 value"。
- *实践*：外层 rollout 循环 + 内层优化循环；奖励塑形 = 末 token 奖励 + KL；cliprange 0.2；当新策略 logprob < 参考策略时裁剪 KL（稳定性）。
- *代价*：实现复杂、价值模型吃显存、需要额外调参。
- **DPO（Direct Preference Optimization）**——"不用流泪的 RLHF"：
- *思路*：在**非参数假设**下（策略可以是任意分布），RLHF 目标的闭式最优解把奖励与策略联系起来：r(x,y) = β·log(π(y|x)/π_ref(y|x)) + β·log Z(x)。把这个"隐含奖励"代入偏好（Bradley-Terry / Stiennon）损失 → 得到直接作用于偏好对的监督损失，无需奖励模型、无需 rollout。
- *解释*："对好东西加正梯度、对坏东西加负梯度"，幅度由隐含奖励模型的预测误差加权。
- *变体*：SimPO（不用参考模型）、长度归一化 DPO（Tulu 3）、IPO 等。
- **RLHF 的陷阱**：
- **过度优化**：越过某一点后继续优化奖励会损害真实质量——对人类偏好与有噪声的 LM 偏好都成立，但对无噪声的 LM 偏好不成立（那是在过拟合奖励模型）。
- **模式崩塌 / 熵坍缩**：RLHF 后的模型不再是合法的概率模型——默认失去校准性。
- *高度情境依赖*："很多结果高度取决于实验设置的具体细节"——PPO 有时优于 DPO，有时相反。

代码示例：DPO 损失（作业 5 可选补充部分的核心）

代码（Python）：

```
import torch
import torch.nn.functional as F

def dpo_loss(log_pi_w, log_pi_l, log_ref_w, log_ref_l, beta=0.1):
    """DPO：在偏好对上使用隐含奖励的 Bradley-Terry 损失。
    log_pi_w/l：chosen/rejected 在当前策略下的对数概率。
    log_ref_w/l：同一序列在冻结参考策略下的对数概率。
    """
    log_ratio_w = log_pi_w - log_ref_w          # chosen 的隐含奖励
    log_ratio_l = log_pi_l - log_ref_l          # rejected 的隐含奖励
    logits = beta * (log_ratio_w - log_ratio_l) # Bradley-Terry logit
    return -F.logsigmoid(logits).mean()          # 最大化 P(chosen > rejected)
```

代码做了什么： 实现 DPO 目标：损失为 −log σ(β·(r_w − r_l))，其中奖励由策略与参考策略的对数比"隐含"给出——一个作用于偏好对的简单二分类损失。

实现深挖： – **为什么不需要奖励模型**：DPO 把奖励重参数化为 β·log(π/π_ref)（外加一个在成对差分中抵消的配分常数），于是"奖励模型"就是策略本身。这正是讲义里的非参数最优技巧：先把 RLHF 的约束优化求出闭式解，再把隐含奖励代入偏好损失。 – **为什么有 β**：它是 KL 正则强度；β 越大越靠近参考策略。它也缩放梯度：更新是"对 chosen 加正、对 rejected 加负"，并按隐含奖励错得有多离谱来加权。 – **为什么冻结参考模型**：π_ref 锚住更新；若去掉它（SimPO），就需要其他归一化（如长度归一化）来避免奖励被"薅"。

与作业的联系：作业 5 的可选补充部分（在 Llama 3.1 8B 上做 SFT + DPO，数据用 Anthropic HH 偏好对）实现的就是这个损失。必修的作业 5（GRPO/RLVR）是它的**同策略（on-policy）**表亲——讲义中 PPO→DPO 的对比解释了课程为何为必修部分选择 GRPO（无需价值模型、无需奖励模型、奖励可验证）。

代码示例：PPO 风格的裁剪代理目标（概念）

代码（Python）：

```
def ppo_loss(log_pi_new, log_pi_old, advantages, clip_eps=0.2):
    ratio = torch.exp(log_pi_new - log_pi_old)      # 重要性比
    unclipped = ratio * advantages
    clipped = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) * advantages
    return -torch.min(unclipped, clipped).mean()    # 悲观下界

# LM 版本：对参考策略的逐 token KL 惩罚 + 末 token 奖励
# advantages = (reward - value) 或（GRPO 中）组内归一化奖励
```

代码做了什么： 计算 PPO 的裁剪目标：取未裁剪与裁剪后代理目标的较小值，从而防止策略在一次更新中跑得太远。

实现深挖： – **为什么要裁剪**：当新策略偏离旧策略时重要性比会爆炸；裁剪到 [1−ε, 1+ε] 给更新设了上界——把 TRPO 的信任域落到实处。 – **为什么 LM 场景仍需要它**：奖励稀疏且在序列级；对参考策略的逐 token KL 惩罚是防止漂移的主要手段，而 γ=λ=1 的 GAE 在 bandit 设定下退化为"奖励减价值"。 – **为什么课程更偏好 GRPO（下一讲）**：PPO 的价值模型吃显存且难调；在可验证奖励（数学题）下，用组内归一化优势可以完全去掉价值模型。

与作业的联系：作业 5 的必修部分使用 **GRPO**（去掉价值函数的 PPO 变体）——这段 PPO 背景是作业要求你理解的基线，而作业的"策略梯度估计器变体"任务（重要性权重裁剪）正是该目标的直接推广。

关键要点

1. 模仿（SFT）抽取预训练行为；优化（RLHF）对准偏好——G–V gap 说明"示范"不等于"偏好"。
2. SFT 数据的质量以微妙方式胜过数量：风格/长度驱动偏好判断，长尾知识一旦微调就会引发幻觉，而少量恰当的安全/指令数据就能带来很大改变。
3. RLHF = 带 KL 控制的奖励优化：PPO（价值模型 + 裁剪）是经典但难调的做法；DPO 用非参数隐含奖励技巧去掉了奖励模型。

4. 成对反馈是通用货币：人类有噪声、标注人群分布会改变行为，LM 裁判出人意料地好（接近人类一致率）——但所有反馈都偏爱长回答。
5. 警惕过度优化（过拟合奖励）与模式崩塌（不再是校准好的分布）。

常见陷阱

- **在长尾知识上做 SFT**：微调模型不知道的事实会诱发幻觉；SFT 应以抽取为主，而非注入。
- **忽视长度偏差**：人类与 LLM 裁判都奖励更长的回答——要做长度归一化或去偏（包括 DPO 的变体）。
- **RLHF 中省掉 KL 控制**：没有参考策略惩罚，模型会漂移并崩塌。
- **过拟合奖励**："越过某点后优化奖励就会过拟合"——要在留出评测上监控真实质量并早停。
- **信任有噪声的标注者**：众包的正确性难以验证；带清单的专家/LM 反馈更可靠。
- **跳过中训练直接后训练**：忘记两阶段配方会导致通用能力灾难性遗忘。

复习题

1. **问：** 为什么 DPO 既不需要奖励模型也不需要 rollout？是什么假设让它成立？
2. **答：** 非参数假设（最优策略可以是任意分布）让带 KL 正则的 RLHF 目标有闭式解，从而 r(x,y) = β·log(π/π_ref) + 常数。把这个隐含奖励代入 Bradley–Terry 偏好损失后，RLHF 就变成一个作用于偏好对的监督分类损失。
3. **问：** 什么是 G–V gap？为什么它使 RLHF 优于纯 SFT？
4. **答：** 人们未必写出自己偏好的内容（生成 ≠ 评价）。SFT 拟合的是示范回答的分布；RLHF 最大化反映真实偏好的奖励——即使数据相同，两个目标也不同。
5. **问：** RLHF 的两个主要失效模式是什么？实践者如何缓解？
6. **答：** (1) 过度优化——奖励继续上升而真实质量下降；通过 KL 正则、留出评测、早停来缓解。(2) 模式崩塌/熵坍缩——模型不再是校准好的分布；通过熵奖励/KL 约束、长度归一化来缓解。

第 16 讲：后训练 II —— 可验证奖励的强化学习（RLVR）

日期：5 月 20 日（周三，Spring 2026）| 讲师：Tatsu Hashimoto | 材料： `lecture_16.pdf` *| 截止：作业 4 到期、作业 5 发布*

概览

RLHF 难以干净地扩展（过度优化）；而**可验证奖励的强化学习（RLVR）**可以——在奖励精确可验证的领域（数学答案对错、测试是否通过）中优化你真正想要的东西。本讲覆盖 PPO→GRPO、GRPO 的变体与缺陷（baseline 的合法性、长度偏差），以及三个案例研究：**DeepSeek-R1**（GRPO、R1-zero、SFT+RL 配方、蒸馏）、**Kimi K1.5**（长度控制、课程学习、RL 基础设施）与 **Qwen 3**（低数据量 RLVR、思考模式融合、智能体 RL）。

核心概念与定义

- **RLVR**：奖励**可验证**（答案与标准答案一致、测试通过）而非从人类偏好学来的强化学习。它绕开了奖励模型的过度优化问题，并支持干净的扩展（即通向 o1/r1 的路径）。
- **策略梯度谱系**：∇E[R] = E[R·∇log p]（高方差）→ TRPO（信任域）→ PPO（裁剪重要性比）→ **GRPO**（去掉价值模型、使用组内归一化奖励）。
- **LM 中的 PPO 回顾**：动作 = token；末尾一个大的稠密奖励；逐 token KL 惩罚；γ=λ=1 的 GAE（bandit 设定：优势 = reward-to-go − value）。实现复杂度：外层 rollout 循环、内层优化、价值模型（吃显存、需额外调参）、奖励塑形（末 token 奖励 + KL）、裁剪。
- **为什么不用 PPO / 为什么不用 DPO 做推理**：PPO 复杂且价值模型吃显存；DPO 需要成对（Bradley-Terry）数据且是离线的。GRPO：没有价值函数、不需要成对数据、同策略在线——"你完全可以（而且确实有人）写出极小的 GRPO 实现"。
- **GRPO**：
- *优势*：对每个提示采样一组 G 条回答；优势 = (奖励 − 组均值)/组标准差——"组内 z-score"。在同策略在线设定下，它就是"带组内归一化奖励的策略梯度"。
- *目标*：loss = −(1/G)Σ Σ_t [advantage·min(ratio, clip(ratio)) − β·KL]（含对参考策略的逐 token KL 与长度归一化）。
- *算法*：为每条 rollout 计算奖励 → 按组做均值/方差归一化 → 计算 KL 项 → 对损失做梯度更新。
- **GRPO 的理论缺陷**（一段"小型 RL 绕路"）：减去**组均值**是合法 baseline（无偏），但**除以组标准差不是合法 baseline**——它会让梯度产生偏差。无偏变体（Liu 等 2025）接近于带 leave-one-out 的 REINFORCE。此外标准 GRPO 目标存在**长度偏差**：标准差项会放大过易/过难题目的权重；长度归一化与之相互作用，修法需要重新加权长度归一化项。
- **DeepSeek-R1**——标志性的公开 RLVR 配方：
- *R1-zero*（受控设定）：底座 DeepSeek-V3，用 GRPO，奖励为 **准确率奖励 + 格式奖励**（使用思考标签）。涌现现象：CoT 变长、"aha moment"——不过后续分析（Dr. GRPO）认为长度增长部分来自有偏目标，且底座模型本就具备"aha"行为。
- *R1* 增加了：**SFT 冷启动**（长 CoT 初始化：约 1000 道数学/科学题配 Gemini/R1 的长 CoT——"即使样本很少，也足以自举出推理能力"）、**语言一致性奖励**（RL 天然会让语言混杂）、第二阶段使用不可验证奖励（用 V3 做裁判，60 万条），随后是常规 SFT/RLHF 后训练（20 万条非推理 SFT + R1-zero 式 RLHF）。
- *不用 PRM、不用 MCTS*：R1 "终结了关于 MCTS/PRM 必要性的猜测"（过程奖励模型与蒙特卡洛树搜索试过，并不需要）。
- *蒸馏*：R1 生成 80 万条 CoT 轨迹 → 蒸馏进 Qwen 2.5——小模型也能推理。
- **Kimi K1.5**：
- *数据构造*：数学类语料按主题平衡；剔除多选/判断题（假阳性）；只保留模型 best-of-8 失败的样本（难度筛选）。

- *RL*：基于参考的奖励模型；用类 DPO 推导（非参数假设 + 解出 r）；用平方损失作代理；带正则的 baseline 策略梯度。
- *长度控制*：每批次的长度奖励——$\lambda \in [-0.5, 0.5]$；答对的被激励更短；答错的被激励短于组内中心；并且在训练后期才启用。
- *课程学习*：给数据打难度标签、由易到难；按 (1 − 成功率) 采样以避免重复已解出的题。
- *奖励*：代码——有标准解的题目 + 自动生成新测试用例；数学——用 80 万条样本训练 CoT 奖励模型做答案等价性校验。
- *RL 基础设施*：同策略 rollout 即（慢速）推理；训练与推理框架切换；长 CoT 造成批次不均——利用率是一等问题。
- **Qwen 3**：SFT + 推理 RL，GRPO 只用了 **3995 条**样本（低数据量 RLVR！）；难度筛选（best-of-n、剔除不用 CoT 也能做对的题、剔除与验证集相似的题）；**思考模式融合（thinking-mode fusion）**——把非思考与思考数据用标签混合，并用特殊字符串提前终止；随后再做通用 RLHF（数学/STEM 能力会略有下降）。**Qwen 3 Coder Next**：中训练（GitHub、600B 长上下文"仓库级"token、PR + RAG 检索仓库状态、合成代码问答、agent 轨迹）+ 专家模型（web dev、UX、QA、SWE）+ 智能体 RL（80 万个自动构建的 SWE-bench 式环境）。
- **整体图景**：SFT + 推理 RL（GRPO）→（可选蒸馏）→ 通用 RLHF——RLHF 排在推理 RL **之后**。

代码示例：GRPO 损失（作业 5 必修部分的核心）

代码（Python）：

```
import torch
import torch.nn.functional as F

def grpo_loss(log_probs, old_log_probs, rewards, kl, beta=0.01, clip_eps=0.2):
    """带组内归一化优势的 GRPO。
    log_probs/old_log_probs: [G, T]，一个提示组内（G 条回答、T 个 token）
    rewards: [G] 每条回答的标量奖励
    kl: [G, T] 相对参考策略的逐 token KL
    """
    # 优势：组内 z-score（注意：不是无偏的——见第 16 讲）
    adv = (rewards - rewards.mean()) / (rewards.std() + 1e-4)

    ratio = torch.exp(log_probs - old_log_probs)          # [G, T]
    clipped = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps)
    # token 级损失：min(ratio, clip(ratio)) * adv，加 KL 惩罚
    per_token = -torch.min(ratio, clipped) * adv.unsqueeze(1) + beta * kl
    return per_token.mean()                                # + 长度归一化
```

代码做了什么： 实现 GRPO 目标：组内归一化优势（均值/标准差）、逐 token 的裁剪重要性比、以及对参考策略的逐 token KL 惩罚——即讲义提到的完整"tiny GRPO"（nano-aha-moment 风格）。

实现深挖： – **为什么要组内归一化**：每个提示有 G 条 rollout，组均值是合法 baseline（降低方差、无偏）；除以标准差能跨提示统一奖励尺度，但会**使梯度有偏**（讲义中的 RL 绕路——无偏版本是 leave-one-out REINFORCE）。作业 5 正是要求你实现两者并做对比！ – **为什么要裁剪**：与 PPO 同理——给重要性比设上

界；作业会探索**不裁剪**与**裁剪**两种变体。 – **为什么要 KL 项**：让策略贴近参考策略（SFT 模型），防止奖励被薅与模型崩塌；作业还加入了长度归一化项（按 1/|y| 重加权），因为原始 GRPO 偏好更长的回答。 – **为什么标准差里要加 1e-4**：当组内奖励全部相同时（std = 0）的稳定因子——实践中的真实坑。

与作业的联系：作业 5 的必修部分：(1) 在数学数据上做 zero/few-shot 与 CoT prompting 基线；(2) 端到端实现 **GRPO**（vLLM rollout + 策略梯度更新 + 指标）；(3) 实现**策略梯度估计器变体**——baseline 选择（组均值 vs 其他）、重要性权重裁剪、长度归一化。这个代码块就是这三项任务的数学核心。

代码示例：同策略 rollout 循环（RL 基础设施模式）

代码（Python）：

```
def train_step(policy, ref_policy, vllm_server, prompts, tokenizer, g=8):
    # 1. Rollout：从当前策略采样每提示 G 条回答（同策略！）
    responses, response_masks = vllm_server.sample(prompts, n=g)
    # 2. 打分：可验证奖励（精确匹配 / 单元测试）
    rewards = score(responses, answers)                # [num_prompts, G]
    # 3. 计算当前策略与参考策略下的对数概率
    log_probs = policy.log_probs(prompts, responses)           # [P, G, T]
    old_log_probs = policy.log_probs.detach().clone()           # 更新前先存好
    ref_log_probs = ref_policy.log_probs(prompts, responses)    # KL 锚点
    kl = log_probs - ref_log_probs
    # 4. 对 GRPO 损失做一次（或多次）优化器更新
    loss = grpo_loss(log_probs, old_log_probs, rewards, kl)
    loss.backward(); optimizer.step()
```

代码做了什么： 勾画 RLVR 的训练步：从在线策略采样（同策略）、用可验证奖励打分、计算当前/旧/参考策略的对数概率，并优化 GRPO 损失——用梯度累积跨 micro-batch 以节省显存。

实现深挖： – **为什么必须同策略**：rollout 必须来自**当前**策略；这正是 RL"同策略"的含义，也是作业若每个推理批次跑 32 步训练就变成"32× 离策略"的原因——重要性比要用来修正漂移。 – **为什么用 vLLM**：rollout 就是规模化推理；作业里的 `VLLMServer`（带 CoT 停止字符串）就是工程实现。长 CoT 造成的批次不均（见 Kimi 部分）是真实的基建问题。 – **为什么需要参考策略的对数概率**：KL 锚点；每个批次只算一次（冻结参考）是标准的高效做法。

与作业的联系：作业 5 的完整 GRPO 训练循环 = 这段草图：`VLLMCompletion` / `VLLMServer` 接口、`response_mask` 处理、checkpoint 保存/加载（`get_model_and_tokenizer`）、指标（奖励均值、KL、回答长度）以及梯度累积。"GRPO 变体"任务（不裁剪/裁剪、baseline、长度归一化）就是作业的估计器部分。

关键要点

1. RLVR 是 RLHF 在"奖励可验证"领域的可扩展继任者——它优化你真正想要的东西，回避了奖励模型的过度优化。
2. GRPO = PPO 减去价值函数：组内归一化优势 + 裁剪比 + KL 惩罚；简单到约 20 行就能写出来——但它的标准差归一化理论上是有偏的（无偏版本约等于 leave-one-out REINFORCE），并且存在长度偏差。

3. DeepSeek-R1 的配方：冷启动 SFT（约 1000 条长 CoT 样本）→ 用准确率 + 格式（+ 语言一致性）奖励做 GRPO → 通用 RLHF；不需要 PRM/MCTS；蒸馏可以把推理能力迁移到小模型。
4. Kimi K1.5 展示了工程面：难度筛选、课程学习、长度控制奖励以及繁重的 RL 基础设施（rollout 效率、批次不均）。
5. Qwen 3 证明低数据量 RLVR 可行（约 4000 条样本上跑 GRPO），并展示了现代配方：SFT → 推理 RL → 通用 RLHF，配合思考模式控制与智能体中训练。

常见陷阱

- **把标准差归一化后的优势当成无偏**：组均值 baseline 没问题；除以标准差会使估计有偏——作业"GRPO vs 无偏变体"的对比正是为此。
- **长度偏差**：原始 GRPO（以及 RLHF 整体）会拉长回答；使用长度归一化目标（以及 Kimi 式的长度奖励，且在后期启用）。
- **格式奖励被薅**：R1 的格式奖励（思考标签）可被钻空子——要验证内容而不只是标签。
- **用陈旧策略做 rollout**：过度离策略且不做修正（重要性比/裁剪）会让估计器退化；作业中 32× 离策略的步数必须配合裁剪变体。
- **忽视基础设施**：规模化的 RL 是推理受限的；长 CoT 导致批次不均会浪费 GPU——要仔细做 padding/调度。
- **跳过 SFT 冷启动**：R1-zero 能工作，但 SFT 初始化（少量长 CoT 数据）会显著提升稳定性与质量。

复习题

1. **问：** GRPO 从 PPO 中去掉了什么？被去掉的部分由什么替代？
2. **答：** GRPO 去掉了价值（critic）网络及其优势估计。取而代之的是按提示组计算优势：（奖励 − 组均值）/组标准差——即 G 条采样回答内的 z-score——因此不需要训练价值模型。
3. **问：** 为什么 GRPO 的标准差归一化在理论上存在问题？无偏替代方案是什么？
4. **答：** 减去组均值是合法（无偏）的 baseline，但除以组标准差会改变估计量的期望——不是合法 baseline。无偏变体（Liu 等 2025）接近于带 leave-one-out baseline 的 REINFORCE（并对长度归一化项做了修正）。
5. **问：** 为什么 R1 不需要 MCTS 或过程奖励模型（PRM）？
6. **答：** R1 用 GRPO + 结果级可验证奖励（准确率 + 格式）就得到了强推理能力（超过 o1），无需搜索或逐步监督；R1 论文的"不成功尝试"一节记录了 PRM 与 MCTS 被尝试过但并非必需——从而大幅简化了配方。

第 17 讲：对齐 —— 多模态

日期：5 月 27 日（周三，Spring 2026）| 讲师：Percy Liang | 材料： `lecture_17.py` *（第 18–19 讲为客座讲座：Daniel Selsam、Dan Fu，无公开材料）*

概览

世界是多模态的；终极目标是**全能模型（omni model）**——能输入与输出任意模态组合。由于 Transformer "只会说 token"，一切都要被转换成 token。本讲覆盖如何**输入**非文本数据（CLIP/SigLIP 对比学习编码器、ViT）、如何把图像编码**注入** LLM（LLaVA 及其后继、Qwen-VL 1/2/3），以及迈向**生成**的一步——Chameleon 的"全离散（VQ-VAE）token"方案，以及混合文本/图像自回归建模的训练稳定性挑战。

核心概念与定义

- **两个问题**：(1) 如何**输入**非文本数据（理解图像/音频/视频）；(2) 如何**输出**非文本数据（生成图像/音频）。理解与生成所需要的表征可能不同（语义 vs 精细细节）。
- *类比*：描述一张照片（理解）只需要抓住大意；把它重画出来（生成）要求像素级还原。同一种编码很难同时满足两者。
- **CLIP（Contrastive Language-Image Pretraining）**：同时训练图像编码器与文本编码器，使配对的 (图像, 文本) 相似度高、不配对的相似度低——在约 32K 的批次上做批级排序目标。
- *数据*：从网络抓取的 4 亿对 (图像, 描述)（未公开；OpenCLIP 用 LAION-5B 复现）。
- *视觉编码器*：ViT（或 ResNet）；最佳配置 ViT-L/14@336px；attention pooling；图像缩放（短边 336）+ 中心裁剪。
- *文本编码器*：GPT-2 风格 Transformer（63M）；编码 [BOS]…[EOS]，取最高层的 [EOS] 激活。
- *标志性结果*：zero-shot CLIP 在 ImageNet 上超过用 120 万张 ImageNet 图训练的 ResNet-50。消融显示：直接"由图预测文本"的计算效率远低于 CLIP 式排序。
- *类比*：CLIP 像一个学生，通过把成千上万张照片与说明文字配对来学会"猫是什么"——没人告诉它"这是猫"，但描述的共现教会了它这个概念。
- **SigLIP**：思路相同，但把批级 softmax 换成**逐对二分类**（是否配对）——从而把 batch size 与损失解耦，在 <16K 的 batch 下表现更好；32 块 TPUv4 训 5 天 vs CLIP 的 256 块 TPUv3 训 10 天。数据：WebLI（十亿级网络图文对，OCR 过滤，100 种语言）。
- **LLaVA（Large Language and Vision Assistant）**：标准的 VLM 模板——**视觉编码器（CLIP）+ 投影器 + LLM（Vicuna）**。
- *数据*：基于 MS-COCO 图像、由 GPT-4 生成的 15.8 万条对话（把描述/检测框变成问题与对话）。
- *训练*：阶段 1（对齐）：冻结视觉编码器与 LLM，只训练投影器 W；阶段 2（微调）：冻结视觉编码器，训练 W 与 LLM。
- *AnyRes*（LLaVA 1.5+）：按编码器原生分辨率把图像切成 a×b 块、逐块编码后拼接，以保留高分辨率（对 OCR 至关重要）。
- **LLaVA-OneVision**：SigLIP 编码器（取最后一层 Transformer 前后的网格特征）、Qwen-2-72B 解码器、2 层 MLP 投影器；通过为每种模态调分辨率使单图/多图/视频产生大致相同的 token 数（"质量优先于数量"、"由易到难"训练）；具备跨模态迁移（OCR → GUI 智能体、视觉提示 → 视频）。
- **Qwen-VL**：OpenCLIP ViT-bigC + 1 层交叉注意力适配器（带 2D 位置编码，固定 256 长度）+ 特殊 token（`<img>`、`<box>`、`<ref>`）；三阶段训练（冻结 LM 做对齐 → 全参数在高分辨率任务数据上训

练 → 冻结编码器做指令微调）。

- **Qwen2-VL**：更大的 ViT（675M）；**动态分辨率**（224×224 块，2×2 压缩 → 66 个 token）；视频 2 fps、最多 16384 个 token；**MRoPE**（多模态 RoPE：时间/宽/高三轴）；三阶段训练。
- **Qwen3-VL**：SigLIP-2 编码器 + **交错式 MRoPE**（[t w h t w h…] 而不是 [t t t w w w h h h]）+ 显式视频时间戳；**按 token 的平方根归一化损失**以平衡文本与长视频序列；**DeepStack** 跨层适配器（把视觉信息注入多层）；4 阶段预训练（适配器 → 在 8K/32K/256K 长度上全参数训练）+ 长 CoT SFT + 蒸馏 + RL。SOTA，但"做了大量数据工作，细节不多"。
- **Chameleon（迈向全能）**：把**一切**映射为离散 token：图像经 **VQ-VAE**（512×512 图像 → 1024 个 token，码本大小 8192；再解码回去、最小化重建损失），然后用同一个自回归 Transformer 训练混合的文本/图像 token 流——从而以统一方式既能分析又能生成。
- *训练*：阶段 1 占 80%，无监督（2.9T 文本 token + 1.5T 图文 token + 4000 亿交错 token）；阶段 2 占 20%，高质量混合数据。
- *稳定性*：文本 token 熵低、图像 token 熵高 → 导致范数增长与 **logit 漂移**；修法：**QK-norm** 与 **z-loss**（来自第 3 讲）。
- *取舍*：优雅（纯下一 token 预测）但性能较弱——离散化丢失信息（OCR 明显吃亏）。
- **混合模态训练稳定性**：平衡图像/视频（信息密度低）与文本；按 token 的损失归一化（Qwen3-VL 的平方根技巧）可防止视频序列主导训练。

代码示例：对比损失（CLIP 风格）——概念实现

代码（Python）：

```
import torch
import torch.nn.functional as F

def clip_loss(image_embeds, text_embeds, temperature=0.07):
    """对比损失：让每张图与其描述相互对齐（双向）。
    image_embeds, text_embeds: [batch, d]（已做 L2 归一化）
    """
    image_embeds = F.normalize(image_embeds, dim=-1)
    text_embeds = F.normalize(text_embeds, dim=-1)
    logits = image_embeds @ text_embeds.T / temperature      # [B, B]
    labels = torch.arange(logits.shape[0], device=logits.device)
    loss_i = F.cross_entropy(logits, labels)                 # 图 -> 文
    loss_t = F.cross_entropy(logits.T, labels)               # 文 -> 图
    return (loss_i + loss_t) / 2
```

代码做了什么： 在一个批次内构造图像与描述的 B×B 相似度矩阵，用对称交叉熵训练"每张图更偏好自己的描述而非其他描述"（反之亦然）——即 CLIP 目标（SigLIP 则用逐对 sigmoid 的二分类损失替代它）。

实现深挖： – **为什么对称（双向）**：匹配必须是双向的——图像检索文本、文本检索图像；这也把训练信号翻倍。 – **为什么需要大 batch**：对比损失的难度（负样本数量）随 batch 增大而增大——CLIP 用了约 32K；SigLIP 的二分类改写把 batch size 与损失解耦（这是它效率的关键）。 – **为什么有 temperature**：把 logits

缩放到交叉熵可用的范围（实践中可学习）。 – **为什么要归一化**：余弦相似度（L2 归一化）避免嵌入模长带来的伪影。

与作业的联系：作业不直接实现它，但**架构模式**（编码器 + 投影 + LLM）与**评测思维**（第 12 讲的基准，含多模态 HLE）都适用。作业 4 的质量分类器训练也用了类似对比的思想（正负样本对 → fastText）。

代码示例：LLaVA 式视觉语言模型（架构草图）

代码（Python）：

```
import torch
from torch import nn

class LLaVA(nn.Module):
    def __init__(self, vision_encoder, projector, llm):
        super().__init__()
        self.vision_encoder = vision_encoder   # 冻结的 CLIP/SigLIP ViT
        self.projector = projector             # 例如 2 层 MLP（或线性层 W）
        self.llm = llm                          # 语言模型

    def forward(self, images, input_ids, pixel_values=None):
        # 1. 编码图像 -> patch 嵌入（或网格特征）
        img_feats = self.vision_encoder(images)          # [B, num_patches, d_vit]
        # 2. 投影到 LLM 的嵌入空间
        img_tokens = self.projector(img_feats)           # [B, num_patches, d_model]
        # 3. 把图像 token 与文本 token 交错，走标准 LM 前向
        #    （例如把 <image> token 放进序列，配合因果掩码）
        return self.llm(input_ids=input_ids, img_tokens=img_tokens)
```

代码做了什么： 勾画标准 VLM：冻结的视觉编码器产出 patch 特征，投影器把它们映射进 LLM 的嵌入空间，LLM 消费交错的图像/文本 token 流。

实现深挖： – **为什么要冻结视觉编码器**：LLaVA 阶段 1（对齐）冻结编码器与 LM、只训练投影器——便宜且稳定；阶段 2 再解冻 LM。Qwen-VL 的阶段 1 反过来冻结 LM、训练编码器 + 适配器——"冻结哪一部分"是反复出现的设计决策。 – **为什么投影器重要**：线性投影（LLaVA）最简；2 层 MLP（OneVision）与交叉注意力适配器（Qwen-VL）在容量与 token 数之间做权衡。投影器负责把 d_vit → d_model。 – **为什么分辨率处理（AnyRes）必不可少**：CLIP 式的缩放+裁剪会毁掉精细信息（OCR、图表）；按原生分辨率切块能保留细节——代价是更多 token（因此 OneVision 要为每种模态单独调分辨率）。

与作业的联系：这是现代开源 VLM 的架构谱系；虽然作业 1–5 都是纯文本，但**分词**那一讲的原则（第 1 讲："把一切转成 token"）正是它向图像（Chameleon 的 VQ-VAE）的延伸，而训练稳定性技巧（QK-norm、z-loss）你在作业 1 的模型实现中也会再次遇到。

关键要点

1. 全能模型的目标：任意输入 → 任意输出；今天的答案是"把一切转成 token"（文本与 Chameleon 图像用离散 token，CLIP 式注入用连续嵌入）。

2. CLIP/SigLIP 通过对比学习给出与文本对齐的图像编码器；SigLIP 的二分类损失效率高得多（与 batch size 解耦）。
3. 标准 VLM 配方：冻结/半冻结的视觉编码器 + 投影器 + LLM，配合分阶段训练（对齐 → 微调 → 指令微调）与保分辨率技巧（AnyRes、动态分辨率、MRoPE）。
4. 生成需要离散或连续的**输出** token：Chameleon 的 VQ-VAE 方案优雅但丢失细节；扩散模型是高保真生成的另一条路。
5. 混合模态训练并不稳定（logit 漂移、范数增长）：QK-norm、z-loss 与逐 token 损失归一化是实用解法。

常见陷阱

- **破坏分辨率**：缩放到 336×336 再裁剪会毁掉 OCR/阅读类任务；要用 AnyRes/动态分辨率流程。
- **模态不平衡**：长视频序列会主导损失；要按 token 归一化（平方根技巧）或平衡数据。
- **混合训练不稳定**：图像 token 熵高；没有 QK-norm/z-loss 会出现 logit 漂移与范数增长。
- **冻结策略搞错**：忘记各阶段冻结的是编码器、LM 还是投影器，会破坏分阶段训练配方。
- **离散 token 的信息损失**：VQ-VAE 离散化会损害细粒度任务（OCR）；选择 Chameleon 式建模前要清楚这个取舍。
- **位置编码冲突**：RoPE 遇到视频/多图轴需要 MRoPE/交错方案；朴素位置编码无法处理三维（时间、高、宽）token 网格。

复习题

1. **问：** 为什么 SigLIP 在同质量下比 CLIP 便宜这么多?
2. **答：** CLIP 的损失是批次内 B×B 的 softmax（对所有配对做对比），需要巨大 batch 才有足够负样本。SigLIP 把每个 (图像, 文本) 对当作独立的二分类（sigmoid），把 batch size 与损失解耦——因此用少得多的 TPU 就能训好（32 块 TPUv4 训 5 天 vs CLIP 的 256 块 TPUv3 训 10 天）。
3. **问：** LLaVA 训练的各个阶段分别冻结了什么?
4. **答：** 阶段 1（特征对齐）：冻结视觉编码器与 LLM，只训练投影器 W。阶段 2（端到端微调）：冻结视觉编码器，训练投影器 + LLM（在指令数据上）。（Qwen-VL 的变体是先冻结 LM、训练编码器 + 适配器。）
5. **问：** 为什么 Chameleon 需要 VQ-VAE? 它的主要弱点是什么?
6. **答：** 为了让图像能被与文本相同的自回归 Transformer **生成**，图像必须变成离散 token——VQ-VAE 用重建损失把图像 patch 映射为码本索引（512×512 → 1024 个 token）。弱点是离散化丢失细粒度信息（例如 OCR 细节），使它在理解类任务上不如连续编码器方案。

作业汇总

五次作业都在 **PyTorch 中、以极少脚手架**完成：仓库提供单元测试与 adapter 接口用于验证正确性，但所有实现都要你自己写。先在本地 CPU 上调正确性，再上 GPU（选课学生由 Modal 赞助算力）做训练与

benchmark。截止日期见课程日程表。通过 Gradescope 提交；部分作业设有排行榜（在固定预算下最小化困惑度）。**AI 政策**（2025–26）禁止用 AI 实现作业的任何部分——只可用于概念提问与 API 文档查询，并需把仓库的 `AGENTS.md` 粘贴到聊天对话开头。

作业 1：基础（分词、模型、训练）— `assignment1-basics`

目标： 实现训练一个标准 Transformer 语言模型所需的全部组件，并真正训练出一个最小模型（TinyStories，然后 OpenWebText）。

主要实现任务（摘自 handout）： 1. **BPE 分词器**（第 2 节）：在语料上学习 merge（词表初始化、用 GPT-2 风格 regex 做预分词、merge 计算、special token 如 `<|endoftext|>`、并行预分词、优化合并步骤）；保证 encode/decode 往返正确。 2. **Transformer 语言模型**（第 3 节）：token embedding、**RMSNorm**、**RoPE**、**因果多头自注意力**（QKV 投影、因果掩码、softmax）、逐位置 **SwiGLU 前馈**、pre-norm 块、LM head → 下一 token 概率；handout 明确了各维度约定（B、S、D 等）。 3. **交叉熵损失 + AdamW 优化器**（第 4 节）：标准 NLL 损失，以及作为 `torch.optim.Optimizer` 子类实现的 AdamW（一阶/二阶矩、权重衰减、学习率调度；handout 中给出了 SGD 的完整示例）。 4. **训练循环**（第 5 节）：checkpoint 保存/加载（序列化模型 + 优化器状态）、训练配置（batch size、LR 等）、解码支持（贪心/采样）、困惑度评估。 5. **资源核算**：给定配置下 Transformer 各组件的 FLOPs 与显存。 6. **运行**：先在 TinyStories 上训练小模型（handout 有样例输出），再换到 OpenWebText。 7. **排行榜**：在 B200 上 45 分钟内最小化 OpenWebText 困惑度（讲义给出了去年的排行榜链接）。

最关键的讲座内容： 第 1 讲（分词/BPE——直接蓝图）、第 2 讲（资源核算、Adam 系优化器、训练循环）、第 3 讲（现代架构变体：pre-norm、RMSNorm、RoPE、SwiGLU、无 bias）。

作业 2：系统（Profiling、Kernel、分布式）— `assignment2-systems`

目标： 用高级工具对作业 1 的模型做 profiling 与 benchmark；用自己的 Triton 实现 FlashAttention-2 优化 attention；构建显存高效、可扩展的分布式训练。

主要实现任务（摘自 handout）： 1. **Benchmark + profiling 框架**：测量各算子的运行时间与显存；使用 Nsight Compute 与 NVTX range；回答"前向+反向哪个 kernel 最耗时"这类问题；混合精度（autocast）数据类型相关问题。 2. **激活重计算**：为 TransformerBlock 包装重计算；用 `saved_tensors_hooks`（pack/unpack hook）验证显存节省。 3. **FlashAttention-2 Triton kernel**：前向（分块 QK^T、在线 softmax、带掩码写回）与反向（重算打分矩阵），用 `torch.autograd.Function` 承载；handout 附有 WeightedSum 的 autograd 教学示例与精度容差指南。 4. **分布式数据并行训练**：梯度 all-reduce（反向 hook、异步通信），与单卡对比 benchmark。 5. **优化器状态分片**：梯度 reduce-scatter、本地 AdamW 更新、参数 all-gather（ZeRO-1）。 6. **全分片数据并行（FSDP）**：连参数也分片，前向/反向按需 all-gather、梯度 reduce-scatter；验证显存扩展；训练一个原本装不下的模型。 7. **排行榜**：提交你的最佳结果。

最关键的讲座内容： 第 5 讲（GPU 模型、roofline）、第 6 讲（Triton kernel：分块、在线 softmax、融合——四类 kernel 模式）、第 7 讲（集合通信、DDP、带宽测量）、第 8 讲（ZeRO 1–3/FSDP 的显存与通信核算、重叠）。

作业 3：扩展（Scaling Laws）— `assignment3-scaling`

目标： 拟合扩展律，为一次大规模训练（48 B200 小时）预测算力最优配置，而实验预算只有 12 B200 小时。

主要实现任务（摘自 handout）： 1. 调用**训练 API**（HTTP 端点、`X-API-Key` 头）：提交带超参数（层数、embedding 大小、head 数、batch size、学习率、训练 token 数）与最长墙钟时间的实验；轮询状态；获取验证损失。 2. 设计搜索空间策略：IsoFLOPs 剖面（Chinchilla 方法 2）——对每个算力预算，以 D = C/6N 扫模型规模；或联合拟合（方法 3）、Kaplan 风格、muP 相关思路（handout 欢迎）。 3. 拟合幂律 N_opt(C) 与 D_opt(C)；**外推**到 48 B200 小时的预算；提交预测的算力最优超参数与预测最终验证损失。 4. 报告：完整方法论，需详细到可复现；部分成绩取决于你预测模型的**实测**性能。

最关键的讲座内容： 第 9 讲（扩展律基础、IsoFLOPs 流程、6ND 法则、幂律拟合、Chinchilla vs Kaplan）、第 11 讲（案例研究：用 WSD 重启让拟合变便宜、DeepSeek/MiniCPM 配方、muP、临界 batch/LR 的扩展）。

作业 4：数据（数据加工流水线）— `assignment4-data`

目标： 把原始 Common Crawl 转成可用的预训练数据，涵盖转换、过滤、去重；并衡量对模型质量的影响。

主要实现任务（摘自 handout）： 1. **Common Crawl HTML 转文本**（WARC/WET 文件）：用 trafilatura/resiliparse 解析；对比 WET 与 WARC；用 `concurrent.futures` 并行处理。 2. **过滤**：(a) 基于规则的过滤（Gopher/C4 标准——例如词数太少、字母内容比例过低、boilerplate、非英语的文档要删）；(b) **有害内容分类器**（提供的 NSFW 与仇恨言论分类器）；(c) **PII 移除**（邮箱、IP 地址）；(d) 可选的额外过滤。 3. **去重**：先精确哈希，再用 **MinHash** 做近重复检测（含 Jaccard 相似度与 LSH banding）；必要时用你作业 1 的 BPE 分词器做分词。 4. **训练**：构建 token 化数据集，在"过滤后 vs 未过滤"数据上训练小 LM（handout 提供了分词与用保存好的 token ID 训练的起始代码）。 5. **排行榜**：在给定 token 预算下最小化困惑度。

最关键的讲座内容： 第 13 讲（来源：Common Crawl 的 WARC/WET、版权、数据集谱系、质量分类器）、第 14 讲（完整流水线：转换；过滤框架——目标 vs 原始数据、KenLM/fastText 分类器、毒性过滤；去重——MurmurHash、MinHash、LSH 阈值与 (b, r) 数学；混合——epoch 陷阱、UniMax、模拟 epoch）、第 12 讲（以困惑度作为评测指标）。

作业 5：对齐与推理强化学习（GRPO）— `assignment5-alignment`

目标： 通过监督式提示基线 + 强化学习（GRPO）训练语言模型解数学题，并探索策略梯度估计器变体。*可选第二部分：* 用 SFT + DPO 做指令跟随与安全对齐。

主要实现任务（必修，摘自 handout）： 1. **Zero-shot、few-shot 与思维链（CoT）提示**基线（在数学评测数据上）。 2. **GRPO 实现**：vLLM rollout 服务（`VLLMCompletion` / `VLLMServer`、停止字符串）、带 response mask 的分词、当前/旧/参考策略下的对数概率、组内归一化优势、含 KL 惩罚 + 长度归一化的裁

剪比损失、梯度累积、带指标的完整训练循环（奖励、KL、回答长度）。 3. **策略梯度估计器变体**：baseline 选择（组均值 vs 其他）、重要性权重裁剪（不裁剪 vs 裁剪）、长度归一化重加权——分析方差/期望；handout 推导了"未裁剪的 token 级重加权"估计器及其裁剪版本。 4. 在小模型（如 OLMo-2-0425-1B）上运行，每个推理批次做约 32 步离策略训练；测量准确率提升。

可选第二部分（补充——SFT + DPO）： 1. 在 MMLU、GSM8K、AlpacaEval、SimpleSafetyTests、Anthropic HH 上做 zero-shot 基线。 2. 在指令-回答数据上对 Llama 3.1 8B 做**监督微调**。 3. 在成对偏好数据（Anthropic HH）上做**直接偏好优化（DPO）**，用 Llama 3.3 70B 作裁判评测 AlpacaEval 与安全性。

最关键的讲座内容： 第 15 讲（SFT 数据、RLHF 目标、PPO vs DPO、过度优化/模式崩塌）、第 16 讲（GRPO——你要实现的正是这个算法，及其偏差与变体；R1/Kimi/Qwen3 配方；RL 基础设施）、第 10 讲（推理基础设施——vLLM、KV cache、生成）、第 12 讲（补充部分使用的评测基准）。

作业与讲座对应表

作业	主要相关讲座	核心技能
1 — 基础	1、2、3	BPE、Transformer 模块、AdamW、训练循环、资源核算
2 — 系统	5、6、7、8	profiling、Triton kernel、FlashAttention-2、DDP/ZeRO/FSDP
3 — 扩展	9、11	扩展律拟合、IsoFLOPs、外推与预测
4 — 数据	12、13、14	HTML→文本、分类器过滤、MinHash/LSH 去重、混合、PPL 评测
5 — 对齐	10、15、16	prompting、GRPO、策略梯度变体、（可选 DPO/SFT）

术语表（Glossary）

- **Activation checkpointing（激活重计算）**——反向时重算前向激活而不保存；用算力换显存（亦称 gradient checkpointing / rematerialization）。
- **Adam / AdamW**——自适应优化器，维护逐参数的一阶/二阶矩（AdamW 加解耦权重衰减）；LLM 的标准优化器（fp32 下 8 字节/参数）。
- **All-reduce / reduce-scatter / all-gather（集合通信）**——跨设备求和并复制；求和并分片；把分片汇聚到所有设备。all-reduce = reduce-scatter + all-gather。
- **Arithmetic intensity（算术强度）**——每搬运一字节所做的 FLOPs；与硬件的加速器强度（峰值 FLOP/s ÷ 带宽）比较，判断是 memory-bound 还是 compute-bound。
- **AWQ（激活感知权重量化）**——训练后量化，依据激活幅度把少数（被大激活通道命中的）权重保留更高精度。
- **Bank conflict（bank 冲突）**——warp 内多线程访问同一 shared memory bank 导致的串行化；用 swizzling/padding 缓解。

- **bf16 / fp16 / fp8 / fp4**——浮点格式：bf16 是 2 字节但动态范围同 fp32；fp16 范围小（有下溢风险）；fp8（E4M3/E5M2）与 NVFP4 用于低精度计算/推理。
- **BPE（字节对编码）**——子词分词器：从字节出发，反复合并最高频相邻对直到达到目标词表大小。
- **Chinchilla 扩展律**——算力最优扩展：N_opt ∝ C^0.5、D_opt ∝ C^0.5，约每参数 20 token（对比 Kaplan 的 0.73/0.27）。
- **Collective operation（集合操作）**——所有设备参与的通信模式（broadcast、scatter、gather、reduce 及其 "all-" 变体）。
- **Compute-bound vs memory-bound**——计算时间还是访存时间占主导（由算术强度与加速器强度之比决定）。
- **Continuous batching（连续批处理）**——迭代级调度，在每个解码步加入/移除请求，而非静态批次。
- **DDP（分布式数据并行）**——复制参数、切分 batch、all-reduce 梯度。
- **DPO（直接偏好优化）**——无需奖励模型的 RLHF：用偏好对分类损失优化隐含奖励 β·log(π/π_ref)。
- **Deduplication（去重）**——去除精确与近重复文档（精确哈希；近重复用 MinHash + LSH）以省算力并避免记忆。
- **Fair use（合理使用）**——版权例外原则（四要素：目的、作品性质、使用量、市场影响）；部分 LLM 训练在具体案件中被判合法。
- **FlashAttention**——融合、分块的 attention，配在线（telescoping）softmax；不物化 S/P 矩阵；block 级显存 O(1)。
- **FLOPs 与 FLOP/s**——浮点运算次数（工作量）与每秒浮点运算次数（速度）；MFU = 实际/峰值。
- **FSDP / ZeRO-3**——分片参数、梯度与优化器状态；按需 all-gather 参数、reduce-scatter 梯度；通信约 3×#params。
- **GQA（分组查询注意力）**——KV 头少于 query 头以缩小 KV cache；MQA 即 1 个 KV 头。
- **GRPO（组相对策略优化）**——去掉价值模型的 PPO：优势 = (奖励 − 组均值)/组标准差（每提示 G 条 rollout）；已知有标准差归一化偏差与长度偏差。
- **IsoFLOPs**——扩展律方法：固定算力 C_i、扫模型规模、取最小损失；拟合 N_opt(C)、D_opt(C) 的幂律。
- **KV cache**——缓存 key/value 向量，使每 token 生成为 O(1) 成本（而非 O(T) 重算）；其大小决定生成阶段的访存受限程度。
- **Linear attention（线性注意力）**——核为恒等的 attention：$Q(K^TV) = (QK^T)V$，序列长度线性；递推形式 $S_t = S_{t-1} + k_t v_t^T$（对偶性）。
- **LSH（局部敏感哈希）**——b 个 band、每 band r 个 min-hash；碰撞概率 $1-(1-s^r)^b$ 是 S 形曲线，阈值在 $(1/b)^{(1/r)}$。
- **Memory coalescing（显存合并访问）**——warp 的访问合并为 128 字节事务；快速 HBM 读取的前提。
- **MFU（模型浮点利用率）**——实际 FLOP/s ÷ 峰值 FLOP/s；≥0.5 已算不错。
- **MinHash**——满足 Pr[h(A)=h(B)] = Jaccard(A,B) 的哈希方案；即集合在随机哈希下的最小哈希值。

- **MLA（多头隐层注意力）**——把 K/V 压成低维隐向量 c（DeepSeek v2），缩小 KV cache；需额外保留非旋转维度以兼容 RoPE。
- **MoE（混合专家）**——多个专家 FFN + 路由器；在每 token FLOPs 不变的前提下增加参数；用 top-k 路由 + 均衡损失训练。
- **muP（最大更新参数化）**——宽度感知的初始化 + LR 缩放，使最优超参数可跨规模迁移；会被 RMSNorm 增益/强 weight decay 破坏。
- **PagedAttention**——给 KV cache 做虚拟内存式分页（vLLM）：不连续 block、前缀共享、写时复制。
- **Perplexity（困惑度）**——exp(平均 NLL)，即 (1/p(D))^(1/|D|)；越低越好；随机猜测时约等于词表大小。
- **Prefill 与 decode**——推理的两个阶段：并行处理 prompt（compute-bound）与一次生成一个 token（memory-bound）。
- **PPO（近端策略优化）**——带裁剪重要性比、价值模型与 KL 控制的 RL 算法；经典 RLHF 优化器。
- **QK-norm / z-loss**——稳定性技巧：在 softmax 前归一化 Q、K；惩罚 log-sum-exp 以防止 logit 漂移。
- **RLHF**——基于人类（或 AI）成对反馈的强化学习，配合 KL 控制来优化偏好。
- **RLVR**——可验证奖励的 RL（精确答案、测试通过）：可扩展，避免奖励模型的过度优化。
- **RMSNorm / LayerNorm**——归一化层：RMSNorm 只按均方根缩放、不减均值也无 bias；LayerNorm 做中心化与缩放。
- **Roofline model**——以算术强度为横轴、可达 FLOP/s 为纵轴的图；拐点即加速器强度。
- **RoPE（旋转位置编码）**——按位置旋转 Q/K 的坐标对，使 attention 分数只依赖相对位置。
- **Scaling law（扩展律）**——损失与数据/模型/算力之间的幂律关系；支持小规模 → 大规模预测。
- **Speculative sampling（投机采样）**——草稿模型提议 token、目标模型并行打分；修正拒绝采样对目标模型是**精确**的。
- **SFT（监督微调）**——在指令-回答示范上训练；最擅长抽取预训练已具备的行为。
- **SwiGLU / GeGLU**——门控 FFN 激活（swish/高斯误差门 ⊙ 线性分支）；2023 年后模型的标准；FFN 维度约 8/3× 模型维度。
- **Tensor parallel（张量并行）**——把权重矩阵切分到多设备，逐层 all-gather 激活；需要 NVLink 级互联。
- **Tiling（分块）**——在共享内存 tile 上计算以减少 HBM 流量（矩阵乘、FlashAttention）；全局读取降低 T 倍。
- **Tokenizer（分词器）**——encode(文本)→token ID 与 decode(ID)→文本；标准做法是 BPE；必须往返正确并处理 UTF-8 与 special token。
- **Upcycling**——用预训练的稠密模型初始化 MoE。
- **WSD 学习率**——warmup–stable–decay 调度；可重启，便于廉价拟合扩展律。
- **ZeRO-1 / ZeRO-2**——仅分片优化器状态 / 再加上梯度；通信量与 DDP 相同而显存更优。

本笔记整理自公开的 CS336 课程网站（Spring 2026 开设；并参考 Spring 2024/2025 存档以核对日程）、可执行讲义文件、讲义幻灯片 PDF 与作业 handout。视频录播（非公开 YouTube 播放列表）、Slack，以及第 7 讲并列的 PDF（私有 GitHub 仓库）属于受限资源，已在相应位置标注。